

A Unified Approach to Bulk-Synchronous Parallel Observability Using Relational Dataflows

by
Ankush Jain

A DISSERTATION PROPOSAL
Submitted in partial fulfillment of
the requirements for the degree of

DOCTOR OF PHILOSOPHY
Electrical and Computer Engineering
Carnegie Mellon University

Committee:
Prof. George Amvrosiadis (Chair)
Prof. Greg Ganger
Prof. Andy Pavlo
Prof. Vyas Sekar
Dr. Brad Settlemyer

CARNEGIE MELLON UNIVERSITY
Pittsburgh, Pennsylvania
August 2025

Contents

Contents	i
1 Introduction	2
1.1 Motivation: Efficiency Bottlenecks in BSP	2
1.2 Our Work: A Comprehensive Study of BSP Efficiency	3
1.3 Thesis Statement	5
1.4 Document Structure	5
2 Background: Efficiency Bottlenecks in BSP Systems	6
2.1 The High-Performance Computing Environment	6
2.2 Data Management Challenges	7
2.3 Compute Efficiency Challenges	7
3 CARP: Adaptive In-Situ Range Partitioning	8
3.1 Motivation: Efficient Ingestion for Range Queries	8
3.2 CARP: System Design	8
3.3 Key Results: Ingestion and Query Performance	9
3.4 Lessons	10
4 Lessons from Optimizing Adaptive Mesh Codes	11
4.1 Motivation: Computational Efficiency in AMR Codes	11
4.2 Challenges of Telemetry-Driven Placement	11
4.3 Contribution 1: A Diagnostic Workflow	11
4.4 Contribution 2: The CPLX Placement Policy	12
4.5 Key Results	13
4.6 Lessons	13
5 ORCA: A Unified Framework for Declarative Observability	14
5.1 Motivation: Programmable and Real-time Cluster Observability	14
5.2 Background: The Observability Gap in BSP Systems	15
5.3 ORCA Architecture	15
5.4 Potential Usecases and Impact	16
6 Thesis Completion Roadmap	18
6.1 Current State	18
6.2 Evaluation Roadmap	18
6.3 Timeline	19
7 CARP Paper	19

7.1	Introduction	19
7.2	Background and Motivation	21
7.3	Range Query Challenges	22
7.4	Design Principles of CARP	23
7.5	Design of CARP	26
7.6	Implementation	32
7.7	Evaluation	33
7.8	Discussion	41
7.9	Related Work	42
7.10	Conclusion	43
8	AMR Paper	43
8.1	Introduction	44
8.2	Why Telemetry-driven Placement?	45
8.3	Challenges of Telemetry-Driven Placement	47
8.4	Denoising Telemetry For Placement	49
8.5	Designing a Placement Policy	53
8.6	Evaluation of Placement Policies	58
8.7	Lessons From Our Placement Exercise	63
8.8	Related Work	64
8.9	Conclusions	65
9	ORCA Paper	65
9.1	Introduction	65
9.2	Background and Motivation	67
9.3	Requirements for Closed-Loop Observability	70
9.4	Design	71
9.5	Implementation	76
9.6	Evaluation	78
9.7	Related Work	87
9.8	Conclusions	87
	References	88

Abstract

The Bulk Synchronous Parallel (BSP) paradigm powers our most critical computational tools, from large-scale scientific simulations to distributed machine learning. As these applications scale, their efficiency has datacenter-level impact on both costs and time-to-discovery. Yet the tightly synchronized nature of BSP creates two distinct challenges: on the data path, bursts of checkpointed state strain storage and analytics infrastructure, while on the compute path, global synchronization amplifies sensitivity to execution variability. While traditionally addressed by separate tools, this thesis demonstrates that both problems can be unified by treating them as challenges of organizing and processing streaming data for insight.

We first address these data and compute path challenges through two separate efforts. On the data path, we built CARP to enable efficient, streaming ingestion of data from large scientific applications for range queries without expensive post-processing. CARP uses reconstructed histograms to adapt to dynamic and skewed key distributions, achieving query performance comparable to a full sort with $5\times$ faster ingestion. On the compute path, we tackled low cluster efficiency in adaptive mesh codes, typically attributed to load imbalance. Our work revealed that apparent load imbalance is often confounded by environmental noise and other execution artifacts, necessitating a custom query-driven diagnostic workflow to isolate root causes. This process enabled us to establish a predictable baseline and design a tunable placement policy (CPLX) that improves runtime by up to 21.6% over this baseline.

Both our efforts revealed that efficiency gains are predicated on a common input: summarized views of distributed application state. These views are not only necessary to surface bottlenecks, but also inform the selection, execution, and tuning of interventions. Existing observability tools force a choice between limited metrics that obscure underlying issues and comprehensive traces, which are expensive to collect and postprocess and unviable at scale. We propose a fundamental shift: instead of proactive collection of all event logs, treat them as fragments of the global view, and use declarative dataflows for programmable, on-demand collection and real-time aggregation of these fragments. ORCA enables these capabilities through an aggregation and control plane optimized for BSP telemetry structure, leveraging a tree-based overlay with SQL semantics and distributed query planning to minimize data movement. This replaces a fractured ecosystem of bespoke tools with a unified, query-driven framework for real-time observability across all layers and phases of distributed execution.

1 Introduction

1.1 Motivation: Efficiency Bottlenecks in BSP

The Bulk Synchronous Parallel (BSP) paradigm powers some of our most critical computational tools, from large-scale scientific simulations in astrophysics and climate science to distributed training of foundation models in machine learning. These workloads — consisting of large parallel computations interspersed with I/O and global synchronization — currently operate at the scale of tens of thousands of accelerator nodes [1–4] and continue to grow with time [5]. As these applications scale, their efficiency has datacenter-level impacts on both cost and time-to-discovery.

The tightly synchronized nature of BSP creates distinct challenges along the data and compute paths. On the data path, bursts of checkpointed state create massive I/O demands on the storage and analytics infrastructure [6–8]. On the compute path, frequent global synchronization amplifies the system’s sensitivity to execution variability [9–13]. Runtime artifacts that would be manageable in loosely coupled systems, such as minor OS-level interruptions or thermal throttling, become critical scaling barriers when every worker must wait for the slowest participant. We now examine each path in more detail.

Data Management. Scientific applications generate massive volumes of data, particularly during checkpointing, which places intense pressure on I/O systems. Reading and analyzing this data after the fact is costly and can significantly slow time-to-insight. To mitigate this, many workflows now employ in situ analytics [14, 15], which process data as it is produced to reduce storage overhead and improve interactivity. While promising, most in situ frameworks remain specialized: they are typically tailored for visualization [16–20] or fixed-function transformations [21], and offer limited flexibility for general-purpose data organization. Critically, support for efficient range queries — essential for analyzing continuous scientific variables — is still missing. As a result, users are often forced to perform expensive post-processing sorts [22–24], reintroducing the I / O bottlenecks that in situ techniques aim to eliminate.

Compute Efficiency. Achieving high utilization remains challenging for many real-world BSP workloads [25]. In adaptive mesh codes [26], for instance, workload characteristics evolve dynamically as the simulation tracks changing physics, making it difficult to model or predict execution behavior. Even basic questions about compute load distribution, placement sensitivity, and communication overheads often lack clear answers. These effects are further confounded by environmental variability such as hardware faults or system noise—factors that can only be disentangled through in-situ observation. Traditional reliance on fine-grained tracing [27, 28] complicates diagnosis: traces are large, opaque, and difficult to query, while profiling tools lack the granularity to expose short-lived slowdowns that affect global synchronization.

1.2 Our Work: A Comprehensive Study of BSP Efficiency

We initially investigated BSP inefficiencies along the data and compute paths through two separate efforts: one focused on ingestion for range queries, the other on placement policies for dynamic workloads. While seemingly distinct, both efforts reflected specific instances of a broader pattern—establishing visibility into distributed cluster state as a precursor to context-aware interventions. This realization led to the design of ORCA, a programmable substrate for observability and control in production environments. The following sections describe each effort in turn.

1.2.1 Data Path: Streaming Data Reorganization

Motivation. Scientific analyses on continuous physical properties such as wavefronts or energy bands require efficient range queries [29]. Existing workflows typically achieve this by using a bulk sort [22–24] to reorder data after it is written, at the cost of multiple additional passes over the data. While in-situ ingestion systems exist for hash-partitioned layouts [30], they are unsuitable for range queries as hash-partitioning destroys key locality in the on-disk layout. Our goal was to build a completely in-situ ingestion mechanism for range queries, that did not require any reordering of on-disk data.

Challenges. While streaming ingestion for range queries can be done using range partitioning, doing it in a single pass requires perfect knowledge of the key distribution to generate balanced partitions. We found that to be impractical, as real scientific datasets are often highly skewed and evolve over time. Even well-chosen static partitions become imbalanced, resulting in write bottlenecks and degraded query performance. To be practical, a system must ingest data with arbitrary, unknown, and dynamic distributions, adapt to maintain balanced partitions on the fly, and scale across thousands of cores.

Our Contribution. We address these challenges with CARP [31] (*Continuous Adaptive Range Partitioner*), an adaptive partitioner that handles arbitrary key distributions and produces a partially ordered layout in a single streaming pass. CARP provides query latencies comparable to those of a full sort without any ingestion overhead, making it 5× faster than prior work.

1.2.2 Compute Path: Understanding and Mitigating Load Imbalance

Motivation. Poor utilization in adaptive mesh codes is a well-known problem, typically characterized as load imbalance [25, 26, 32]. We aimed to design placement policies that would incorporate runtime telemetry into placement decisions—not only to mitigate load imbalance, but also to balance other properties that we found to materially affect application runtime.

Challenges. Our efforts to collect and interpret telemetry, and create models of code performance that could be exploited in a placement policy encountered several persistent challenges.

1. Raw telemetry measurements did not correlate with expected behavior, forcing deep investigation of cross-stack performance anomalies, and revealing issues with hardware, network

stack parameters, operation ordering etc. These constituted runtime noise that had to be eliminated before telemetry and placement effects could be reliably observed.

2. Identifying, isolating, and tracing transient performance issues in a large parallel code required collecting and analyzing a large volume of telemetry. Existing performance tools were unhelpful: profiling tools did not provide the required granularity, while traces were slow and not analyzable.
3. Computing placements efficiently remained difficult, as even the simplest variant of the underlying problem is NP-hard and must be solved within tight timing constraints—under 50 ms per solution for our selected codes.

Our Contributions. We developed a diagnostic workflow and conducted an empirical tuning study to disentangle the effects of load imbalance from other runtime bottlenecks. These insights guided the design of CPLX, a hybrid placement policy with tunable control over two key tradeoffs: compute load balance and communication locality. By balancing both objectives, CPLX achieves up to 21.6% runtime improvement over tuned baselines [33].

1.2.3 ORCA: Transferring Lessons Learned to General-Purpose Observability

Motivation. Our work on both CARP and adaptive mesh codes revealed a common prerequisite for efficiency: the ability to construct a global view of distributed state to guide intervention. For CARP, this meant reconstructing the global key distribution to drive adaptive data reorganization. For mesh codes, it meant building a coherent performance picture to distinguish algorithmic bottlenecks from system noise. This visibility currently relies on tracing [27, 34], where prohibitive collection and analysis costs make it unviable at scale. Our goal was to build a general-purpose substrate that could efficiently materialize system views using on-demand and in-situ aggregation of telemetry.

Our Approach. ORCA (*Observability with Real-time Control and Aggregation*) is a general-purpose dataflow framework for cluster observability. Users define telemetry and control requirements as SQL queries over Arrow [35]-formatted data streams. These queries are compiled into distributed execution plans over a tree-structured overlay of aggregators and executed over per-timestep telemetry. Data is collected and analyzed only in response to active queries, enabling analytical costs to scale with analytical requirements. An asynchronous but timestep-consistent control path provides primitives to act on the basis of the feedback.

Challenges. To the best of our knowledge, ORCA is the first query-driven real-time observability system for tightly-coupled parallel applications. This requires solving challenges across the stack: from high-throughput, low-overhead data collection and scalable in-situ query processing, to the design of a control path that provides timestep-consistent semantics for an asynchronous human operator.

1.3 Thesis Statement

This thesis argues that persistent efficiency bottlenecks in *Bulk Synchronous Parallel* (BSP) systems are fundamentally visibility problems, stemming from an inability to reconstruct global views from distributed local state in real time, and proposes a programmable, query-driven observability and control plane to fill this gap.

1.3.1 Potential Impact

Immediate Impact. Application-level observability becomes a first-class capability. ORCA replaces brittle traces and low-resolution profilers with a lightweight, always-on, query-driven system that supports real-time diagnosis and runtime tuning. Developers can rapidly inspect distributed behavior, toggle probes, and react to anomalies without modifying application code or halting execution.

Longer-Term Impact. ORCA evolves into a flexible substrate for in-situ analytics and adaptive control. Its open architecture supports programmable analysis pipelines, integration of diverse telemetry sources, and runtime composition of custom indexing and aggregation strategies. Over time, it can subsume visualization, introspection, and control workflows into a unified framework that prioritizes insight per unit of data moved, enabling principled trade-offs between data fidelity, performance, and resource usage.

1.4 Document Structure

The rest of this document is organized as follows.

- Chapter 2 provides background on the HPC environment, application types, and known efficiency challenges.
- Chapter 3 describes CARP, an adaptive range partitioner.
- Chapter 4 describes our work on tuning and load-balancing adaptive mesh codes.
- Chapter 5 describes ORCA, a general-purpose observability and control framework.
- Chapter 6 outlines the remaining work required to complete the thesis.

2 Background: Efficiency Bottlenecks in BSP Systems

We first review key concepts and challenges in the bulk-synchronous parallel (BSP) paradigm, beginning with an overview of the computing environment (§2.1), followed by a review of data management (§2.2) and compute efficiency challenges (§2.3). While we primarily focus on scientific computing, we also discuss distributed machine learning training workloads, as their shared bulk-synchronous characteristics create similar challenges and inform our later work.

2.1 The High-Performance Computing Environment

System Architecture. Large-scale HPC clusters typically consist of $O(10,000)$ nodes [36] connected by a high-performance fabric [37–39]. Each node typically consists of CPUs and typically multiple GPUs, which are increasingly important for scientific computing [40, 41]. Storage is traditionally provided by parallel filesystems [42] but object stores [43] and richer commercial offerings [44, 45] are increasingly common [46].

Applications. Scientific applications target fundamental questions across domains such as cosmology and climate science, problems that require simulating complex physical phenomena over extended spatial and temporal scales. These applications can occupy the majority of a supercomputer’s resources for months at a time [1, 2, 47]. From a systems perspective, they can be simplified into two common archetypes: *particle codes* [48] and *mesh codes* [26]. Particle codes track the state and interactions of billions individual discrete entities as they move through the simulation domain. Mesh codes discretize the spatial domain into grids and compute the evolution of properties of interest at each grid point.

The BSP Model. The BSP model consists of alternating compute and synchronization phases. Execution begins with decomposition of the problem into independent subproblems assigned to workers, and proceeds as discrete timesteps. Each timestep consists of a compute phase followed by a global synchronization phase, where synchronous collectives [49, 50] ensure all ranks advance together. Every few timesteps, a checkpoint is triggered for both fault tolerance and offline analysis. This tight coupling is the root of cluster efficiency problems: synchronous checkpoints create coordinated I/O bursts that stress the storage system, while global collectives make overall performance acutely sensitive to the slowest worker. Growth in cluster sizes leads to increased checkpointing frequency, as the increased failure probability makes forward progress more valuable [6].

Distributed ML Training. Distributed ML training represents a similar and increasingly important cluster workload, occupying tens of thousands of GPUs and running for months [3–5]. The cluster architectures are comparable, though ML clusters often add a *frontend* Ethernet network for general datacenter connectivity [51]. While the parallelization strategies differ—ML employs a mix of data, tensor, and pipeline parallelism—the execution model still relies on the same core BSP primitives of synchronous collectives [51] and periodic checkpointing [7].

2.2 Data Management Challenges

Checkpointing creates I/O-bound bottlenecks in BSP systems, as coordinated writes across thousands of ranks stress parallel filesystems and consume significant runtime [52]. Traditional workflows then require scientists to re-read and analyze this data after simulation completion, consuming additional I/O bandwidth and computational resources while delaying scientific discovery. Ingesting data using conventional databases is not viable, as databases incur multiple on-disk writes for each application write [53] (*write amplification*), offering poor tradeoffs for ingestion-heavy workloads.

In-situ processing [15] analyzes data while it streams from application to storage, reducing both I/O overhead and time-to-insight. This approach is employed for multiple scientific use cases ranging from visualization [16, 17, 20, 54] to indexing [21, 55]. In-situ reorganization using hash-partitioning and shuffling has been demonstrated to be viable at HPC scale [30]. However, unlike databases that provide general query interfaces over composable indexes, existing in-situ systems are purpose-built for specific use cases and lack programmability. A critical gap remains in range query support, essential for analytics on continuous physical attributes but absent from existing in-situ frameworks.

2.3 Compute Efficiency Challenges

Cluster utilization challenges persist in BSP systems: a study of exported traces found more than 60% of runtime spent in MPI [49] operations (communication and synchronization), even at modest scales. We now provide additional context on performance tooling and workload characterization bottlenecks that contribute to these challenges.

Performance Analysis Tools. While real time cluster-level monitoring is common [56], application-level performance analyses are largely post-mortem [27, 34, 57, 58]. Two main approaches are tracing and profiling. Tracing generates fine-grained event logs for each worker (*rank*), which are then aggregated into a composite view via postprocessing. The volume of generated trace data presents an analytics challenge, delaying insight [59], and remains unviable at the largest scales [60]. In comparison, profiling generates phase-wise summaries of execution for each rank. While far more scalable, this obscures fine-grained interactions that degrade tightly-coupled BSP execution.

Workload Characterization Bottlenecks. While efficiency challenges are known and attributed to load imbalance [25, 26, 32], deeper understanding remains elusive, partly because of visibility bottlenecks discussed above. Characterization of workload properties such as compute kernel distributions, communication bottlenecks, placement and load-balancing requirements remain poorly understood. This is further exacerbated by the separation of analyses from data collection, as the latter are undertaken by separate groups without access to the underlying hardware that generated the data [61]. As a result, distinguishing workload characteristics from environment artifacts becomes even more difficult.

3 CARP: Adaptive In-Situ Range Partitioning

3.1 Motivation: Efficient Ingestion for Range Queries

Range queries are an essential component of a complete analysis toolkit. Analyses on scientific attributes of interest, such as wavefronts and energy bands, naturally map to range queries [29] because of the continuous nature of this data. However, supporting efficient range-based analyses remains expensive. Efficient range queries require an ordered data layout, which is created using bulk post-processing sorts after the application has finished executing. These sorts require multiple passes over the data — $2\times$ for in-memory [22, 23] sorts and $4\times$ for out-of-core [24] sorts — diverting finite I/O bandwidth, slowing down ingestion, and delaying the time to insight. As an alternative to post-processing, in situ indexing techniques have gained traction [14]. These techniques process data while in transit from application to storage, reducing the need for additional passes. DeltaFS [30] reorganizes data using hash-based partitioning, and has been demonstrated to scale up to 131K ranks. However, partitioning by key hash destroys the ordering needed for efficient range queries.

CARP [31] is a single-pass streaming in situ range partitioner for scientific data. It is designed to ingest at the maximum throughput permitted by the storage system by never reordering previously written data, while still enabling subsecond range queries. The fundamental challenge is maintaining balanced partitions for streams with skewed and dynamic key distributions. Our analysis demonstrates that scientific key distributions are highly skewed and evolve significantly as simulations progress, rendering any static partitioning approach ineffective (Fig. 1). CARP makes no assumptions about the workload key distribution, and instead discovers and adapts to it at runtime.

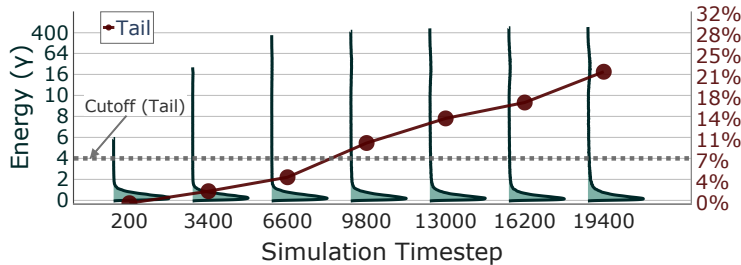


Figure 1: Energy distributions across 7 timesteps of a particle simulation (VPIC [29]). Intra-timestep distributions are highly skewed, with a long tail that grows significantly over time.

3.2 CARP: System Design

CARP is implemented as an in situ shuffling pipeline built around an all-to-all shuffler, *3-hop*, from the DeltaFS [30] project (see Figure 2). Each application rank acts as both a shuffle sender and receiver. In the common case, all ranks are assigned disjoint ranges, and shuffling simply forwards data to the rank responsible for the partition containing the indexed attribute value. On

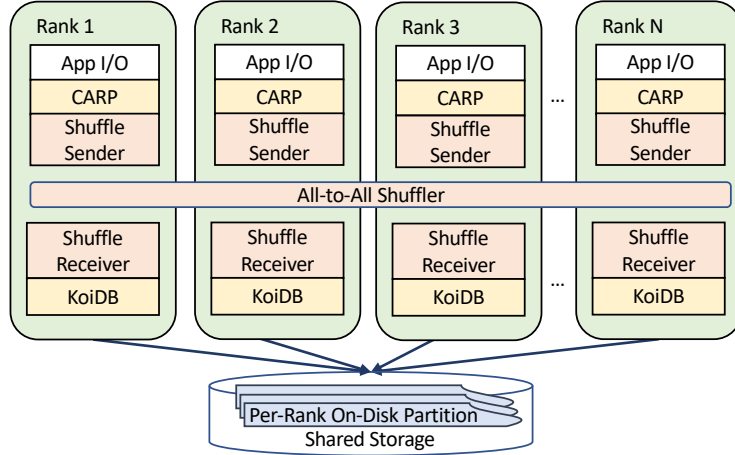


Figure 2: Example of an N -rank application using CARP. Writes are intercepted and partitioned via an all-to-all shuffler. Each rank is both a shuffle sender and a receiver. Partitioned data collected by receives is stored by a lock storage backend (KoiDB).

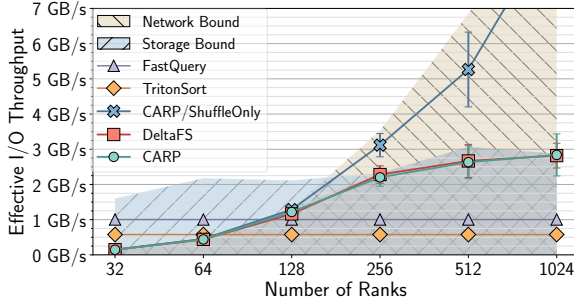
the receiver side, each rank accumulates writes into an embedded storage backend called *KoiDB*, which periodically flushes to a per-rank append-only log on the parallel filesystem.

CARP augments this shuffling pipeline with a primitive called *renegotiation*, which discovers the current global partitioning and computes new partition assignments. Any rank may invoke renegotiation. When triggered, each rank summarizes its local key distribution and sends it to the lead rank (rank 0), which merges the summaries to reconstruct the global key distribution. The lead rank then divides the global keyspace into partitions with equal area under the curve and broadcasts the new boundaries to all ranks. When partitions are updated, ranks flush their KoiDB instance and resume shuffling. For scalability, renegotiation is implemented as a tree-based reduction protocol.

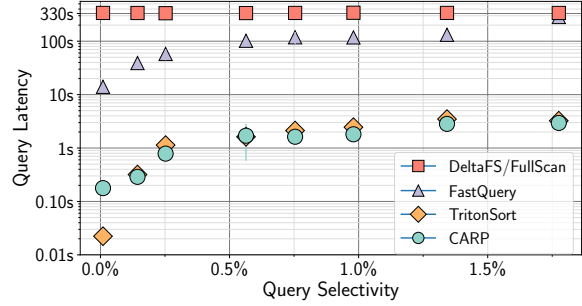
In addition to renegotiation, CARP includes mechanisms to trigger the process, bootstrap the distribution, and buffer data for unassigned ranges via the *out-of-bounds* (OOB) buffer. Filling the OOB buffer is one trigger for renegotiation, and its contents are factored into the partition computation. The KoiDB backend performs additional reprocessing to maintain partition quality despite out-of-order receives and rapidly changing boundaries, before periodically appending data to the per-rank log.

3.3 Key Results: Ingestion and Query Performance

Ingestion Performance. Fig. 3a shows the ingestion performance of CARP compared to TritonSort [24] (an out-of-core sort), FastQuery [29] (an auxiliary bitmap index), and DeltaFS [30] (in-situ hash partitioning). Being embedded in-situ approaches, CARP and DeltaFS are evaluated with different application ranks, while TritonSort and FastQuery are run as postprocessing stages



(a) Ingestion Throughput vs Application Scale



(b) Query Latency vs Selectivity

Figure 3: (Left) CARP incurs no overhead over unpartitioned I/O and is 3–5 $\times$ faster than post-processing. (Right) CARP matches the query latency of TritonSort’s fully-sorted data layout and outperforms FastQuery by 100 $\times$.

directly on the storage nodes. CARP achieves the maximum throughput permitted by the storage and network bounds. CARP has no measurable overhead on the ingestion path. At 1024 ranks, it is 2.8 $\times$ faster than FastQuery and 4.9 $\times$ faster than TritonSort.

Query Performance. Fig. 3b shows the query performance of CARP compared to FastQuery, TritonSort, and a complete sequential scan (DeltaFS/FullScan) for 8 queries with different selectivities. For all but the most selective query, CARP queries are as fast as a full sort and 1-2 orders of magnitude faster than FastQuery. Being an auxiliary index, FastQuery is forced to perform millions of random I/Os, resulting in relatively poor query performance approaching the cost of a complete sequential scan. The gap between CARP and TritonSort for the most selective query is explained by CARP being a partitioner, and is thus forced to read a complete partition for highly selective queries. Given the small absolute value (25ms) for such queries, the gap is not meaningful for analytics queries.

3.4 Lessons

CARP demonstrates that a greedy, single-pass approach can produce a partially ordered layout that supports efficient range queries without compromising ingestion performance. By approximating a clustered index over streaming data, it narrows query scope to a small number of partitions, reducing retrieval cost. The performance gap between CARP and auxiliary index-based approaches highlights the importance of physical partitioning as a first-class indexing mechanism. More broadly, CARP shows that scientific data systems benefit from the same design principles as databases, yet they currently lack the crucial ability to compose different indexing strategies at query time. The practical realization of these benefits requires adopting interoperable data formats and query engines that can leverage the available indexing and partitioning schemes.

4 Lessons from Optimizing Adaptive Mesh Codes

4.1 Motivation: Computational Efficiency in AMR Codes

Adaptive Mesh Refinement (AMR) [26, 62, 63] is a widely used technique for efficiently simulating physical systems that exhibit large variations in spatial or temporal scales, such as astrophysical phenomena, fluid dynamics, and climate modeling. AMR codes dynamically refine and coarsen the computational grid to concentrate resources in regions of interest, resulting in highly irregular and rapidly evolving mesh structures. This approach enables fine-grained resolution where needed, but creates significant computational challenges: the workload becomes highly imbalanced, dependencies between tasks grow complex, and the dynamic nature of the mesh makes it difficult to achieve efficient parallel execution and balanced load across compute resources [25, 26, 32].

Our goal was to develop placement policies that leveraged runtime telemetry to improve placement and efficiency. Access to our research cluster enabled us to experiment with fine-grained telemetry and placement under full-stack administrative control, providing the ability to analyze and intervene at all levels of the software and hardware stack.

4.2 Challenges of Telemetry-Driven Placement

Developing telemetry-driven placement policies in AMR codes exposed several key challenges:

- **Establishing measurement fidelity.** Initial telemetry was often unreliable and did not match intuition (Fig. 4). Only by using full-stack access and low-level debugging could we identify and correct sources of error across hardware, drivers, MPI, and the application.
- **Scaling analysis for high-volume telemetry.** Analyzing per-rank, per-timestep telemetry required moving beyond standard profiling and tracing tools to query-driven workflows that could surface root causes from massive data.
- **Low-overhead, effective placement.** Even with reliable telemetry, incorporating it into placement imposed strict timing constraints and required new algorithms to balance objectives like load and locality in dynamic, NP-hard settings.

4.3 Contribution 1: A Diagnostic Workflow

The process of developing placement policies required understanding what our telemetry was capturing. Initially, our telemetry showed no correlation between expected workload drivers and observed performance (Figure 4b). Deeper investigation revealed cross-stack artifacts: fail-slow hardware, system tuning issues, and fabric-level problems that had to be systematically mitigated (Fig. 4a). Only after this full-stack debugging and tuning process could we obtain reliable telemetry that correlated with expected application behavior.

While our full-stack access enabled these investigations, we found identifying and isolating these transient artifacts to be difficult. We initially used standard profiling and tracing tools [57], but found the behavior of our applications to necessitate structured and targeted telemetry collection for

analytics. Profiling often obscured visibility into transient artifacts, while tracing resulted in large volumes of analytically intractable data. We organically evolved a workflow for targeted collection of queryable telemetry, starting from CSV exports analyzed with pandas [64], and culminating in ClickHouse [65] and SQL queries. This custom approach enabled correlation of performance data across ranks and system layers, allowing us to isolate the true sources of performance anomalies.

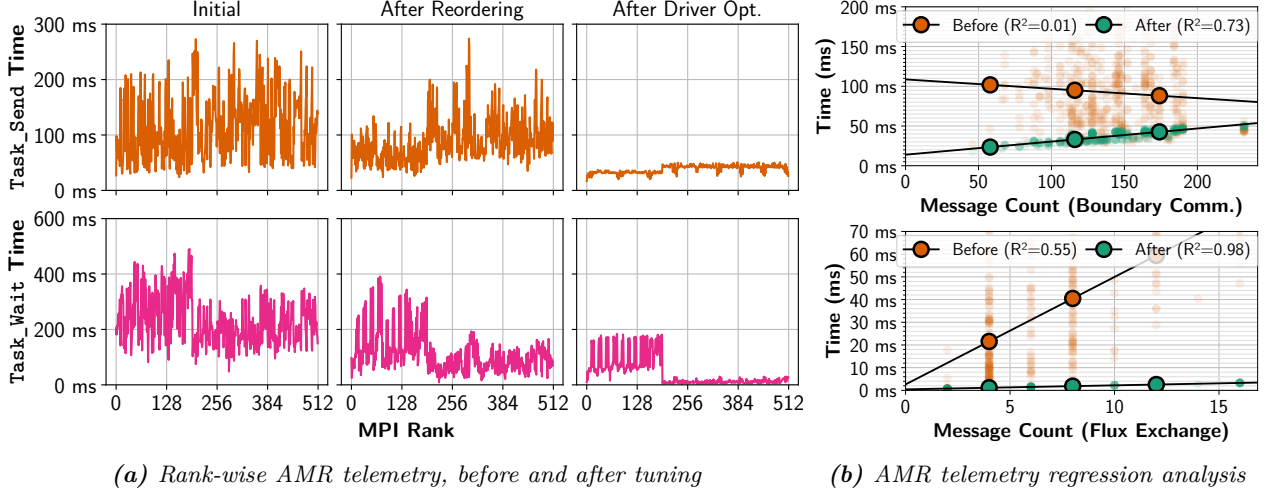


Figure 4: (Left) Fine-grained per-rank telemetry from communication functions showing significant variability, that progressively reduces with tuning. (Right) Regression analysis of communication telemetry showing poor initial correlation with work, that improves significantly with tuning.

4.4 Contribution 2: The CPLX Placement Policy

With reliable telemetry established, we turned to the challenge of developing more effective placement policies. Placement strategies for AMR codes have traditionally prioritized maximal preservation of communication locality [66–70], using tools like *Space-Filling Curves* [71] (SFCs) to co-locate spatially adjacent grid blocks. However, we recognized that for computationally variable workloads, an exclusive focus on locality is fundamentally at odds with achieving good load balance. This led us to reframe placement as not about maximizing a single objective, but about explicitly managing the trade-off between communication and computation to minimize overall runtime.

To navigate this trade-off, we developed CPLX, a policy that provides predictable control over locality and load balance through a single tunable parameter, X . CPLX operates as a hybrid of two underlying policies: a locality-preserving dynamic programming approach (CDP) developed by us and the load-balancing-focused *Longest Processing Time* (LPT) heuristic [72]. The algorithm first computes a locality-aware placement using CDP. It then identifies the $X\%$ most imbalanced ranks (both overloaded and underloaded) and rebalances *only* their assigned work using LPT, leaving the rest of the placement untouched. This mechanism allows CPLX to smoothly interpolate between a fully locality-preserving policy ($X = 0$) and a pure load-balancing policy ($X = 100$), enabling empirical tuning of the trade-off.

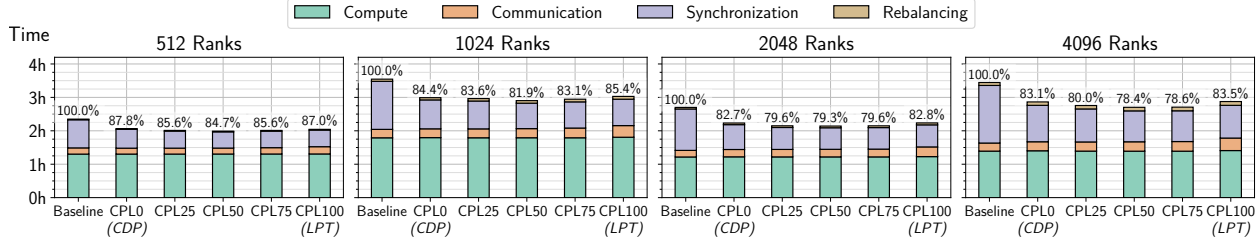


Figure 5: End-to-end AMR code runtime as a function of policy at four scales (512–4096 ranks), classified into four phases. CPLX achieves up to 21.6% runtime reduction over already-optimized baselines. Total runtime follows a U-shaped curve as X increases, with best performance at intermediate values, from balancing the constituent tradeoffs. Impact of CPLX increases with scale.

4.5 Key Results

We evaluated our placement policies on our in-house research cluster using the Phoebus [73] code built on the Parthenon AMR framework [62], at scales ranging from 512 to 4096 ranks. We chose the Sedov blast wave [74] problem as our workload, as its dynamic refinement patterns make it a challenging test case for placement policies. All comparisons are against the tuned baseline using standard SFC-based placement.

As shown in Figure 5, CPLX achieves up to a 21.6% runtime reduction over the optimized baseline. Overall performance follows a U-shaped curve as X is varied, with the best results achieved at intermediate values. At $X = 0$, CPLX is locality maximizing at the expense of suboptimal load balance, while at $X = 100$, CPLX completely disregards locality to maximize load-balance. Minimum runtime is achieved at intermediate X values by balancing the constituent trade-offs. The impact of CPLX and managing the trade-off both increase with scale.

4.6 Lessons

Elevated synchronization time in AMR codes is commonly ascribed to algorithmic load imbalance. While execution variability is a known issue [9–11, 13, 51, 75, 76], it is often overlooked in performance analyses, which are typically conducted offline and disconnected from runtime context [25]. Our work shows that accurate diagnosis requires holistic, in situ performance analysis to isolate the true sources of variability. We find that algorithmic techniques can improve placement, but they also expose tunable tradeoffs—such as load balance versus communication locality—whose optimal configuration also benefits from real-time visibility into application behavior. Our organically evolved pipeline demonstrates the value of structured, queryable telemetry, and motivates the realization of real-time observability capabilities tailored to the needs of tightly-coupled parallel applications.

5 ORCA: A Unified Framework for Declarative Observability

5.1 Motivation: Programmable and Real-time Cluster Observability

Both CARP and our work on AMR placement reveal a persistent and recurring need to reconstruct cluster state to synthesize views that enable better decision-making. CARP automatically uses this reconstructed state to inform online data reorganization, and our AMR work reveals the urgent need for such capability on the compute path, where disambiguating between load imbalance, system variability, and execution strategies is essential for computational efficiency.

Although the telemetry workflow evolved during our AMR placement work demonstrated the importance of structured and queryable fine-grained telemetry, the pipeline itself remained ad hoc and limited. We derive three requirements for practical observability capabilities for production clusters from our experience.

- **Real-time and Coordinated Analytics.** Insights should be available in real time, with minimal application overhead, while also allowing the option to persist detailed telemetry for offline analysis. This requires coordinated, timestep-wise telemetry collection from all ranks.
- **Complete Programmability.** Effective observability requires supporting analytics over arbitrary data sources across the application stack, with varying schemas and aggregation requirements. We found fixed aggregations or rigid schemas to quickly become limiting as requirements evolve—programmability is essential for sustained utility.
- **Interactivity.** Observability workflows vary significantly across the lifetime of a long-running BSP application, and an effective framework must support the full span of these workflows while incurring costs commensurate with the analysis requirement. Lightweight monitoring may suffice in the common case, but when performance anomalies arise, detailed telemetry becomes essential. The framework must enable dynamic transitions between these modes without fixed overhead penalties.

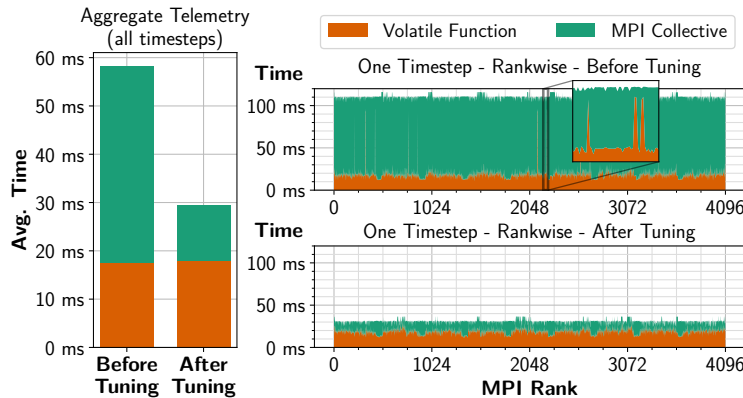


Figure 6: An example of a BSP performance problem from a real AMR code, simplified as a volatile function followed by global synchronization. (Left) Aggregated telemetry shows elevated average synchronization times, but does not reveal the problem structure. (Right) Fine-grained telemetry shows random spikes in the volatile function delaying synchronization. Tuning the volatile function does not affect its average time, but leads to a 50% reduction in their combined time.

5.2 Background: The Observability Gap in BSP Systems

Fig. 7 compares different observability approaches for BSP telemetry, modeled as a two-dimensional grid of ranks and timesteps. Among existing approaches [27], comprehensive tracing [34, 57, 77] is the only method that preserves the fine-grained telemetry needed to diagnose complex performance issues in large-scale BSP systems (see Fig. 6). However, the high application overhead, unstructured and analytics-unfriendly output [28, 59], and delayed feedback from post-processing make tracing impractical for production use. As clusters and their resource footprints continue to grow, these limitations become more acute, necessitating a ground-up rethinking of observability architectures to keep pace with modern system demands.

Profiling [57] and sampling [56] tools, while lower overhead, impose arbitrary fixed collection and aggregation patterns that obscure the structure of underlying problems and mask important temporal or spatial correlations. Time Series Databases (TSDBs) [78, 79] — widely used for metrics in distributed services — are not a direct fit for scalable BSP observability, as they are not optimized for continuous time-windowed aggregations rather than discrete timestep-synchronized analytical queries over BSP telemetry. As a result, cluster efficiency challenges persist not due to algorithmic bottlenecks, but because of persistent limitations in what can be observed, analyzed, and acted on in real time.

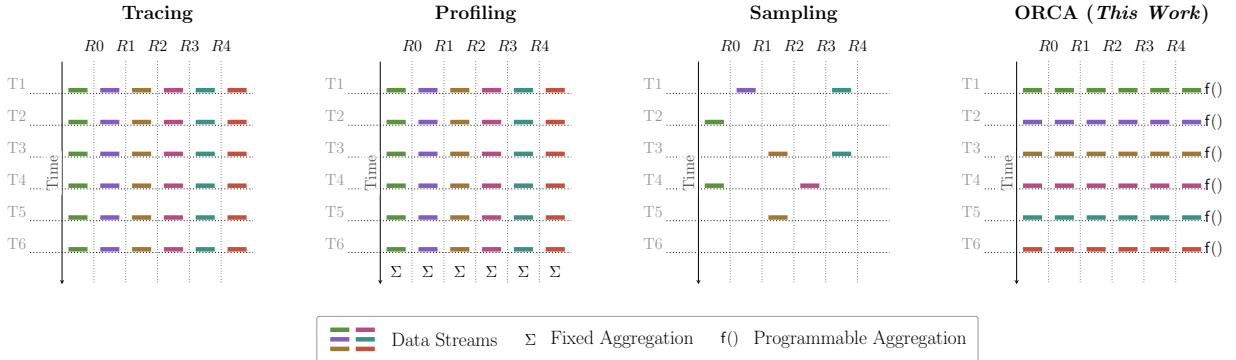


Figure 7: A comparison of different observability approaches for BSP telemetry, modeled as a two-dimensional grid of ranks and timesteps. Tracing captures all event data, but writes it as separate per-rank streams to storage, posing analytical challenges. Profiling and sampling offer arbitrary fixed reductions of data, obscuring insight. ORCA enables coordinated collection of timestep-wise telemetry from all ranks, with programmable in-situ aggregation.

5.3 ORCA Architecture

ORCA is designed to address the challenges of scalable, real-time observability in large parallel systems via programmable aggregation and control paths. The framework enables users to express analytic intent as SQL over telemetry. Unlike tracing, which collects and stores all event data, ORCA leverages distributed query planning to transfer only the data necessary to materialize each requested view. The entire pipeline, from collection to aggregation to storage, is optimized

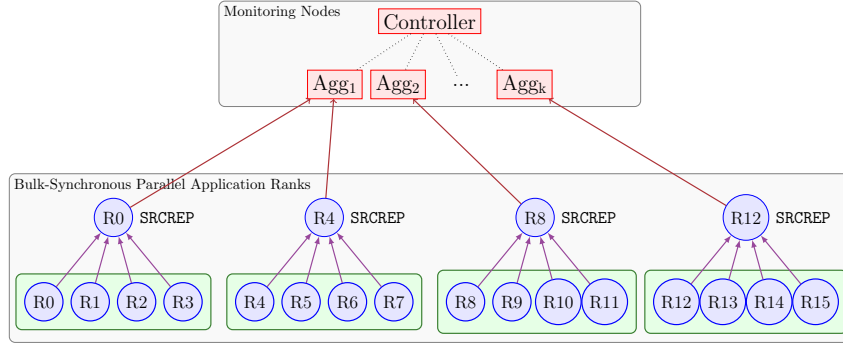


Figure 8: An example of an ORCA tree-based overlay topology, with 16 MPI ranks across 4 nodes connecting to a controller via a hierarchy of aggregators, and optionally a tier of local lead ranks. Application ranks form the synchronous domain while ORCA executes asynchronously.

to preserve the BSP structure of the data. Its control semantics enable dynamic transitions between different views at runtime, supporting interactive, low-overhead workflows. Below, we briefly describe the core components of the ORCA architecture.

- **Tree-Based Overlay Network (TBON):** As shown in Fig. 8, ORCA organizes application ranks into a logical tree-based overlay with a controller at the root, a tier of aggregators, optionally a tier of lead ranks (aggregating within node groups), and application ranks as leaves. Aggregators handle data collection from a subset of ranks, using RDMA for data transfer when available.
- **Distributed Query Planning:** Telemetry is represented in Apache Arrow columnar format, and the overlay implements distributed query planning using Apache Datafusion. Projection, filtering, and partial aggregation are pushed down to lower tiers, so only necessary data is transferred up the tree.
- **Control Semantics.** ORCA implements a control path using a novel variant of two-phase commit, called *timestep-linked two-phase commit* (TS2PC). TS2PC is an asynchronous protocol that enables control actions to be applied consistently at a specified timestep across all ranks. This provides low-overhead replicated state machine semantics and supports dynamic observability workflows.

5.4 Potential Usecases and Impact

ORCA enables a range of use cases across high-performance computing and machine learning workflows, while providing a foundation for broader analytics and control in the future.

HPC Workloads. ORCA enables continuous, always-on observability for large-scale simulation codes. Unlike traditional workflows that require choosing between lightweight profiling and expensive tracing, ORCA supports a seamless spectrum: developers can inject detail dynamically and escalate from low-overhead monitoring to deep, on-demand root-cause analysis as needed. This

removes silos between telemetry collection modes and provides actionable insight without workflow disruption.

ML Workloads. Distributed ML workloads, because of their shared BSP characteristics, present similar performance and observability challenges as HPC codes [51]. For example, PyTorch [80] enables tracing each node using Kineto [34], but relies on post-processing to aggregate per-node telemetry into global views [58]. Limitations discussed in the context of HPC observability — data volume, delayed feedback, and lack of analytical capabilities — apply equally here. Beyond performance, recent ML research has emphasized the need for runtime observability over application state to enforce correctness invariants [81]. We find that ORCA naturally accommodates such use cases because of its focus on programmability, enabling a common substrate for multiple forms of observability.

Longer-Term Vision. ORCA’s software-based architecture — incorporating open standards such as Apache Arrow [35], Datafusion [82], and Mercury [83] over libfabric [84] — makes it naturally extensible. Many scientific analyses depend on multidimensional and unstructured meshes, which are not straightforward to represent in the relational model of standard SQL. Prior work [85–88] provides clear paths for extending SQL to cover these data types, and ORCA is amenable to absorbing these capabilities, and enabling in-situ visualization workflows within the same pipeline. Previous work has also demonstrated the utility of accelerators for in-situ analysis on Arrow data [89, 90]. ORCA makes realizing these capabilities practical by exposing accelerators and smart network resources as nodes within its dataflow graph. This enables a unified analytics and control fabric for scientific and data-intensive computing ecosystems to emerge organically over time.

6 Thesis Completion Roadmap

ORCA is under advanced stages of development. We first briefly describe the current state of the system, and then outline the remaining work required to complete the thesis.

6.1 Current State

Completed Developmental Work. Most core components of ORCA have been implemented. Completed components include overlay bootstrapping with multiple aggregators and topologies, telemetry collection and forwarding mechanisms, with per-timestep analytics semantics. Observability requirements are programmed using a domain-specific language (DSL) called OrcaFlow: the DSL enables defining transformations as SQL, applied in-situ while data is routed to configurable *sinks*. The TS2PC protocol for the control path has been implemented. End-to-end telemetry visualization is available via a DuckDB [91] and a FlightSQL [92] server embedded into the controller, streamed to Grafana [93]. ORCA reuses the same infrastructure to export internal metrics for realtime visualization and performance analysis. Observability requirements from recent machine learning literature [81] have been reviewed and incorporated in system semantics.

Validated Capabilities. A single-aggregator deployment of ORCA has been validated to be stable in a stress test with 128 MPI ranks loading the aggregator with 2–3 GB/s of telemetry. Analytics were performed locally on the aggregator, achieving near-line rate performance under ideal conditions on 40 Gbps Infiniband NICs. Stress testing surfaced stability issues, which were mitigated by tuning pipeline parameters and implementing flow control mechanisms. We found basic Arrow-based in-memory analytics to be much faster than our ingestion rate. The RDMA-based transport layer has been tuned and shown to be stable.

Remaining Developmental Work. The primary remaining component is distributed query planning to fragment the OrcaFlow pipeline across the overlay at operator-level granularity. Development of the query planner is completed and integration with the overlay is underway. Python bindings for Pytorch [80] integration are also under development for the planned experiments.

6.2 Evaluation Roadmap

The evaluation will demonstrate the practical benefits of ORCA’s relational approach in terms of query performance, system scalability, and end-to-end impact. The plan consists of the following experiments:

- **Microbenchmarks:** Compare query efficiency and analytical flexibility using on-disk Parquet [94] telemetry versus traditional trace formats. AMR-inspired queries [33] will serve as the HPC baseline, and Pytorch HTA [58] will be used as the ML baseline.
- **Synthetic stress tests:** Measure ORCA throughput and scalability as a function of aggregator count, and quantify application jitter as a function of data load.

- **End-to-end workload studies:** Measure runtime overhead and impact on scientific progress for a 4,096-rank AMR simulation [33] and a data-parallel ML training job, comparing with the overhead of full tracing. Evaluate ORCA in two configurations: a lightweight in-situ aggregation setup and a full configuration that persists all telemetry at tracing volume. The ML training job will be a data-parallel job running on a single node, and the AMR simulation will be run on a single node.

Additionally, an analysis of observability requirements identified from ML training literature, demonstrating its compatibility with ORCA semantics, is ongoing. This is not expected to have an experimental component.

6.3 Timeline

The thesis is expected to be completed by the end of 2025. The following timeline is proposed:

- July–Aug 2025: Complete distributed query planning and wrap up core development.
- Sept–Oct 2025: Evaluation, paper writing, and job search.
- Nov–Dec 2025: Complete and defend thesis.

7 CARP Paper

Abstract

Ingestion of data generated by high-performance scientific applications continues to stress available storage resources. Efficient range-based analyses on this data can be enabled by re-ordering it on attributes of interest, but require expensive post-processing sorts to realize the query benefits of reordering. In-situ indexing techniques, while write-efficient, are orders of magnitude slower at range queries than sorted indices. Range queries are necessary for analyzing continuous physical attributes and tracking phenomena such as energy bands and wave fronts.

We present CARP, a scalable data partitioner for range queries that reorders data in-situ as it is streamed to storage during application I/O. Motivated by our findings that real application distributions tend to be highly skewed and dynamic, CARP dynamically discovers and adapts its data partitions to track these characteristics. As a result, CARP can approximate the query performance of a sort without any ingestion overhead, making it $5\times$ faster than prior work.

7.1 Introduction

Organization of data generated by high-performance applications continues to be a challenge due to storage bandwidth constraints. Many high-performance applications consist of distinct computation phases, often called *epochs* or *timesteps*, that are periodically interrupted by checkpointing I/O phases. Examples of such applications include particle-in-cell frameworks [29], mesh-based codes [62, 63, 73], and machine learning model training. Periodic I/O phases enable scientists to perform queries with a temporal component, such as tracing trajectories of high-energy particles, or tracking shock waves in a fluid simulation. For efficient execution of such queries, persisted

data is typically indexed in a post-processing stage. This post-processing requires additional time and resources, increasing the time to discovery. It also increases with the scale of the simulation, especially as computational bandwidth growth outpaces that of storage bandwidth [14].

To reduce the time to discovery, in-situ approaches that index data as it is written by the parallel application have been proposed [95] as an alternative to post-processing. DeltaFS [30] intercepts application writes and organizes them into hash-based partitions. This enables point queries that retrieve individual items by key, but it breaks key locality. Range queries require key locality to efficiently find all items with keys in a given range. Range queries are an important tool for scientists interacting with continuous attributes such as temperature and velocity. They can be used to filter data into ranges of relevant values, such as isolating blast wave fronts or tracking energy bands. However, existing solutions for range query indexing force undesirable tradeoffs between write performance, query performance, and resource utilization (§7.2).

To enable efficient range queries without post-processing, it is necessary to develop reordering-based in-situ indexing mechanisms. Reordering creates key locality in the data layout, which enables range queries to be served efficiently via a small number of contiguous writes. However, reordering of on-disk data stresses write performance by requiring I/Os for index maintenance. This leads to slower checkpointing and idling of compute nodes. Enabling efficient querying without adversely affecting write performance requires reordering data in the network data plane, before writing to storage. Doing so without requiring additional compute resources requires a lightweight embedded reorganization mechanism with a low memory footprint. These challenges are further compounded by the nature of scientific key distributions — our analysis of real-world codes (§7.3) shows that scientific key distributions are highly skewed and vary over time. Reorganization mechanisms processing such data need to be adaptive to ensure that load hotspots in the reordering pipeline do not slow down application writes.

We present CARP (*Continuous, Adaptive, Range Partitioner*), an adaptive in-situ indexing system that enables range queries without requiring data post-processing. CARP requires no changes to scientific applications and operates by intercepting writes in flight to storage. User input on key distribution characteristics is not required. Partitions are instead discovered at runtime and adapted as the key distribution changes. CARP is write-optimized and does not divert any storage bandwidth from the application for reordering. Our evaluation shows that CARP incurs no overhead on application runtime and can achieve sub-second range query latency that is up to two orders of magnitude faster than state-of-the-art auxiliary indexing approaches. By partitioning data at runtime, CARP is up to $5\times$ faster than post-processing approaches.

The contributions of this work are summarized as follows:

- After a brief discussion on background (§7.2), we characterize workload distributions and their drifts in modern scientific applications (§7.3) and demonstrate that static partitioning is inadequate for such workloads (§7.7.2).
- We present CARP (§7.5), an in-situ data processing framework guaranteeing data locality and load balancing at scale. We identify requirements for a CARP storage backend and present

Table 1: Range query indexing approaches.

Approach	In-situ	Efficient Indexing	Efficient Querying
Bulk Sorting (Clustered)	×	×	✓
Bulk Sorting (Auxiliary)	×	×	×
Bitmap Indexes (FastQuery)	×	×	×
DB Indexes (LSM-Tree, B-Tree)	✓	×	✓
DeltaFS	✓	✓	×
CARP (This paper)	✓	✓	✓

KoiDB (§7.5.4), a reference backend implementation.

- We show that partial locality in data layout can be created in a single pass without requiring any reorganization and provide nearly-optimal range query performance (§7.7.1), followed by a discussion on adapting CARP for different requirements (§7.8).

We have released the source code [31] for both the CARP framework and the KoiDB backend.

7.2 Background and Motivation

Data generated by many scientific applications consists of simulated entities such as particles and mesh cells, each storing several attributes (e.g. energy, velocity) into a *record*. This data is written out at fixed intervals and not mutated once written, making bulk indexing feasible. The data is then analyzed using point queries (e.g., retrieving particles by ID) or range queries (e.g., retrieving particles with energy in a certain range). Range queries are needed for interacting with continuous attributes and for filtering data using ranges of relevant values (e.g. energy bands, temperature and blast wave fronts).

An efficient range-query index must locate ranges of values quickly. Sorted indexes do this by creating indexed attribute (henceforth referred to as *key*) locality in the data layout, so that the keys for a range can be located in one or few data partitions. Clustered indexes further improve query performance by storing all records with their corresponding keys. This allows for data for a query to be retrieved efficiently via fewer, contiguous reads. In contrast, auxiliary indexes need multiple random reads to retrieve all attributes of interest in the range. In general, a database table can have only one clustered index. The clustered index is typically reserved for the attribute most commonly used in queries [96, 97].

Table 1 shows that no current approach enables both in-situ efficient indexing and efficient querying for ranges. We summarize the characteristics of the indexing approaches below. We discuss their *index construction* performance and break down their query performance into *index*

lookup and data retrieval.

Index Construction. Bulk parallel/distributed sorting [22–24] approaches support range queries but require post-processing. These can be used to build either clustered or auxiliary sorted indexes. FastQuery [98], on the other hand, builds auxiliary bitmap structures where keys are sorted and stored separately with a pointer to their records. Clustered indexes enable more efficient queries but require more I/O to reorder and write the entire dataset. FastQuery does not reorder data, but results in data scans, computing large bitmap vectors, and writing auxiliary structures to disk.

To avoid expensive post-processing, techniques have been proposed to intercept application writes and index them in-situ before data is written to disk. DeltaFS [30] shuffles data into hash-based partitions and requires no post-processing, but it can only support point queries on the partitioned data. Distributed databases maintain online indexes but offer lower write efficiency as they periodically reorganize on-disk data [53]. Databases are designed for mixed workloads with updates, while lightweight indexes are better suited to scientific data.

Index Lookup and Data Retrieval. Efficient range queries require quickly finding matching records using the index and retrieving the records from storage using large sequential reads. Only sorted and clustered indexes provide both of these properties. Auxiliary indexes can return the offsets at which matching rows are present, but retrieving those records requires multiple random reads. Additionally, FastQuery queries require provisioning enough compute nodes to fit large index structures in memory, and queries can only be processed once these indexes are loaded.

With CARP, we aim to pair the query latency of a sorted and clustered index with the reduced overhead of an in-situ approach. We discuss the challenges of meeting these goals in the next section.

7.3 Range Query Challenges

Sorted, clustered indexes provide the lowest query latency but require post-processing to reorder data. Sorting large on-disk datasets is even more expensive than building auxiliary indices, as it requires multiple read/write passes over the data [24].

As an alternative to reordering via post-processing, data can be partitioned into *range-based partitions* while in transit from the application to storage. This accelerates queries, as only a small number of relevant partitions need to be retrieved from storage for each query. While range partitioning is a well-known technique in distributed databases, adapting it to in-situ partitioning of scientific data requires overcoming two key challenges we describe next.

Write Rate. To organize data in a way that does not reduce the application write throughput, it is not sufficient to partition data online. The indexing pipeline must be designed to not require any storage bandwidth for reordering or index maintenance. Conventional databases use reordering to maintain their partitions, which reduces effective storage bandwidth [99, 100]. As on-disk partitions grow beyond configured sizes, they are split into smaller partitions and migrated across nodes.

The maximum achievable write rate is a function of the average number of I/O operations performed for each application write, also called the *Write Amplification Factor* (WAF). A WAF

of 10 will turn an I/O phase lasting 12 minutes into 2 hours, which is wasteful for write-intensive large-scale scientific applications. WAF for write-optimized single-node database indexes has been measured at 19-37 [53] and is much higher for B-tree indexes and distributed databases [101]. In-situ strategies that have a high WAF would not outperform post-processing approaches (WAF of 2-3 $\times$). To maximize gains from an in-situ indexing approach, we constrain our design to a WAF of 1 $\times$.

Adaptivity. Effective range-partitioning requires the key space to be partitioned such that the generated data partitions are relatively balanced. This is important for both write and query performance. On the write path, this ensures that the application is not slowed down by a straggler writing a significantly larger partition than others. On the query path, imbalanced partitions will lead to much slower performance for queries corresponding to those partitions.

However, we find that keys in scientific data are often skewed in unpredictable ways and change as the simulation progresses. Specifically, we studied the energy distributions of two applications — VPIC and Phoebus. VPIC is a plasma physics code that simulates particle physics phenomena such as magnetic reconnection [29]. Phoebus is a mesh-based hydrodynamics code that we run with a Sedov blast wave setup [73]. We ran both codes at 512 ranks and studied the energy distributions in the output data.

Fig. 9 shows the energy distributions of both the codes over time. For VPIC (Fig. 9a), we find that the energy distributions are highly skewed with most particles falling between 0 and 1 with long tails that get longer and heavier as the simulation progresses. Towards the second half of the simulation, 20-30% of the data is contained within the tail generating a bimodal distribution with a second mode between 16 and 64. For Phoebus (Fig. 9b), we also see a highly skewed distribution with a tail. Initially, there is a high energy explosion but most of the mesh has no energy. Over time, the energy from the explosion dissipates into a bigger area and moves the distribution into a medium energy band.

These findings highlight the challenges with partitioning keys from scientific workloads. A highly precise reconstruction of the key distribution is necessary to divide the keyspace into a large number of balanced partitions. It is also impractical for any user-provided static range partitioning scheme to accommodate such skewed and dynamic key distributions. A distributed database would only be able to balance its partitions after a series of splitting, rewriting, and merging operations, incurring prohibitive index construction costs. An ideal range query index would discover and adapt to workload characteristics and reorder data without consuming excess storage bandwidth.

7.4 Design Principles of CARP

We have designed a streaming in-situ partitioner for large parallel applications, CARP, which intercepts writes from applications, transforms them, and persists them in a range query-friendly layout. A logical view of CARP’s partitioning process is shown in Fig. 10. CARP has no impact on application runtime, as it uses spare CPU and network capacity to transform data at a rate

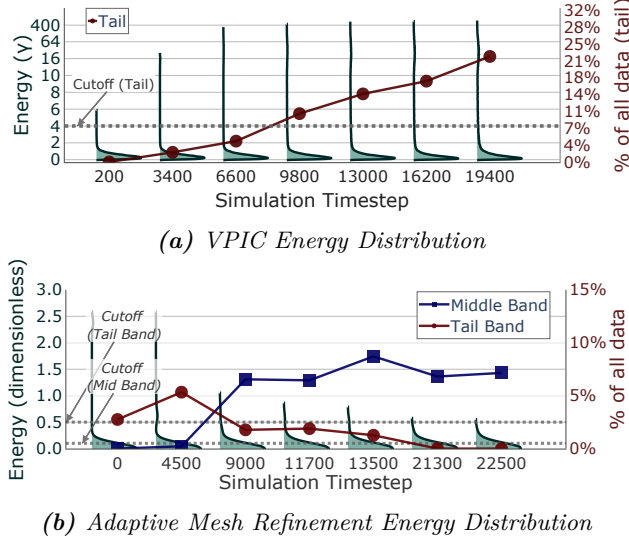


Figure 9: Energy distributions across 7 timesteps of VPIC (top) and AMR (bottom) simulations. Distributions are shown as histograms overlaid with line plots for interesting bands. Both VPIC and AMR distributions are highly skewed and shift significantly over time.

necessary to saturate storage. This partitioned output requires no post-processing and can be queried directly once the application terminates.

CARP augments conventional range partitioning with novel techniques to achieve its write and query performance goals. We build upon a simple insight — data does not need to be fully sorted to achieve the performance benefits of a sorted, clustered index. It is sufficient to partially order data in a way that allows queries to be processed via a small number of large reads. CARP exploits this relaxation to achieve its write performance and adaptivity goals.

We employ a greedy approach to generate best-effort range partitions in a single streaming pass so as to not require expensive reorganization of written data. To generate balanced partitions, CARP employs a novel primitive to construct and monitor the global key distribution. This distribution is divided into partitions with equal areas under the curve, and each partition is assigned to a rank. In the common case, all ranks shuffle data to corresponding partitions. As the workload’s key distribution drifts, CARP recomputes partition boundaries and continues shuffling. Shuffled data is written to disk via a storage backend called *KoiDB* — *KoiDB* further optimizes CARP output to improve range query performance. Next, we describe how CARP achieves the design goals laid out in the previous section.

Write Performance. To ingest data at storage layer throughput, CARP is designed to not divert any storage bandwidth for reorganization. Even when partitions need to be recomputed, CARP does not touch previously written data — it simply records a change in partition boundaries on each rank and continues appending data. CARP also employs standard storage optimization practices such as batching small writes into large immutable units called *SSTables* and having its on-disk layout be an append-only log.

Query Performance. Efficient queries in CARP are enabled by 1) creating a highly parti-

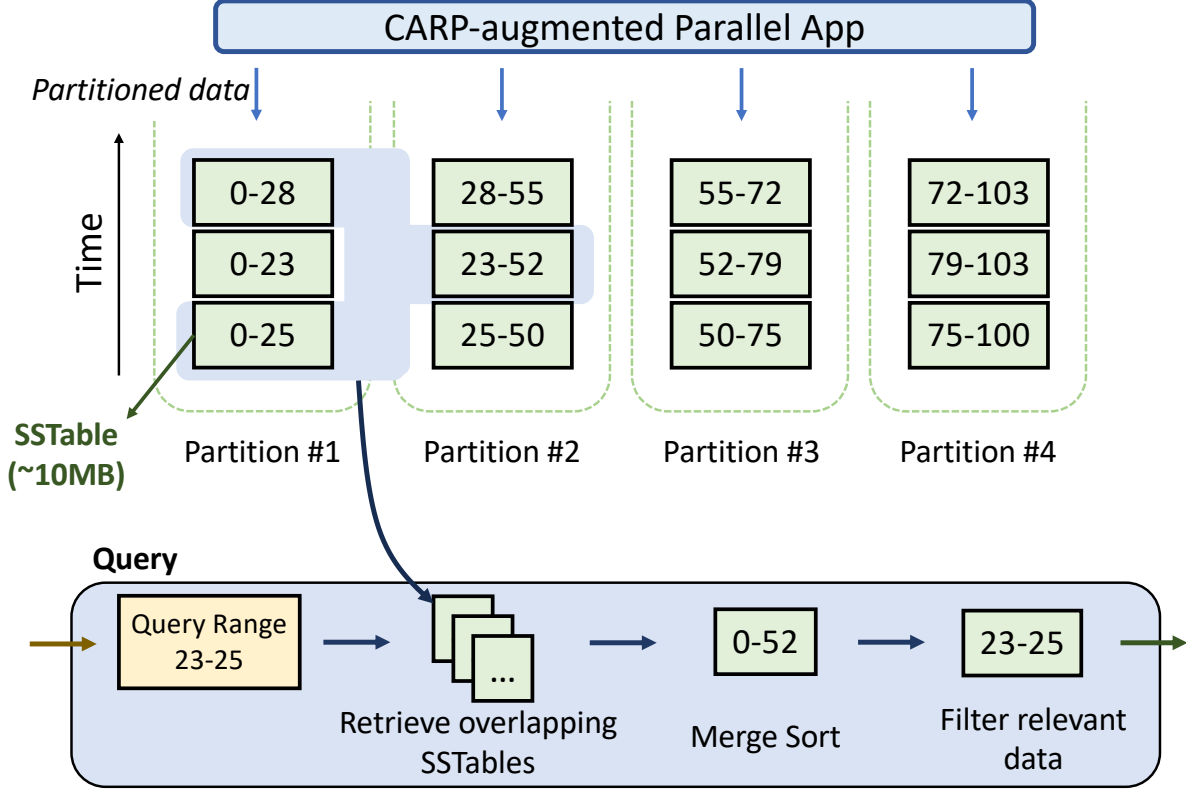


Figure 10: A logical view of CARP’s data layout and querying. Incoming data is partitioned and stored as SSTables, and partition boundaries shift with key distribution changes. Queries retrieve relevant SSTables and merge them.

tioned layout, and 2) enabling efficient retrieval via large sequential writes. CARP creates one partition per application rank, automatically ensuring more data partitions as application scale increases. A selective range query only needs to retrieve a fraction of the total data from overlapping partitions. In addition, data is written in large SSTables, which can be read efficiently via large read requests. CARP merges its partially ordered SSTs at query time via a merge-sort operation, which is cheap compared to the I/O cost of retrieving data.

Adaptivity. To adapt to skewed and dynamic key distributions, we recompute CARP partitions as the incoming distribution changes. Unlike conventional databases, we do not repartition data that has already been written to disk. We simply record the change in partition boundaries and continue appending new data. Over the lifetime of a run, the range allocated to a CARP partition changes multiple times. As a corollary, data belonging to one point in the range may end up in different partitions at different times in the run. Instead of incurring write amplification to consolidate this data, we simply postpone this consolidation (also called *compaction*) to query time.

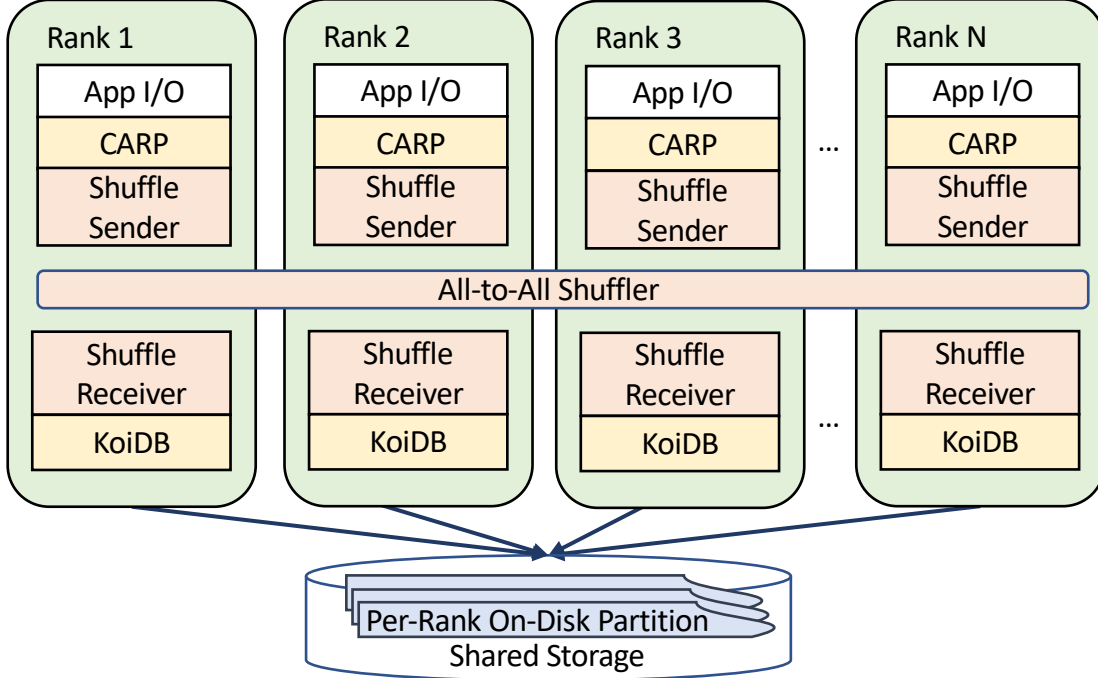


Figure 11: Example of an N -rank application using CARP. Writes are intercepted and partitioned via an all-to-all shuffler. Each rank is both a sender and receiver. Partitioned data collected by receivers is stored by a local storage backend (KoiDB).

7.5 Design of CARP

In this section we provide an overview of CARP’s architecture. We describe the scalable way data is routed to ranks by key (§7.5.1), the triggers that allow CARP to determine that data needs to be repartitioned (§7.5.2), the protocol that allows ranks to determine a new partitioning for the data (§7.5.3), and a reference storage backend design for logging partition data efficiently to storage (§7.5.4).

7.5.1 Communication: Scalable data flow

Figure 11 shows the architecture of CARP using an example parallel application with N ranks. Application data is intercepted by CARP as a stream of records, one per each particle or cell. Each application rank owns one partition and acts as both a producer and a consumer of data. These records are routed to the corresponding partition on the basis of the key via an operation called *shuffling*. On the receiving end, the local partition is managed via a local instance of a storage backend called KoiDB (§7.5.4). This KoiDB instance is responsible for collecting data and writing it out to a per-rank on-disk log. For efficient shuffling, we leverage the scalable all-to-all communication overlay from the DeltaFS project [30], as it has been demonstrated to scale up to 131,072 ranks.

Due to the transmission delay introduced by shuffling, data written through CARP is only available to query at the end of a checkpointing *epoch*. At the end of each epoch, CARP flushes all

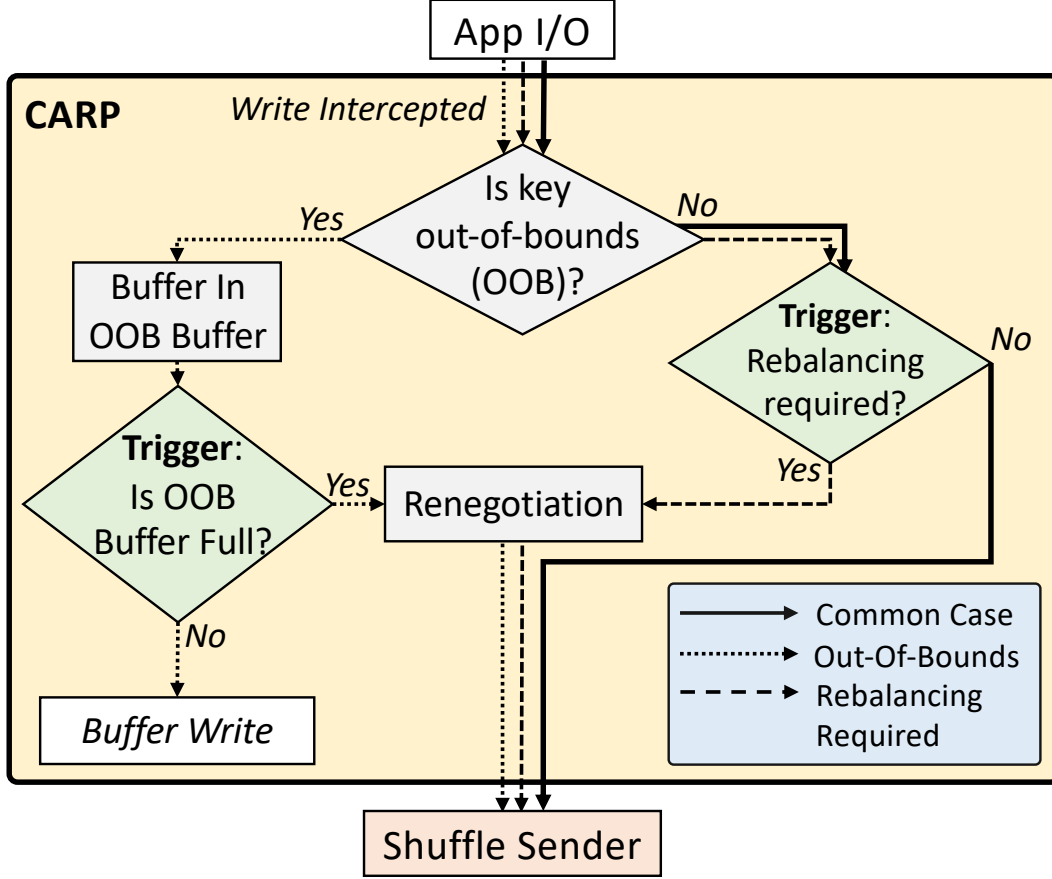


Figure 12: CARP data and control flow. Application data is routed by the shuffle sender to its partition. But, if its key is outside all partition bounds, then data is added to an Out-Of-Bounds (OOB) buffer. When the OOB buffer fills up or partitions become imbalanced, CARP renegotiates the partition table.

data to the disk. By doing so, it aligns its fault-tolerance semantics with those of typical scientific applications.

7.5.2 Detection: Triggering repartitioning

CARP uses a triggering mechanism to determine if the current partitioning of data is leading the system to a state of load imbalance (i.e., routing more data to a subset of the partitions). Imbalanced loads affect the write path by creating stragglers and affect the query path when larger partitions are traversed to return query data. Figure 12 shows the data flow from the application to the shuffle sender layer. This is the path that CARP monitors to determine whether to trigger data repartitioning. We describe CARP’s control flow through the three possible cases of data flow from the application to the shuffle sender layer below.

Common Case. In the common case CARP shuffles data as per the assigned partition ranges as described previously in §7.5.1. Collectively these ranges form a *partition table* that is replicated across all ranks. A distribution of keys transmitted by each rank is stored locally and used when

it becomes necessary to participate in a global renegotiation of the partition table (§7.5.3.1).

Out-Of-Bounds. As simulations progress new keys are generated. If a rank is asked to insert a key that is currently out of the partition table bounds, then there is no valid destination for the data. In this case CARP temporarily buffers the data in an in-memory buffer on the sender called the *Out-Of-Bounds (OOB) Buffer*. Once the OOB buffer reaches a predetermined threshold, a renegotiation of the partition table is triggered.

Rebalancing Required. This trigger is only needed to adapt to distribution changes *within* an epoch — for new epochs CARP bootstraps partitions from scratch. To resolve intra-epoch drift, a renegotiation should be triggered to compute a new and more relevant partition table. Instead of using background communication mechanisms to robustly detect load outliers, we have found it simpler to assume that a rebalancing is required at periodic intervals within an epoch. Some applications such as AMR (Adaptive Mesh Refinement) codes are aware of when they refine and can signal CARP for more precise control over renegotiation. We explore the impact of a fixed interval further in our evaluation.

7.5.3 Renegotiation: Determining new partitions

The goal of renegotiation is to rebalance the partition boundaries used by the shuffle layer to route data to its destination. Renegotiation replaces the partition table stored in each rank with a new more balanced one based on the latest statistics of the global distribution of the keyspace. Renegotiation is similar to distributed snapshotting algorithms such as *Chandy-Lamport* [102] but is designed to trade off some accuracy for scalability and performance. As a result, the computed global distribution is not a perfect representation of the actual distribution, but our evaluation shows that any estimation errors result in negligible imbalance in partition load. Our scalable implementation of renegotiation is described in §7.6. The steps involved in the renegotiation protocol are as follows:

1. A rank triggers a renegotiation round by notifying all ranks to start the process. This pauses shuffling.
2. Local distribution estimates are computed on all ranks and then collected and merged on rank 0 to form an estimate of the global distribution.
3. A new partition table is computed and broadcast.
4. All ranks switch to the new partition table and flush their Out-Of-Bounds buffers according to the new partitions.
5. Ranks reset local distribution statistics, resume shuffling.

Local distribution estimates are computed and merged using summary statistics primitives. Renegotiation is initiated by trigger that can fire on any rank. We describe CARP’s summary statistics primitives and trigger design next.

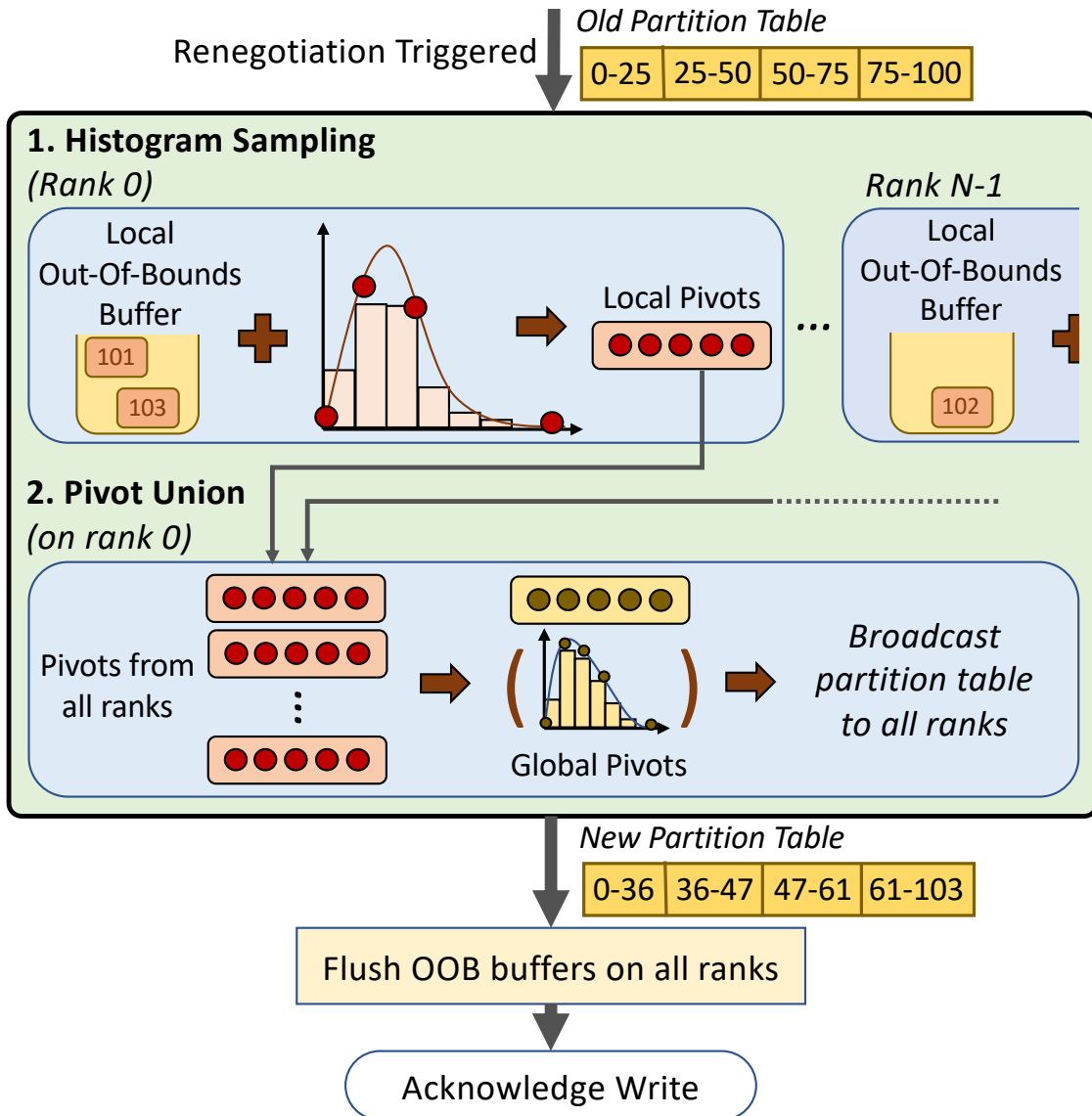


Figure 13: Partition table renegotiation relies on pivots computed from histograms of keys tracked on each rank. Pivots are merged, and the new global histogram is divided into equal-mass bins before being broadcasted.

7.5.3.1 Summary Statistics

CARP ranks use summary statistics primitives to construct the global distribution during renegotiation. We use *histogram-based sampling* to track and aggregate ranks’ local key distributions. Different quantile estimation techniques can be plugged into CARP, but we have found that histogram-based sampling is efficient to compute and allows for control over trading compactness for accuracy. CARP tracks rank-local key distributions using histograms. Each rank’s histogram consists of one bin per application rank (or partition). For each processed key the corresponding bin counter is incremented. This histogram represents a lightweight, lossy representation of the local key distribution. When a renegotiation is invoked the global distribution is constructed using two primitives: histogram sampling and pivot union, as shown in Fig. 13.

When ranks are notified of a renegotiation round, they compute *pivots* via the **histogram sampling** primitive. Pivots are a compact and lossy representation of a distribution that can be efficiently computed, communicated, and merged. Pivots are a set of k points in the keyspace that divide the histogram into k partitions of equal mass, i.e., bins containing the same number of samples. Increasing k reduces representation lossiness but requires more network communication for the additional information. For skewed distributions, this results in more pivots concentrated in histogram regions where bin counts are high and vice versa. Pivots are calculated by linear interpolation between bin boundaries. We also factor in the keys in the local OOB buffer for pivot computation.

Pivots from all ranks are collected at a designated rank (Rank 0) and aggregated to form pivots representing the global distribution via the **pivot union** primitive. This operation uses the information captured in rank-local pivots to construct a global distribution. This global distribution is then resampled to compute the new partition table that is broadcast to all ranks.

7.5.3.2 Triggers

Renegotiation triggers are evaluated at each rank to determine when to renegotiate, as shown in Fig. 12.

The **Out-of-Bounds trigger** is used to extend partition boundaries when many keys outside the partition table boundaries are encountered. We partition the known keyspace without any gaps, so an invocation of this trigger strictly extends the allocated keyspace. An in-memory *Out-of-Bounds* (OOB) buffer is used on each rank to temporarily store keys that are currently outside the bounds of the current partition table. The OOB buffer has a predetermined size and when it fills up a renegotiation is triggered. The contents of all ranks’ OOB buffers are factored into the new partition table so that the newly computed table has destinations for those keys. Keys in the OOB buffers are flushed to their respective destinations once the renegotiation completes. We have found OOB buffers with a capacity of 512–1024 items per rank sufficiently effective.

This trigger is also used to bootstrap CARP at the beginning of each epoch. As there is no partition table when an epoch begins, all ranks collect writes into their OOB buffers, and invoke a

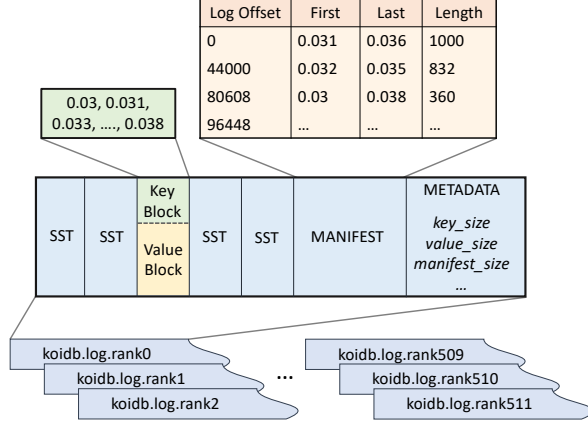


Figure 14: KoiDB on-disk log format. Each log consists of SSTables organized into contiguous key and value blocks. Each SST has an entry in the manifest along with metadata.

renegotiation to find the first partition table to start shuffling with.

The **Rebalancing trigger** is used to account for key distribution drift over time causing partition load to become imbalanced. In principle this trigger could be designed to fire only when key distribution drift is detected, however we have found that invoking it periodically is both simpler and effective. We found frequent fixed-interval renegotiation to provide effective partitioning without having a measurable runtime overhead (§7.7.3.4).

7.5.4 Storage: Logging for efficiency

KoiDB is our reference implementation of CARP’s storage backend. Its output is written to a parallel file system and can be directly queried by a query client on a single node. As query clients access files in read-only mode, multiple concurrent query clients are automatically supported. KoiDB collects data from the shuffle receiver and writes it to an append-only log on shared storage. KoiDB instances are local to each rank and operate independently of each other. We discuss using CARP with other storage backends in §7.8.

Fig. 14 shows KoiDB’s on-disk log format. KoiDB produces this output by first collecting shuffled records in a memory buffer. When the buffer fills, KoiDB compacts the data into an *SSTable* (or SST) that is then appended to the log. Compaction operations include (optionally) sorting the contents by key, serializing the keys and values into separate sub-blocks for more efficient query-time parsing, and writing the serialized SST to storage. A manifest entry containing the SST’s key range and location is also written into a manifest block that is used to find relevant SSTs for queries. Storing this manifest results in a small space amplification (~.01%). KoiDB uses double-buffering to allow compaction to run in the background while shuffling continues in the foreground.

We now describe two additional optimizations KoiDB applies to SSTs for query performance:

Repartitioning to refine SSTs. CARP partitioning quality can be degraded by *stray keys* created when CARP partition tables are updated. These keys end up being misdelivered because

their correct shuffle destination changes between dispatch and delivery due to renegotiation. Letting them be written to the wrong partition reduces SST selectivity and will increase query latency as more SSTs need to be read for each query. One way to avoid stray keys is by flushing the network before each renegotiation, but this increases the cost of invoking the primitive. KoiDB mitigates this by simultaneously maintaining multiple open SSTs and separating stray keys into a different SST. As we show in §7.7.3.3, this improves the selectivity of CARP partitions by up to $48\times$.

Subpartitioning to create smaller SSTs. To further reduce the read amplification for highly selective queries, KoiDB can subdivide a rank’s SSTs into smaller ranges before writing them out. Smaller SSTs can be retrieved faster, but a large number of subpartitions can also increase CARP’s CPU and metadata overhead and adversely affect runtime.

7.6 Implementation

In this section we describe CARP’s implementation and its relationship to scalability, memory footprint, and runtime overhead.

CARP [31] consists of $\sim 10,000$ lines of C/C++ code. Application writes are intercepted using a dynamically preloaded shared library. The CARP API can also be called directly without using a preload library. CARP uses two instances of the DeltaFS shuffle service [103] for communications: the *data* instance batches messages into 32KB buffers for high throughput, while the *control* instance is used for renegotiation control messages and uses no batching for low latency. The shuffle service is built using the Mochi [104] Mercury RPC library [83]. We use RPCs as both shuffling and renegotiation are asynchronous operations better suited to RPC semantics (it also allows us to avoid complications with multiple concurrent MPI communicators). CARP creates its own threads for network communication, RPC callbacks, and asynchronous compaction in KoiDB. This setup allows straggler ranks to use multiple cores and provides some tolerance for load imbalance.

Scalability. CARP is designed to scale by composing scalable operations — the shuffle service has been demonstrated to scale to 131,072 ranks [30], and the renegotiation protocol is designed as a reduction operation with a logarithmic scaling factor (evaluated in §7.7.3.1). Each instance of CARP’s storage backend (KoiDB) is an independent instance and can scale trivially. By default, CARP creates one file per rank, but at larger scales, the total number of files can be reduced (if needed) by having a subset of ranks be shuffle receivers.

We call CARP’s scalable implementation of renegotiation the *Tree-based Renegotiation Protocol (TRP)*. A naive implementation of renegotiation, as described in §7.5.3, would require directly collecting all ranks’ pivots on one rank before computing the new partition table. Such an implementation will have limited scalability, as its memory and network communication footprint will scale linearly with ranks. Our TRP implementation, on the other hand, uses a reduction tree-based design [105, 106] which scales logarithmically.

TRP is built on the observation that pivot unions (§7.5.3.1), being associative and commutative have all the properties of a lossy reduction operation. Pivots represent distributions and can be merged in any order, but the pivots lose some information in the process. To limit the impact of

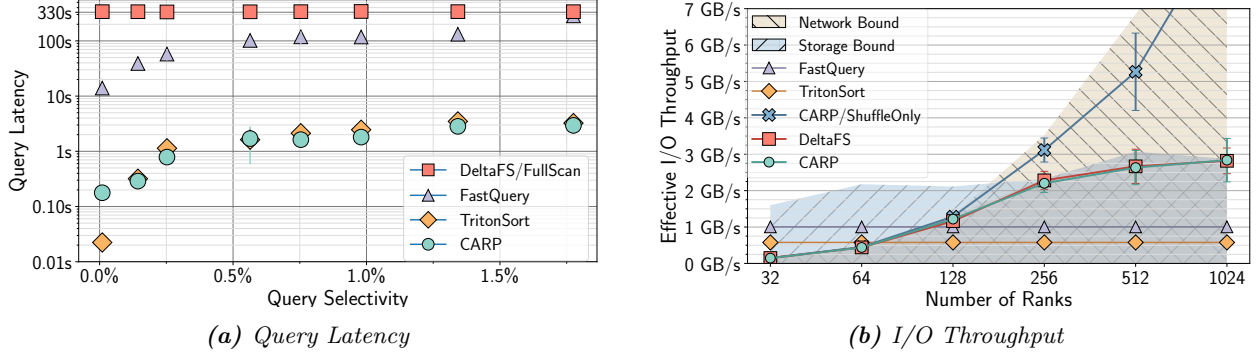


Figure 15: (Left) CARP matches the query latency of TritonSort’s fully-sorted data layout and outperforms both FastQuery by 100 $\times$. (Right) CARP incurs no overhead over unpartitioned I/O and is 3-5 $\times$ faster than post-processing.

this lossiness, TRP employs a shallow tree hierarchy with a large fan-out (depth of 3, fan-out of up to 64). The tree’s leaf layer consists of all CARP ranks, while intermediate layers consist of equally spaced ranks. Subsets of pivots are reduced on intermediate layers, and a final reduction happens at the root.

Memory Footprint. As CARP runs in application processes, it needs to have a low memory footprint to avoid competing with the application for memory. CARP achieves this by streaming application data directly to the shuffle pipeline. The primary source of memory overhead is due to buffers used to batch I/O for network and storage efficiency. To illustrate this overhead, we use a run with 4096 ranks and default CARP parameters that results in 27MB per rank¹. This is less than 1% of the per-core memory budget on the LANL Trinity supercomputer [107].

Runtime Overhead. CARP only runs during application I/O, so it does not impact the compute phase. Data shuffling and triggered renegotiations are potential sources of additional runtime overhead. Shuffle bandwidth increases with the number of writes and quickly outstrips storage bandwidth in our experiments. Thus, the shuffle service itself does not have any runtime overhead. CARP could underutilize storage by pausing I/O for renegotiation, but our experiments show that shuffle receivers buffer enough outstanding writes to keep storage busy and mask this overhead. If needed, CARP can continue routing data using the old partition table while a renegotiation is underway, however we have not found this necessary in our experiments.

7.7 Evaluation

Baselines. We use DeltaFS [30], FastQuery [98], and TritonSort [24] as baselines representing the state of the art from different research communities. DeltaFS represents the state of the art for write-efficient online hash-partitioning, FastQuery employs auxiliary indices, and TritonSort performs data reordering.

¹Each rank uses 2MB for shuffle RPC buffers [30], 24MB for two KoiDB memtables, 16KB for 4096*4B partition table entries, 16KB for partition shuffle counts, and 32KB for 512-entry OOB buffer with 64B records.

Benchmark workloads. We evaluate the performance of CARP using traces collected from a 512-rank VPIC simulation [29], a popular particle simulation code described in §7.3, and YCSB [108], a standard benchmark suite with varying key-value workloads. To eliminate variability across simulation runs and ensure reproducibility, we replay traces captured from VPIC rather than using it as a streaming workload. We use the computational plasma trace described in section 7.3 for all our experiments. We index VPIC traces using energy as the key. This results in a 4 byte floating-point key and a 56 byte record for the rest of the particle data.

Experimental setup. We use a 32-node compute cluster with each node having two 8-core Intel Xeon E5-2670 CPUs and 64GB DRAM. The nodes are connected via 40Gbps Infiniband QDR. Simulation output is written to a shared Lustre storage cluster consisting of 20 nodes identical to those in the compute cluster, each having a 240GiB SSD. DeltaFS, FastQuery, and CARP experiments use the compute cluster, while TritonSort runs directly on the Lustre nodes. TritonSort has identical storage resources as other approaches and is able to achieve higher throughput out of storage by not incurring the overhead of Lustre’s coordination. While TritonSort appears to have lower compute resources, all applications are bottlenecked by the storage bandwidth and their runtime is dictated by their write amplification. FastQuery provides a number of tunable parameters, and we report the numbers from the best performing parameters on our cluster.

The rest of this section is organized as follows: In §7.7.1 we evaluate CARP’s query performance. We evaluate CARP’s runtime overhead and compare it with existing approaches in §7.7.2. Finally, in §7.7.3, we show selected results of a sensitivity analysis aimed at understanding the impact of tunable parameters and optimizations on CARP’s runtime overhead, load balancing effectiveness, and index quality.

7.7.1 Query Performance

In this subsection we evaluate CARP’s partitioned output using two aspects: query latency and read amplification. Query latency is a function of how much data the index needs to read and how efficient the storage layout is in serving that data. For CARP, minimum effective query selectivity is capped by the size of one CARP partition — 0.18% (or $\frac{1}{512}$) for 512 ranks. This percentage decreases with scale as the number of partitions increases and when subpartitioning is enabled. We now discuss CARP’s query performance over two query suites: one constructed by us, and a YCSB query suite.

Observation 1: *CARP can serve most queries as efficiently as a fully sorted layout and 1-2 orders of magnitude faster than state of the art auxiliary indexing approaches.*

Query Suite. For query latency experiments we index 12 timesteps from the simulation. Each timestep is 188GB of data, and the total simulation data is 2.2TB. Query latencies for eight queries with different selectivity are shown in Fig. 15a (each query is repeated 3 times and averaged). We compare the query latency for CARP’s partitioned ordered output with that of FastQuery and a sorted, clustered index. We also provide numbers for a *full scan* over unindexed data for

reference. The sorted output uses 12MB SSTs and a manifest with one entry per SST, a format similar to KoiDB. We refer to the sorted, clustered index as TritonSort for convenience, but all sorts generate identical outputs. Queries for both CARP and TritonSort are processed by the same query client. The manifest is read first to find corresponding SSTs and then key blocks of corresponding SSTs are read in parallel. CARP SSTs overlap and must be merge-sorted for ordered range query semantics. The latency numbers for CARP include this sorting cost. The query client for CARP and TritonSort is run on a single compute node with access to the Lustre filesystem. FastQuery numbers are measured using its own query client reading from its natively supported HDF5 format. All query latency numbers include the time taken to fetch keys matching a query from a storage cluster to a query client and the compute time to subsequently process/filter the data fetched.

For both CARP and TritonSort, query latency appears to be linearly proportional to the amount of data read. Despite incurring an additional sorting cost, CARP’s total query latency is similar to TritonSort. CARP is slower for highly selective queries (177ms vs 22ms for a query with 0.01% selectivity) because it is forced to read full partitions, but it is able to match (and even outperform) TritonSort for larger query ranges with selectivity $> 0.05\%$. FastQuery is much slower than either of the two approaches. This is partially due to it needing to read large bitmap indices, but largely due to it being an auxiliary index, requiring small, random reads.

Observation 2: *Despite incurring extra sorting overhead, CARP matches TritonSort’s query latency performance for all except extremely selective queries ($< 0.05\%$ selectivity).*

YCSB Benchmark Suite. To thoroughly evaluate the query performance of CARP’s approximate partitioning we use Workload E from YCSB (Fig. 16). Workload E consists of short range queries and inserts in a 19:1 ratio. The range queries consist of up to 100 keys generated from a Zipfian distribution and are reordered by a random hash function. We drop inserts from our benchmark suite as CARP and TritonSort are not online stores but are instead transient services with different insert and query pathways. We define the workload’s ranges in terms of fully ordered SSTs. We interpret YCSB query ranges as SST numbers in TritonSort’s output and translate them into equivalent key ranges that we use for both CARP and TritonSort. The range SST# (251, 350) would therefore query the key range stored in TritonSort SSTs numbered 251 to 350. Since KoiDB SSTs are smaller and more fragmented, the same query with CARP would result in more SSTs read.

We construct 4 query batches of different fixed widths (5, 20, 50, and 100 SSTs) corresponding to query selectivity of 0.03%–0.6%. The starting index is drawn from YCSB’s Zipfian distribution in the range SST# (0, 18266), where 18266 is the number of SSTs in each timestep. We use 16 I/O threads to fetch SSTs for each query in parallel. For each width, we construct a batch of 1000 queries per timestep with the order randomized by YCSB’s hash function (`fnvhash`). Fig. 16 compares the total time taken by a batch of queries for two different timesteps and 4 query widths for each timestep, and it reaffirms the trends from Fig. 15a. CARP’s median selectivity of 0.07% (with 4-way KoiDB subpartitioning enabled) enables us to be competitive for queries with a width of 20 SSTs and greater. For larger queries, the cost of sorting starts to become a greater proportion

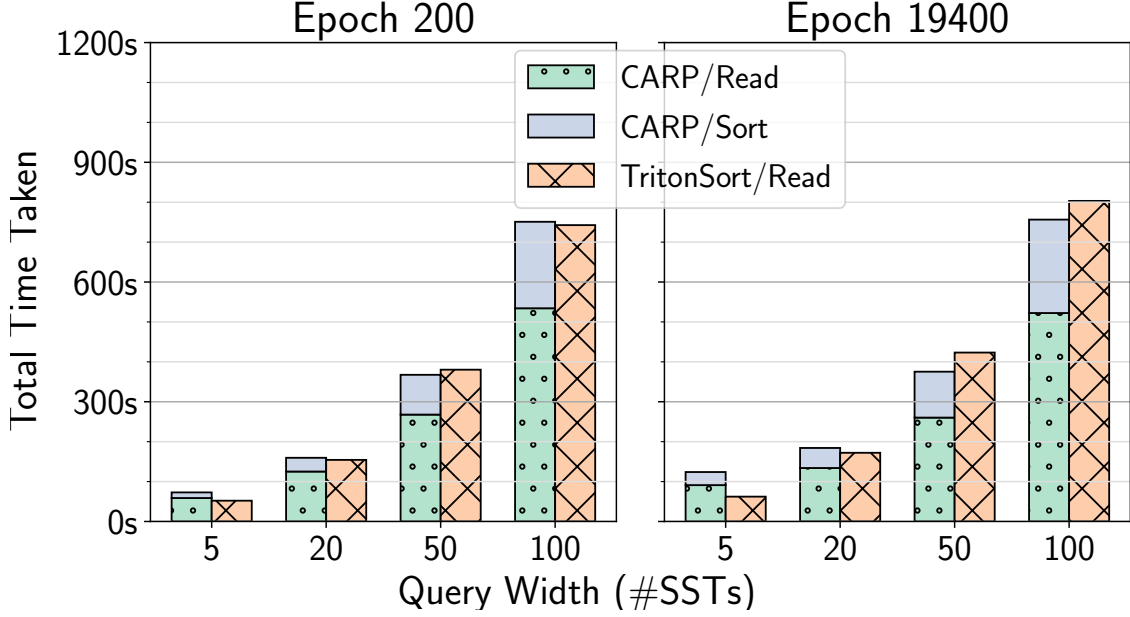


Figure 16: Time taken for a YCSB query suite, demonstrating similar trends to the left-hand figure. CARP is slower for highly selective queries but comparable/better for larger queries despite the sorting overhead.

of CARP’s query latency, but the aggregate latency is still close.

A surprising takeaway from these latency experiments is that CARP’s partial ordering seems to result in the most efficient storage layout. CARP reads are faster than the fully sorted layout, and query performance is similar despite us incurring the extra overhead of merging SSTs. We attribute this to our layout being amenable to parallel reads from different storage nodes of a parallel filesystem — it has enough contiguity to be read efficiently vs small random I/Os, but is distributed enough to allow for parallel processing of a query.

7.7.2 Write Path

Observation 3: CARP is 2.8–4.9× faster than post-processing approaches with no overhead on application performance.

Fig. 15b measures the effective I/O throughput of four different index building approaches ingesting 188 GB of VPIC data (corresponding to 3.5B particles or one timestep at 512 ranks). The effective I/O throughput is computed by dividing the application data volume by the total runtime. For in-situ approaches, their runtime overhead is included in the application runtime, while for post-processing approaches, it is obtained by adding the post-processing time. Each run is repeated 9 times and averaged.

To measure I/O throughput, we keep the total amount of data generated constant across different scales as the write performance of SSDs decreases as they fill up. Allowing the amount of data written to scale up with the number of ranks results in higher scales unfairly getting a lower storage throughput. The achievable storage bandwidth from our cluster (*Storage Bound* in Fig. 15b) in-

creases with the number of application ranks: from 1.6 GB/s at 32 ranks to saturating the storage with 3 GB/s at 512 ranks. At 1024 ranks, the increased contention from a large number of parallel writers causes a small dip in achievable throughput. For 1024 ranks, we divide the 512-rank trace into two halves to keep the total data constant.

We compare CARP with DeltaFS, FastQuery, and TritonSort. CARP and DeltaFS are embedded within the application and hence are evaluated at different scales. FastQuery and TritonSort are post-processing approaches and are run after VPIC completes. For the post-processing approaches, effective throughput is obtained by dividing the total data written by the total time taken (VPIC time and post-processing time). For the in-situ approaches VPIC time includes the online index building time.

FastQuery and TritonSort. As shown in Fig. 15b, the additional time taken by post-processing approaches (FastQuery and TritonSort) significantly reduces the effective application bandwidth from 3GB/s to ~ 1 GB/s and ~ 0.6 GB/s respectively. This represents a slowdown of $2.8\times$ for FastQuery, and $4.9\times$ for TritonSort. TritonSort incurs a much larger slowdown as it creates a clustered index, which requires four passes over all data for out-of-core sorts. FastQuery scans the data once, creates index structures, and writes them to storage, but takes an additional 24% space to store indexes for a single attribute.

DeltaFS. As discussed in §7.2, DeltaFS embeds with VPIC and reorganizes data via an all-to-all shuffle while it is in transit from the application to storage. DeltaFS uses the same 3-hop shuffle used by CARP (§7.5.1). DeltaFS uses a hash of the particle ID to partition incoming data. At smaller scales, DeltaFS is bound by the available shuffle throughput (*Network Bound* in Fig. 15b). As the aggregate shuffle throughput increases with scale, DeltaFS is able to fully saturate the storage layer, incurring no overhead over raw VPIC throughput.

CARP. We measure CARP network overhead (*CARP/ShuffleOnly*) and end-to-end performance (*CARP*) separately in Fig. 15b. For *CARP/ShuffleOnly*, we drop data once it is received at a shuffle receiver so as to not be bound by storage performance. We see that *CARP/ShuffleOnly* scales with the available shuffle bandwidth, incurring a small overhead for renegotiation rounds and the residual load imbalance. However, this overhead does not matter for the end-to-end performance as CARP quickly becomes bound by the available storage bandwidth. When the network bandwidth exceeds storage bandwidth, CARP has no impact on end-to-end performance. By avoiding post-processing, CARP enables indexes to be built $2.8\text{--}4.9\times$ faster.

Observation 4: *Static partitioning can quickly devolve to reading orders of magnitude more data than CARP by not adapting to key distribution drift.*

Fig. 17 is a study of how frequently partition tables need to be recomputed for our VPIC trace to generate balanced partitions. We construct partitions from a perfect knowledge of different simulation timesteps and measure how well they fit subsequent timesteps. The green line simulates a static partitioning approach where partitions are computed from a perfect knowledge of the first timestep but are not changed afterwards. The load balance of a static partition scheme worsens as the key distribution drifts over time. Next, we measure how well a partition table from the

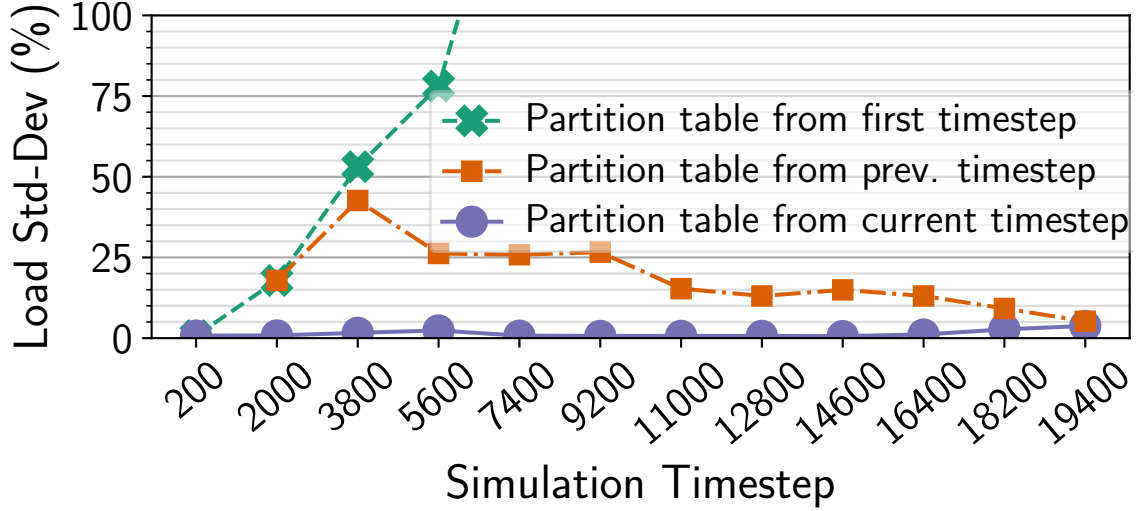


Figure 17: Simulated performance of different static partitioning schemes in generating balanced partitions (lower load std-dev implies better partition balance). Reusing a partition table from the first timestep results in the worst load-balance. Tables from the previous timestep fare better, but perform poorly when the simulation is the most active. Tables from the current timestep fit the best (true by definition).

previous timesteps fits the current timestep. This works better but still demonstrates a significant load imbalance around timestep 3800 when simulation entropy is high. As the simulation converges and the entropy between adjacent timesteps reduces, reusing older partition tables fares better. Finally, we see that partition tables obtained from the current timestep fit that timestep very well. This is true by definition, and the minor load imbalance arises from the lossiness of the histogram approach to capture distributions.

We conclude that any partitioning approach should recompute its tables at least once every timestep, more if there is intra-timestep entropy. We emphasize that the purple line in Fig. 17 represents an upper bound on CARP’s load balance, as the partitions used for this benchmark are *oracle partitions*. Since CARP partitions data as it arrives, it does not have the required information to compute these.

7.7.3 Sensitivity Analyses and Microbenchmarks

7.7.3.1 Renegotiation Protocol Scalability

Fig. 18a shows the time taken by a single renegotiation round across different scales and pivot counts. As described in §7.6, we implement renegotiation using a reduction tree and hence expect it to scale logarithmically. Fig. 18a shows the logarithmic scalability of our implementation from 16 to 2048 ranks for 6 different values of the pivot count parameter. Increasing the number of pivots computed increases the size of the messages exchanged. This results in proportionally higher round latency. 512 pivots are sufficient for CARP to accurately track distributions (discussed below), and the latency of a single round is only 150ms even at 2048 ranks.

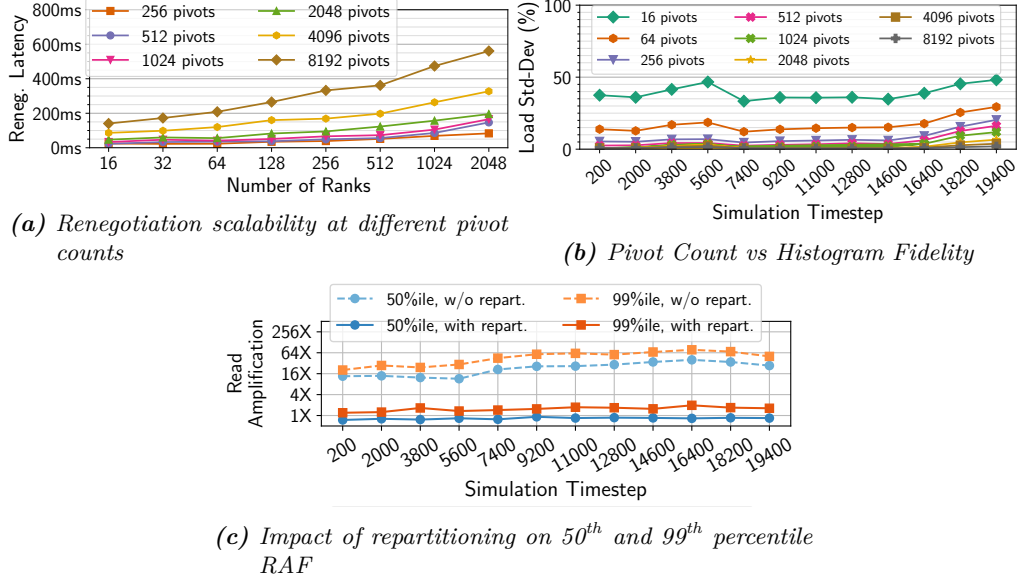


Figure 18: (Top) Renegotiation scales logarithmically with the number of ranks and takes longer if more pivots are exchanged. (Center) 512 pivots enable reasonably accurate reconstruction for even highly skewed distributions, with diminishing returns after. (Bottom) Both median and tail RAFs are significantly lowered by KoiDB partitioning, from 16-64 $\times$ to 1-2 $\times$, across all timesteps.

The absolute values of renegotiation latency are handicapped by us using an emulated network stack (IPoIB) for a fair comparison with baseline approaches that use sockets. We expect these numbers to be much lower if run natively on a modern interconnect.

7.7.3.2 Impact of Pivot Count

Fig. 18b shows the results from a micro-benchmark that tests the lossiness of our pivot calculation scheme. Pivots are an approximate representation of a distribution. The higher the pivot count, the better the approximation. We compute oracle pivots from a full key distribution of each of the 12 different timesteps for different pivot counts and check how well the partition table calculated from those pivots fits that timestep’s keys. We use standard deviation in partition load (**std-dev**) as a proxy for the lossiness of the pivot calculation. A lossless scheme would result in perfect partitions, resulting in zero **std-dev**. Higher **std-dev** implies more lossiness.

In Fig. 18b we see that higher pivot counts lead to lower load imbalance with diminishing returns beyond 256 pivots. We also see that the last two timesteps are harder to reconstruct than the others. This is because of the extremely skewed nature of those timesteps’ distributions (Fig. 9). All VPIC timesteps have long tails, but when they become extremely long (towards the end of the simulation) more pivots are required to accurately capture those distributions.

7.7.3.3 Impact of KoiDB

We now summarize how KoiDB (§7.5.4) improves partition quality. We introduce a measure called *Read Amplification (RA)* to measure CARP partition quality. We define read amplification as the

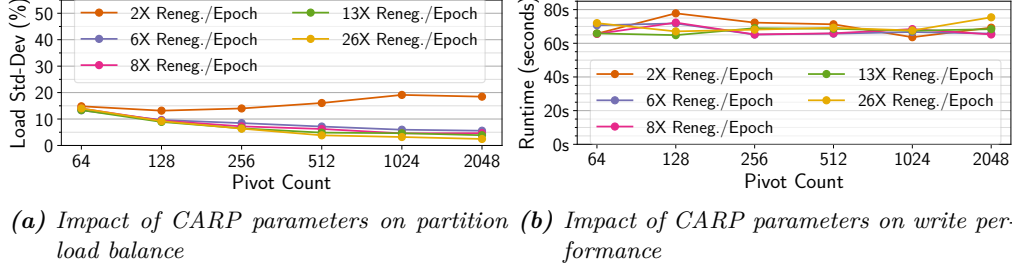


Figure 19: (Top) Renegotiating multiple times within a timestep is necessary for load-balance, but provides marginal benefit beyond a point. More pivot counts produce a better load balance, with diminishing returns. (Bottom) Up to 1024 pivots, increasing the pivot count leads marginal improvement in runtime. Renegotiating frequently does not penalize runtime — the cost of frequent renegotiations is made up by the gains in load balance.

ratio of the size of actual CARP partitions vs (hypothetical) perfectly balanced partitions. An RA of $1\times$ for all partitions is ideal, as balanced partitions are better on both the write and the query paths. It is also unachievable as CARP tries to predict future key distributions and partition in a single pass, which is inevitably an imperfect operation.

We find that repartitioning, by separating out stray keys, reduces the average RA by up to $48\times$ (Fig. 18c). For *subpartitioning*, we find that 2-way and 4-way subpartitioning improve average latencies for highly selective queries by 28% and 43% respectively with no observable runtime overhead. We omit detailed results for subpartitioning as we have observed its impact to be largely predictable, as described in §7.5.4.

7.7.3.4 Tuning CARP

We now summarize key takeaways on the performance impact of CARP’s two main tunable parameters: renegotiation frequency and pivot counts (Fig. 19). We vary renegotiation frequencies between $2\times/\text{epoch}$ to $26\times/\text{epoch}$ and pivot count from 64 to 2048. We then measure CARP’s partition load balance and application runtime performance. Note that it is important to optimize partition imbalance even if it does not affect runtime to provide more predictable and uniform query performance.

We find that these parameters have a noticeable impact on partition load balance but not on CARP’s shuffling runtime. At the lowest values of these parameters (64 pivots and $2\times$ renegotiations per epoch), CARP partitions have a normalized standard deviation of 14%. At the highest values (2048 epochs and $26\times/\text{epoch}$) this drops down to 2%. More pivots provide noticeable load balance improvement up to 512 pivots and then returns diminish. It is beneficial to increase the renegotiation intervals from $2\times/\text{epoch}$ to $6\times/\text{epoch}$, but we get minimal gains beyond that interval. Surprisingly, none of these parameters seem to impact runtime in any measurable way. We attribute this to CARP’s imbalance-tolerant implementation (idle ranks can cede CPU time to straggling ranks, §7.6).

Tuning Conclusion. While CARP can be tuned for incremental performance benefits, it

provides stable performance across a wide range of parameters. Even with the worst performing parameters, CARP partitions only deviate by 14% on average. CARP will still provide excellent query performance for queries not reading from larger partitions. A moderate number of pivots (~512) can represent even extremely skewed distributions with long tails, and renegotiation frequency has marginal impact on partition quality as it only addresses intra-epoch drift.

7.8 Discussion

In this section, we discuss how CARP can be adapted for different query types, analyses, and architectures. We also describe how CARP can interoperate with other storage formats and indexing techniques.

Multi-attribute Queries. In this paper, we focus on using CARP to build a sorted, clustered index on a preferred attribute. For applications such as VPIC, indexing on a single attribute is sufficient to identify and retrieve points of interest (high-energy particles), and apply subsequent transformations in memory. However, CARP can be extended to build sorted, auxiliary attributes on additional attributes. This would require CARP to track distributions of all configured attributes and use a two-stage shuffling pipeline as described below:

- First, the entire row is shuffled using the primary attribute as usual. The receiver serializes all rows, assigns them a unique `row_id`, and writes them to the local storage backend. Each receiver then computes a `(key, partition_id, row_id)` tuple for each additional indexed attribute and shuffles it via the same pipeline.
- Each receiver of the auxiliary attribute tuples writes them to separate storage backend instances, where each row points to the full row on the primary key partition.

The additional attributes will not have the query performance of the primary attribute but will still benefit from the space efficiency and index lookup performance of sorted indices (vs bitmap indexes).

Applications and architectures. CARP can be used to ingest data from any parallel application where storage bandwidth is the bottleneck. The assumptions we make are typical of such analysis workflows:

1. Data is not modified once ingested.
2. Spare network bandwidth is available for shuffling. This is true by definition for any storage-bound workflow and holds for most HPC clusters. The Aurora cluster [109] at ANL, for example, has a bisection bandwidth of 690 TB/s, more than 20× the storage bandwidth (31 TB/s).
3. Write performance is paramount, and the goal is to provide acceptable query performance without compromising write performance. Where the goal is optimal query performance (as with a web service), a distributed database is more appropriate.

Storage Formats. Developing our own storage backend allowed us to identify the most important properties for an efficient storage backend and on-disk format. CARP-partitioned output can be directly written to columnar formats like Parquet. This would automatically accelerate

queries, as Parquet maintains statistics (min/max etc.) on each rowgroup, and CARP-partitioned rowgroups would have a tighter range and require less I/O at query time.

However, as we show in §7.7.3, repartitioning on shuffle receivers significantly improves partition quality. A writer that can receive repartitioning hints from CARP and use them to further optimize data before writing to Parquet or other storage formats can improve query latencies by 1-2 orders of magnitude.

Indexing Techniques. As an alternative to auxiliary CARP indices as described above, it is also possible to combine CARP with other indexing techniques. This also requires query engines that can generate efficient query plans by leveraging the strengths of different indexing techniques. We provide some examples below:

1. Different auxiliary index structures (such as bitmap indexes) can be built either in-situ on auxiliary nodes, or as a post-processing stage.
2. On the query path, raw-data indexing approaches can further improve index quality incrementally, in response to user queries.
3. CARP’s approximately sorted output can be incrementally converted into a fully sorted layout on the query path by writing back the merged SSTs that are computed for user queries.

To leverage the full potential of different indexing techniques, it is necessary to develop end-to-end analysis engines that can understand user analysis requirements and generate an appropriate combination of in-situ embedded, in-situ auxiliary, and (if necessary) post-processing transformations using different techniques.

7.9 Related Work

Data structures such as LSM-Trees [110], B+ Trees [111], and their variants [112, 113] are employed by online databases to order data. These data structures heavily reorganize data internally and are more inefficient at writes than bulk post-processing approaches [101]. Multiple in-situ analysis frameworks have been proposed including PreData [114], GLEAN [115, 116], NESSIE [117], DataSpaces [55], and ADIOS [118]. These systems are designed such that auxiliary nodes are used to perform analysis tasks. ADIOS supports in-situ generation of range query indices using FastBit [119] (the same bitmap index used by FastQuery) using auxiliary nodes. In-situ generation of bitmap indices would be faster than post-processing via FastQuery, but at the cost of dedicated resources, and the space overhead and query performance limitations of bitmap indices would still remain.

Usher et al. [120] describes an in-situ indexing system that aggregates application data into spatial locality-preserving data layouts. Their approach requires provisioning dedicated aggregator nodes which collect spatially-partitioned particle data from the application, add a bitmap index to the data, and write it to storage. Particle codes already have spatial partitions because of how they decompose the problem. This system only preserves these pre-existing partitions. CARP is able to create partitioning along arbitrary dimensions using all-to-all shuffling. Further, CARP does not require dedicated resources, introduces more powerful distribution monitoring constructs,

overlaps indexing and I/O for extremely high storage utilization, and confers maximum benefits of an in-situ partitioning approach. Since CARP does not require any dedicated resources, these two approaches can be composed together for richer partitioning capabilities.

Slalom [121] and VETI [122] describe raw-data indexing approaches. Raw data indexing works on the query path to exploit pre-existing order in data and adaptively reorder data in response to query patterns. Scalable variants of these techniques can complement approximate indexes on the write-path to further optimize query performance for exploratory queries, but they are not a substitute for in-situ indexing on the write path. WiscKey [123] introduces the notion of separating keys and values in LSM-Trees for lower write amplification, similar to our discussion on clustered vs auxiliary indexes. SuRF [124] can help reduce redundant reads for range queries with sparse keyspaces but can not help speed up queries for data with no keyspace gaps.

7.10 Conclusion

CARP demonstrates that a sorted, clustered layout for range queries over a parallel application can be generated in a single streaming pass without any impact on application runtime. This accelerates subsequent analyses regardless of the workflow employed — it can provide highly efficient queries by itself, or accelerate a query plan with auxiliary indices, or provide tighter partitions for columnar storage formats to exploit. Due to its low memory footprint, CARP can co-exist with other in-situ approaches running on the same system. End-to-end analysis stacks that produce an optimal combination of in-situ and post-hoc indexes for a given set of query requirements remain future work.

8 AMR Paper

Abstract

Block-structured Adaptive Mesh Refinement (AMR), while essential for improving efficiency in large-scale irregular and dynamic simulations, poses unique optimization challenges. Previous work has identified load imbalance and synchronization overhead as key obstacles to performance, but the deep understanding of complex runtime behavior needed to systematically address them remains elusive.

In this paper, we integrate telemetry collection, analysis, and intervention to bridge this understanding gap. Establishing reliable, actionable telemetry required systematic tuning to eliminate cross-stack performance anomalies. Leveraging this foundation we design CPLX, a tunable placement policy balancing compute load and communication locality, improving runtime by up to 21.6% over optimized baselines. Our experience highlights the empirical nature of placement optimization, requiring theoretical models to be grounded in observed runtime behavior.

8.1 Introduction

Block-structured Adaptive Mesh Refinement (AMR) codes are critical to modern scientific computing, enabling large-scale simulations in astrophysics, cosmology, climate science, and fluid dynamics [26, 71]. By dynamically refining the mesh around steep gradients, AMR delivers 1–2 orders of magnitude performance gains over uniform approaches [125]. These codes often run for weeks on hundreds of thousands of cores [62, 71, 126], making performance optimization essential. Load balancing remains a major challenge [26, 32]: while octree and space-filling curve (SFC) placements work for uniform workloads [71], they do not account for compute kernel variability. Parallel codes are limited by stragglers—ranks lagging significantly behind peers—whose delay propagates due to frequent global synchronization, and the fine-grained adaptivity of AMR exacerbates this effect. Prior analyses of proxy application traces [25] document significant MPI waiting time (>60% at 1000 ranks), while noting the limitations of standard trace data for the deeper, cross-stack diagnosis required to pinpoint root causes. The root causes, whether algorithmic or infrastructural, remain entangled. We ask: what changes when performance instrumentation has full visibility over a real AMR code?

To answer this, we sought to improve AMR placement using fine-grained runtime telemetry on a research cluster providing full-stack access. Our goal was to replace static heuristics with telemetry-driven workload models. We adopted an integrated workflow that combined telemetry collection, analysis, and intervention. However, initial experiments immediately revealed a deeper challenge: observed metrics often diverged unexpectedly from predictors such as message volume, undermining our ability to construct reliable workload models. Rather than directly enabling better placement, fine-grained telemetry first required us to diagnose and mitigate runtime artifacts to make the data useful for placement decisions (§8.3).

This paper describes the end-to-end process we undertook to develop telemetry-driven placement policies. We began by systematically diagnosing variability across the stack and tuning system parameters. This required extensive analysis of large volumes of telemetry, prompting a shift from standard tracing and profiling tools to a custom analytical pipeline over structured data (§8.4). Tuning the system stack resulted in predictable runtime response as placement aspects such as load balance and locality were varied. Recognizing that these two properties represented competing optimization objectives, we developed CPLX (§8.5), a hybrid placement policy balancing these two objectives via a tunable parameter to minimize overall runtime. Because AMR codes refine frequently, placement must meet strict timing constraints (< 50ms in our target codes). CPLX meets these requirements while leveraging existing octree and SFC infrastructure for compatibility with current AMR frameworks.

CPLX improves runtime by up to 21.6% over optimized baselines (§8.6) through its tunable control over the load-locality tradeoff, validating our process-driven methodology. However, our experience suggests that performance optimization in large-scale systems is inherently context-dependent. While algorithmic approaches are necessary, their application must be embedded in a broader empirical process—one that interprets telemetry in the context of the runtime stack and

prioritizes objectives only insofar as they measurably impact end-to-end performance. Accordingly, we focus not just on CPLX itself, but on the end-to-end process that led to it, and distill methodological lessons applicable to other large-scale tuning efforts. This paper makes three key contributions:

- An empirical case study documenting the challenges of achieving reliable, interpretable telemetry for AMR codes using full-stack access and iterative tuning.
- A novel placement policy, CPLX, that provides tunable control over the tradeoff between compute balance and communication locality (§8.5).
- Broader lessons on telemetry-driven performance analysis and the empirical, context-specific nature of tuning in large-scale parallel systems.

We provide background on block-structured AMR codes and current placement approaches (§8.2), then outline the challenges of incorporating telemetry into placement decisions (§8.3). We present our methodology for obtaining reliable performance measurements (§8.4), followed by the design of our placement policies (§8.5) and their evaluation (§8.6). We then summarize the broader lessons drawn from our experiences (§8.7) before concluding with related work (§8.8).

All our policies and associated artifacts are open source [127].

8.2 Why Telemetry-driven Placement?

Block-structured AMR simulations running on $O(100K)$ cores face significant performance challenges due to computational variability [26, 128, 129]. Frequent mesh refinement necessitates periodic redistribution of work across compute resources. Stragglers, if left unaddressed, are particularly expensive due to frequent global synchronization operations—a single straggler can block the entire simulation at synchronization points, leading to poor cluster utilization [10].

Two complementary approaches exist for computational variability—balancing work among ranks to reduce variability, and asynchronous execution strategies to mask residual variability. While asynchronous runtimes [130–133] are commonly used to mask variability, significant time is lost waiting within MPI routines despite their widespread use ($>60\%$ at 1,000 ranks [25]). Addressing variability via placement is difficult—frameworks often lack meaningful per-block cost inputs, as predictive models are hard to create and codes often assume uniform costs. This gap motivates leveraging runtime telemetry to directly measure workload behavior.

This section provides background for the rest of this paper, beginning with an overview of AMR codes and current placement techniques in §8.2.1, followed by computational characteristics of AMR operations in §8.2.2.

8.2.1 Block-based AMR: Infrastructure and Execution

Structured AMR Codes. Structured AMR implementations organize computational grids hierarchically while maintaining a logically Cartesian structure. Implementations typically fall into two categories: *block-based* and *patch-based*. Block-based AMR partitions the domain into uni-

form sized blocks at each refinement level, managed using octree data structures. This approach enables efficient parallel implementations through simplified mesh management and communication patterns. Examples include Parthenon [62], Enzo-E/Cello [134], and ALPS [32]. Patch-based AMR, in contrast, refines regions using arbitrary rectangular patches rather than fixed blocks, allowing finer control over refinement but requires more complex load-balancing (e.g. AMReX [63], SAMRAI [135]).

Execution Model. Block-based AMR codes operate on structured meshes that are decomposed into mesh blocks, with multiple blocks typically assigned to each simulation rank. As the simulation tracks evolving physical phenomena, blocks may be refined or coarsened to adapt mesh resolution, requiring periodic redistribution across ranks. Computationally, operations on each block are structured as a *directed acyclic graph* (DAG) of tasks—including physics computations, mesh management operations, and inter-block communication. Beyond these per-block operations, AMR codes also invoke global operations for redistribution and synchronization, whose characteristics we detail in §8.2.2.

AMR codes use task-based runtimes to execute DAGs using asynchronous execution strategies. Some frameworks like Parthenon implement these abstractions directly, while others use general-purpose runtimes such as Charm++ [130], Uintah [131], HPX [132], and Legion [133]. Our work incorporating telemetry in work placement complements execution strategies masking residual imbalance.

Placement Mechanisms. When a redistribution is invoked, placement mechanisms compute new block-rank assignments, and blocks are migrated accordingly. Modern block-based AMR codes rely on octree mesh structures combined with space-filling curves (SFCs) to enable efficient parallel mesh management while approximately preserving communication locality. SFC-based partitioning balances uniform block counts effectively but does not account for computational variability between blocks—a gap we address in this work. Octree structures, SFC ordering, and their implications for placement flexibility are discussed further in §8.5 where we present our placement policies.

8.2.2 Characteristics of Key AMR Operations

Compute Tasks. Compute tasks perform the core physical and mesh operations of the simulation. While operations like refinement tagging often have predictable costs, physics kernels are a major source of variability. For instance, regions with steep gradients may require more solver iterations, increasing compute cost. Importantly, this cost is not directly correlated to spatial area, as each block contains the same number of computational points (*cells*) regardless of refinement level. Ultimately, the execution time of these compute tasks represents the core computational cost to advance the simulation state, establishing the fundamental lower bound on runtime.

Communication Tasks. Communication tasks create complex inter-block dependencies through *boundary exchanges*, where each block communicates with up to 26 neighbors in 3D (faces, edges, and vertices). Communication volume depends primarily on the number of physical variables

exchanged and neighbor relationships instead of the refinement level of the blocks. These exchanges typically use non-blocking MPI operations and are latency-sensitive due to small message sizes and the potential for blocking downstream tasks. *Flux correction* follows a similar small-message, peer-to-peer pattern for ensuring consistency of conserved quantities.

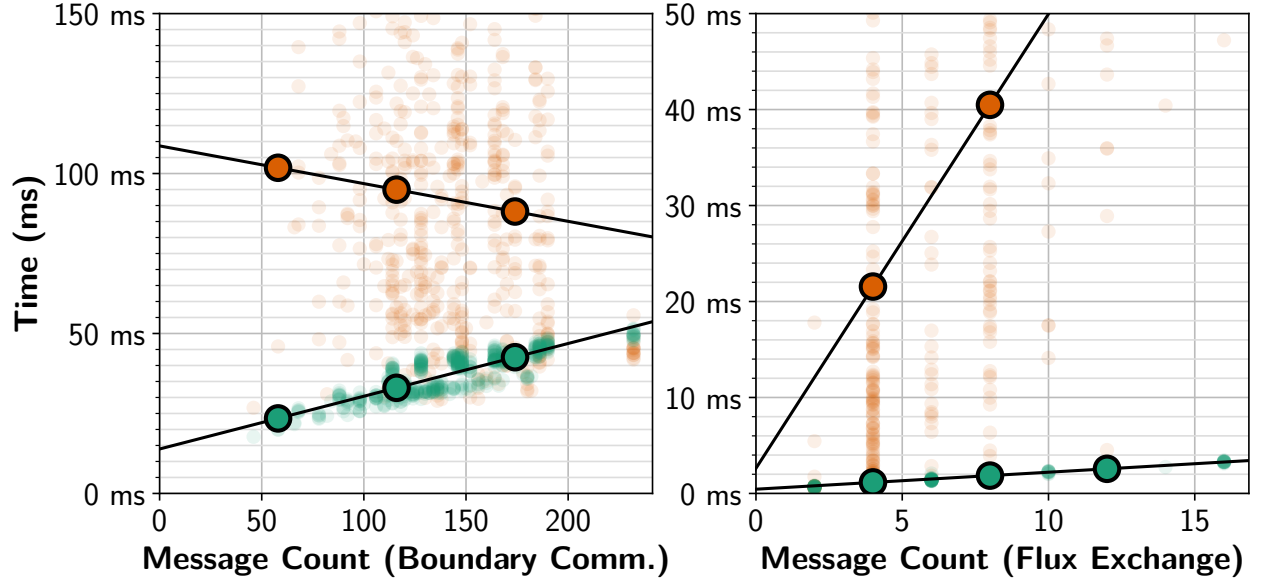
Redistribution. Redistribution operations rebalance blocks across ranks as the simulation evolves, adapting to changes from mesh refinement or coarsening. They are triggered when blocks are tagged for refinement based on physical criteria, like solution gradients exceeding a threshold. Each redistribution computes new block-to-rank mappings and migrates data. Because redistribution may be invoked frequently and must complete within tight performance budgets, placement policies must be fast and capable of handling complex scheduling constraints. The frequency depends on the underlying physics—some problems require frequent adaptation, others are more stable.

Synchronization. Synchronization operations can be explicit (MPI barriers) or implicit (blocking collectives). They inherently expose performance variability by forcing all ranks to wait until the last rank reaches the synchronization point. This waiting time often increases with scale as the likelihood of encountering a straggler grows. While asynchronous collectives are suggested as an alternative [136], synchronous operations remain necessary for certain correctness guarantees, like ensuring that all ranks execute the same timestep.

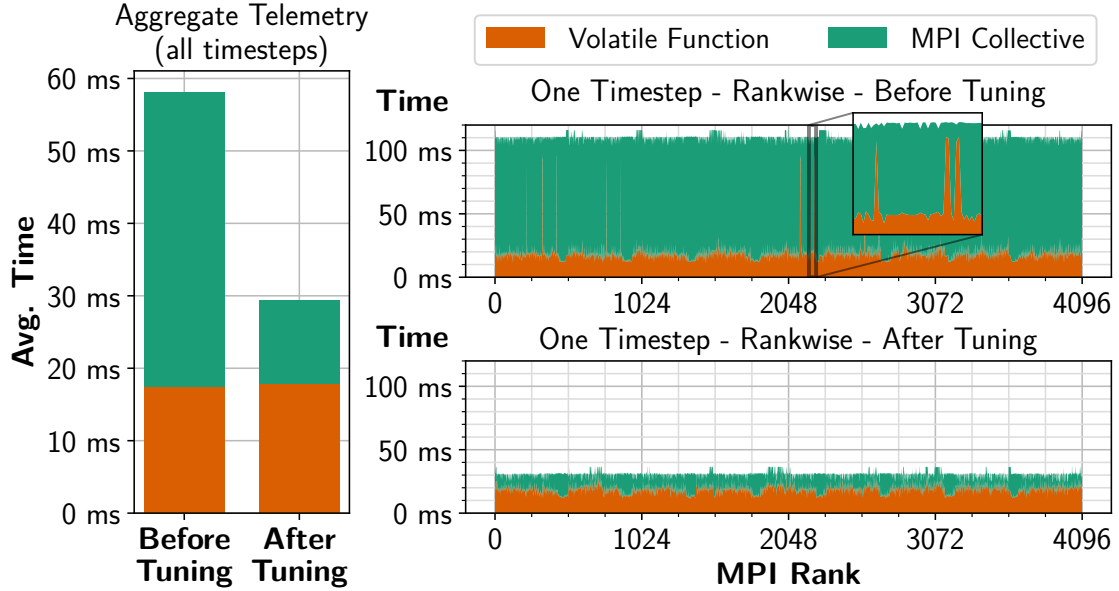
8.3 Challenges of Telemetry-Driven Placement

In principle, fine-grained telemetry should enable placement decisions to be grounded in observed workload behavior. However, our early attempts revealed two fundamental difficulties. First, the telemetry itself was unreliable: metrics such as communication time showed poor correlation with predictors like message volume (Fig. 20a), making it impossible to model baseline runtime. Second, the magnitude of this variability also changed unpredictably as placement properties were modified. For example, optimizing for communication locality did not reliably improve end-to-end performance initially. These observations forced us to treat placement not as a theoretical optimization problem, but as a systems engineering task grounded in empirically observed runtime behavior. The following challenges structure the empirical and algorithmic contributions of this work.

Challenge 1: Cross-stack Performance Anomalies. When initial telemetry proved inconsistent, showing poor correlation between workload metrics and runtime (Fig. 20a), our first challenge was establishing trust in the measurements. In production environments, organizational and operational boundaries both constrain and scope analyses. Our full-stack access eliminated these boundaries: every deviation, across hardware, drivers, MPI, or application layers, became our direct responsibility to diagnose and explain. While this significantly increased diagnostic complexity, it also provided the crucial advantage of unrestricted access to low-level tools like `perf` [137] and eBPF [138], which were necessary to establish the measurement fidelity underlying this work.



(a) Correlation between message counts and timing



(b) Coarse vs fine-grained telemetry from an AMR code

Figure 20: Telemetry challenges in AMR codes. (Top) Initial telemetry shows poor correlation between work (message counts) and communication time. Tuning for system-level issues improves correlation and telemetry reliability. (Bottom) Fine-grained telemetry reveals subtle performance anomalies affecting average collective time by 3 $\times$.

Challenge 2: Analyzing Fine-Grained Telemetry. Diagnosing anomalies required analyzing large volumes of per-timestep, per-rank, and per-block telemetry. Anomalies originated on a small subset of ranks but propagated globally through communication, making root cause identification nontrivial (see Fig. 20b). Tracing tools [57, 139] produced unstructured, high-volume output in formats like OTF2 [139] unsuited for query-driven diagnosis. Our analytics workflow evolved organically, beginning with standard tooling such as TAU [57] and progressing towards custom query-driven

workflows capable of surfacing low-level system effects at scale.

Challenge 3: Effective, Low-Overhead Placement. Reliable telemetry exposed genuine load imbalance that could be targeted by placement, but incorporating this information imposed strict new constraints. Placement decisions had to be computed within tight time budgets, under 50 ms per redistribution for our target codes, to avoid measurable overhead on simulation runtime. Even simplified placement objectives, such as minimizing makespan, are NP-hard [140], and additional objectives further complicate optimization. Placement mechanisms had to encode relevant tradeoffs while remaining fast and robust enough for dynamic AMR workloads.

We address these challenges sequentially: §8.4 describes the tuning and analytics infrastructure used to establish reliable telemetry, and §8.5 presents placement policies enabled by this exercise.

8.4 Denoising Telemetry For Placement

This section describes our methodology for establishing reliable telemetry, addressing the first two challenges from the previous section. We begin with mitigating variability from fail-slow hardware (§8.4.1), followed by examples from different layers of the software stack in (§8.4.2). We then describe the evolution of our analysis workflow in (§8.4.3), followed by a critical path model to formally analyze the impact of task dependencies and scheduling on runtime in §8.4.4. While the specifics of our interventions are necessarily empirical and context-dependent, they serve to illustrate recurring failure modes across the stack and the analytical methods required to uncover them.

Hardware. All experiments ran on a research cluster with 600 compute nodes, each node having Intel Xeon E5-2670 16-core processors, 64GB RAM, and 40 Gbps Qlogic 7340 NICs.

Software. Codes using Parthenon [62], Phoebus [73], and AthenaPK [62], built against MVA-PICH2 [141] and the PSM fabric library [142].

8.4.1 Eliminating Faulty and Fail-Slow Hardware

Hardware problems surfaced early in our study and had to be resolved before any software-level conclusions were meaningful. Initially collected profiling data showed more than 70% of runtime being spent in global synchronization (Fig. 21). Per-rank telemetry showed four-fold compute slowdowns on clusters of 16 ranks—an unmistakable sign of thermal throttling, confirmed by syslogs. Excluding those nodes immediately cut total runtime from 10 h to 2.5 h.

While such severe throttling is less likely to persist undetected in production clusters, it serves as an archetype for hardware-related fail-slow behaviors that often manifest more subtly. Studies report transient degradation from throttling, memory errors, or flaky links commonly impacting production workloads [75, 143]. Such faults can present as legitimate stragglers, but must be identified and addressed separately to avoid misdirected tuning efforts.

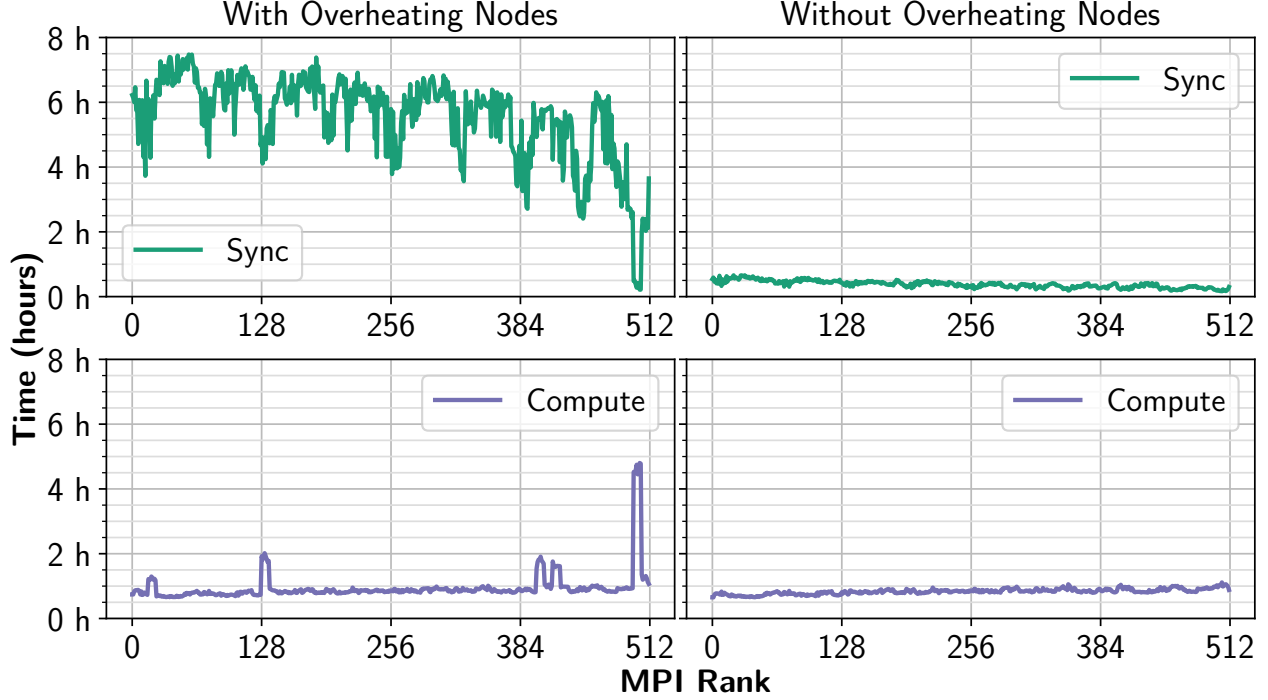


Figure 21: Profiling data from early runs affected by CPU throttling. Compute times on impacted ranks were inflated by up to $4\times$ and appeared in clusters of 16, leading to elevated synchronization delays across the system. Pruning affected nodes reduced overall runtime by $3\times$ by lowering synchronization overhead

To ensure measurement integrity, our launch workflow overprovisioned nodes and ran pre/post-job health checks (syslog and other hardware indicators). Failing nodes were automatically pruned from runs and blacklisted.

8.4.2 Tuning The Software Stack

We found interactions within the software stack—application logic, MPI runtime, and network drivers—to be the primary source of performance variability impacting telemetry reliability. While tools exist to check for basic configuration errors or known MPI misuses [144], they cannot capture the dynamic, workload-dependent performance artifacts arising from suboptimal tuning of parameters like queue depths, scheduling priorities, or internal thresholds. We present three representative examples of mitigations applied across the software stack.

Application-Level Example: MPI_Wait Spikes. In one code, we traced occasional spikes in `MPI_Wait` following `MPI_Isend` to fabric driver behavior: missing acknowledgments (ACKs) triggered a recovery path that unnecessarily blocked the sender. Since receivers had already acknowledged the messages, blocking the senders on acknowledgments was avoidable. We implemented a drain queue that transparently allocated new MPI requests and drained the blocked requests in the background. Fig. 20b shows the spikes and the impact of our mitigations.

Application-Level Example: Task Reordering. In another AMR code, MPI send tasks were sched-

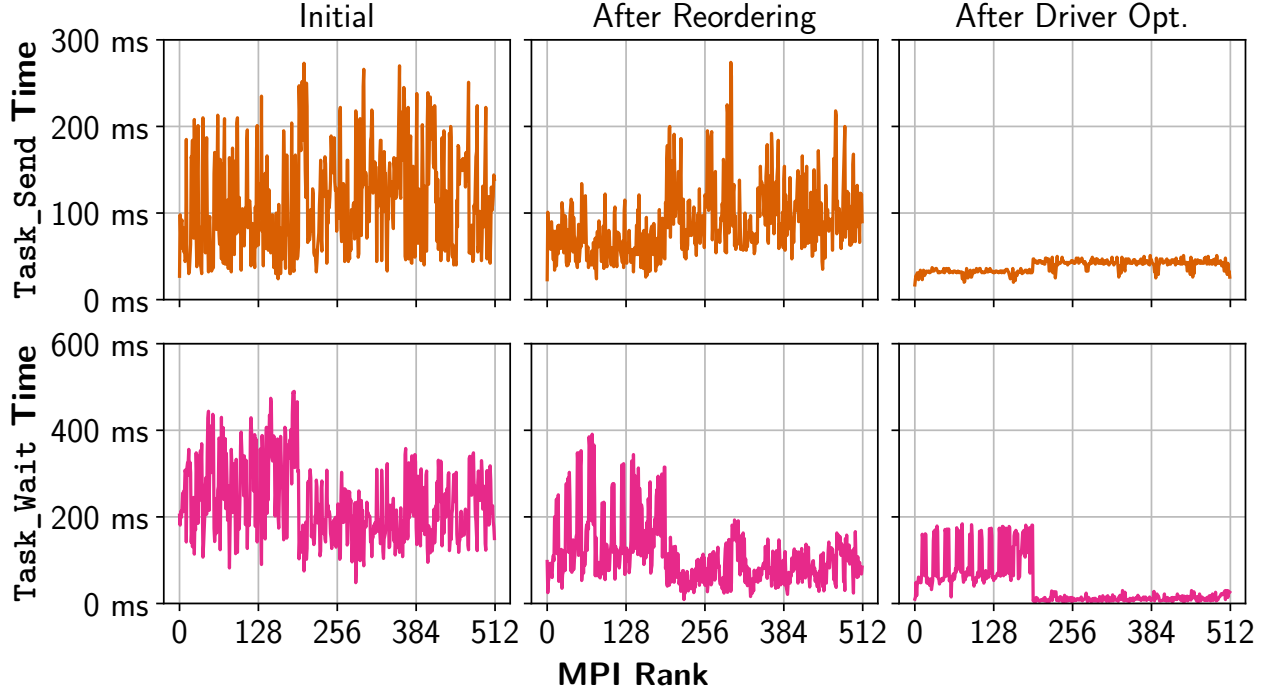


Figure 22: Rankwise boundary communication performance before and after two optimizations. Prioritizing sends in the task schedule reduced noise, revealing faint performance trends. Subsequent queue size tuning further reduced variance, clarifying the underlying telemetry structure.

uled after compute/wait tasks, causing cascading delays on CPUs (masked on GPUs where developed) Prioritizing sends (Fig. 22, middle) unblocked dependent ranks without affecting senders, reducing wait times and jitter.

Network-Level Example: Queue Size Tuning. Persistent variance, contributing to poor correlation between communication time and work (Fig. 20a), was traced using `perf` [137] to contention in the MPI library shared-memory path due to an insufficient preconfigured queue size. Increasing this value reduced MPI wait time variance and significantly improved the correlation between measured communication time and message volume (Fig. 22, right).

Implications. Mitigating the anomalies described above required analytical capabilities we could not replicate with standard tools. Transient spikes would be obscured by aggregation (profiling), while the sheer volume and complexity of fine-grained data made manual trace inspection infeasible. Systematically identifying root causes—correlating events across ranks, time, application phases, and system layers—necessitated programmable low-level interfaces like eBPF [138], complemented by flexible, query-driven analysis. Our struggles with existing approaches directly motivated the evolution of our analysis workflow, detailed next, towards techniques better suited for this challenge.

8.4.3 Evolution of our Analysis Workflow

Addressing the analytical limitations described above required evolving our workflow substantially beyond standard tools: TAU [57] provides useful coarse summaries via profiling, but finer-grained telemetry is only accessible via traces, which were impractical to analyze at scale. We wrote TAU plugins to emit CSVs which we analyzed with `pandas` in python. As we scaled up, parsing time became a bottleneck, and we switched to custom binary formats for efficiency.

As our needs evolved, we wanted programmable telemetry triggers based on reconstructed application state. We implemented our own telemetry collection using the MPI [49] and Kokkos [145] profiling interfaces. Eventually, we outgrew `pandas`’ analytical bandwidth and found our telemetry queries naturally mapped to SQL over data ingested into ClickHouse [65], a columnar analytics database.

In hindsight, our pipeline mirrored modern observability stacks: structured schemas, binary formats, and relational queries. Though still ad hoc, it provided the low-latency, queryable insight needed to support targeted diagnosis and hypothesis-driven exploration, essential for tuning and placement at scale. We explore the broader implications of this model in §8.7.

8.4.4 A Critical Path Model of Execution

Before turning to placement, we introduce a critical path analysis framework to identify optimal task ordering strategies under a fixed placement. This not only grounds the reordering optimization described in §8.4.2, but also helps formally identify other avenues for optimization. We define the *critical path* as the chain of dependent tasks that determines the runtime of the straggler at the next synchronization point. Reduction in critical path length is then modeled as minimizing `MPI.Wait` time on that path, as it is the only flexible-duration component in the execution path. Compute kernels have fixed runtimes, and other communication operations such as `MPI_Isend` and `MPI_Irecv` are similarly fixed, as they simply post buffers to the fabric. We now establish a key principle:

Given a single round of concurrent P2P communication between two synchronization points, at most two ranks can be implicated in the critical path, regardless of scale.

This follows from Lamport’s *happened-before* [146] relation — only dependent operations can create causal relationships. If the critical path is local to a rank, it reflects compute imbalance and involves minimal `MPI.Wait` time. More interesting are two-rank paths, where one rank is stalled waiting on a message from another. These cases are illustrated in Fig. 23a. This model reveals two strategies to shorten such paths:

Operation ordering to reduce wait stalls. The most direct way to shorten the path is to ensure the remote message is sent as early as possible. This is precisely what our task reordering optimization accomplished: by prioritizing sends, we minimized the dispatch delay for messages that were on the critical path.

Overlapping computation to hide wait stalls. Asynchronous runtimes discussed in §8.2.1 aim to hide `MPI.Wait` stalls by overlapping them with independent work. However, this requires both a

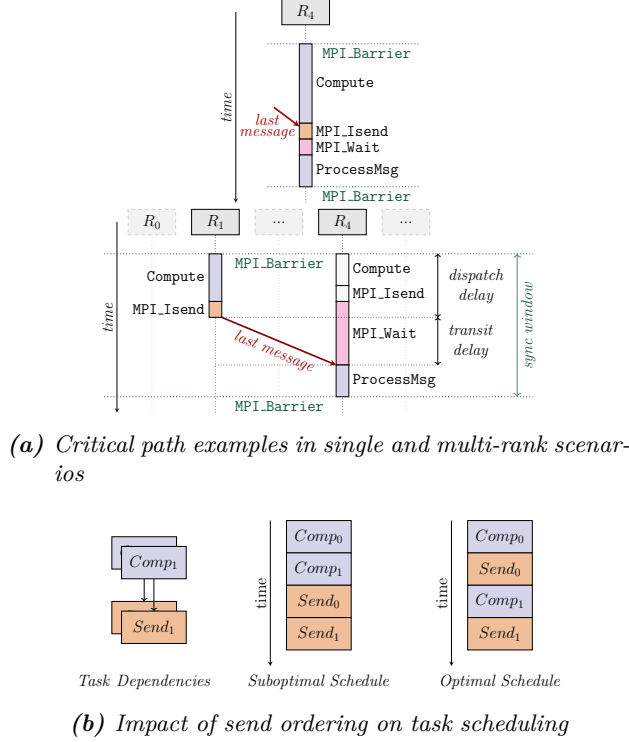


Figure 23: Critical paths within a synchronization window. (Top) Single- vs two-rank critical paths, demonstrating how delay chains can be purely local or involve exactly two ranks with one P2P communication round. (Bottom) A task schedule for two blocks, demonstrating the impact of ordering. Prioritizing $Send_0$ reduces its dispatch time without affecting $Send_1$, minimizing its potential to create a critical path.

non-zero wait time and the availability of independent tasks. Within a single mesh block, tasks are often data-dependent. While multiple blocks on the same rank can provide independent work, this creates a counterintuitive tension: a strict locality-preserving placement may be detrimental, as all blocks on a rank could end up waiting for the same remote straggler, limiting opportunities for independent work.

This analysis demonstrates the complexity of reasoning about fine-grained execution in partially asynchronous codes, where runtime behavior emerges from a large number of complex and subtle interactions. It also highlights the multiple, sometimes conflicting, dynamics introduced by the locality-reducing placements we discuss next.

8.5 Designing a Placement Policy

With the reliable and interpretable telemetry foundation established in §8.4, we turned to the final challenge: using this runtime information to design effective placement policies. Validated measurements confirmed that MPI collective synchronization dominated runtime, accounting for 35%–50% of total time and increasing with scale (512–4096 ranks), while direct communication and redistribution overhead remained below 10%. This cost was clearly a downstream effect of compute

time variability between blocks. Mitigating the impact of this measured variability became the primary placement objective.

Guided by our codes—which, in the worst case, trigger refinement every five 250 ms timesteps—we set a 50 ms placement computation budget to cap overhead at 5% of the total runtime. This constraint ruled out complex optimization objectives such as maximizing communication-compute overlap or encoding network topology, due to limited expected payoff and high constraint complexity. Instead, we focused on two key optimization dimensions:

Compute Load Balance. Directly minimizing the variance in per-rank compute load on the basis of measured block costs was the obvious objective to reduce straggler delays. This problem—formally known as *makespan minimization*—is NP-hard [140], necessitating efficient heuristics.

Communication Locality. Baseline SFC-based placements (§8.5.1) preserve spatial locality but hinder load balance flexibility. The actual runtime benefit of this locality had to be evaluated empirically, motivating exploring locality-aware strategies.

We now describe the baseline infrastructure used and augmented by our policies in §8.5.1, followed by load-centric and locality-preserving policies in §8.5.2 and §8.5.3. We conclude with CPLX (§8.5.4), a hybrid policy that enables tunable control over the tradeoff between locality and load balance. All policies were implemented in the Parthenon AMR framework [62] and are directly available in Parthenon-based codes such as Phoebus [73] and AthenaPK [62].

8.5.1 Existing Placement Infrastructure

Modern block-based AMR codes use octrees and space-filling curves (SFCs) as foundational data structures for mesh management. This section describes these building blocks and the baseline placement policy, as understanding their strengths and limitations is crucial for developing better policies.

8.5.1.1 Octrees and Space-Filling Curves

Octrees provide an efficient hierarchical representation of the mesh structure. When a block needs to be refined, it is split into 8 (2^3) child blocks (in 3D). Only leaf blocks of the octree participate in the simulation. Octrees serve multiple functions in AMR codes—from mesh management to neighbor tracking to block placement—with the latter enabled through easy construction of Z-order space-filling curves via a depth-first traversal of the tree, as shown in Fig. 24. Blocks are assigned sequential *block IDs* in order of this traversal. This ordering approximately preserves spatial locality, as blocks with nearby IDs are more likely to be spatial neighbors. Some locality, however, is inevitably lost as dimensionality reduction is inherently lossy.

8.5.1.2 Baseline Placement Policy

Placement computation is a component of the *redistribution* process, invoked if the mesh structure changes because of (de)refinement operations. Redistribution proceeds in three steps: blocks are

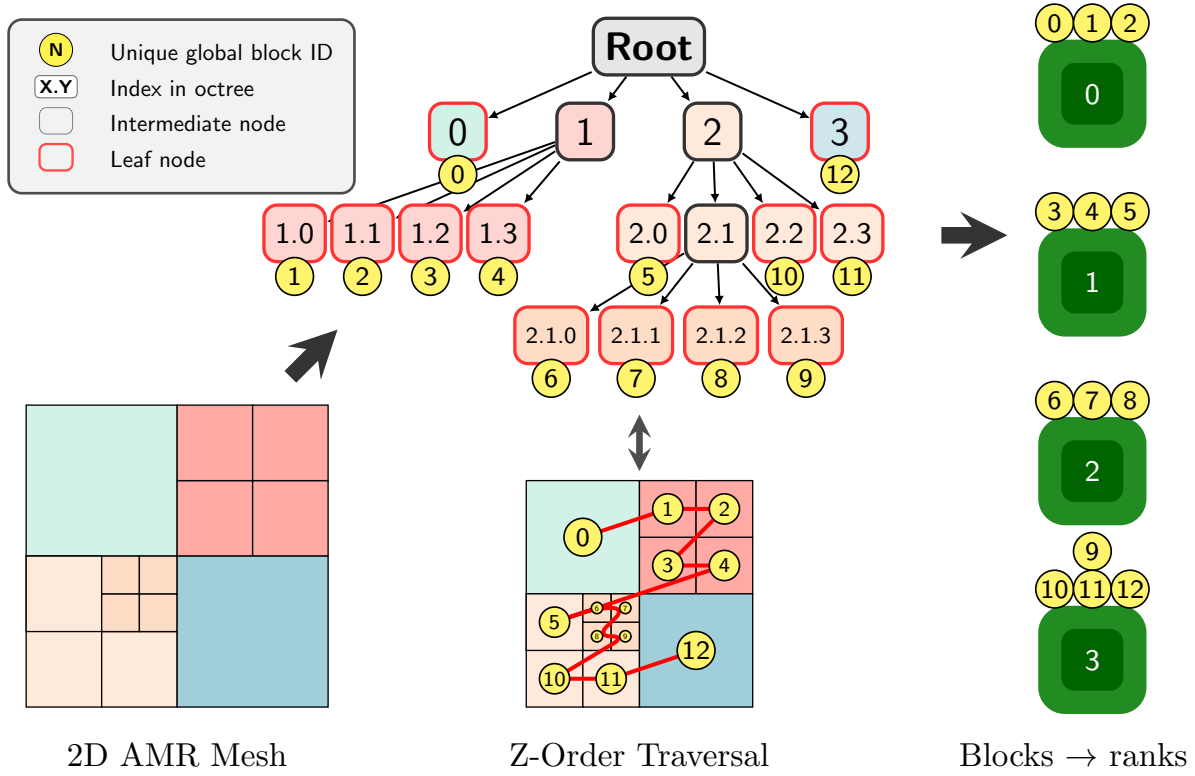


Figure 24: Example of an adaptively refined mesh, octrees, and Z-order Space-Filling Curves (SFCs) in two dimensions. Mesh blocks correspond to octree nodes with refinement creating child nodes. Sequential block IDs are assigned to leaf blocks using a depth-first traversal, equivalent to a Z-Order SFC. Contiguous block ID ranges are then assigned to simulation ranks to balance block counts while preserving locality.

first assigned block IDs using Z-order SFCs, the placement policy computes new block-rank mappings for all blocks, and finally blocks are migrated to their new ranks using MPI P2P operations.

The baseline policy simply orders blocks by block ID and assigns contiguous ranges of $\lceil n/r \rceil$ or $\lfloor n/r \rfloor$ blocks to consecutive ranks, where n is the total number of blocks and r is the number of ranks. While better assignments are theoretically possible, this approach provides a reasonable balance between compute load and communication locality by balancing block counts across ranks while co-locating spatial neighbors.

8.5.1.3 Augmenting Existing Infrastructure

Octrees and space-filling curves are fundamental, well-studied components of modern AMR codes. Rather than replace these proven building blocks, we augment them with mechanisms to handle variable block costs and flexible allocation patterns. While frameworks like Parthenon provide hooks for specifying per-block costs, these are typically initialized to 1 in practice—treating all blocks as computationally equal. This simplified model persists partly because domain scientists rarely provide cost estimation models for their simulations, and partly because the baseline policy’s

strict contiguous allocation constraint limits its ability to accommodate variable costs. Our changes to the infrastructure are minimal but enable significantly more flexible placement policies:

1. We populate the existing cost specification hooks with actual computation costs measured via telemetry
2. We modify the block management infrastructure to support arbitrary (non-contiguous) block-to-rank mappings

The second change required minor modifications to data structures but has no correctness implications, as logical dependencies between blocks remain intact. This allows for non-contiguous placements, but communication costs increase only if policies choose to break locality.

8.5.2 LPT: Prioritizing Load Balance

We first investigated whether pure load balancing, ignoring communication locality entirely, could provide meaningful runtime improvements. The *Longest-Processing-Time First* (LPT) algorithm offers a simple but powerful approach—sort blocks by compute cost and assign each to the least loaded rank. LPT is a classical greedy algorithm for makespan minimization with strong theoretical guarantees—the makespan of an LPT solution is provably at most $4/3$ times the optimal makespan [72]. In practice, LPT performs remarkably well; we could not obtain better solutions from a commercial ILP solver [147] despite letting it run for 200 s.

LPT demonstrated surprisingly strong performance despite completely ignoring communication locality. While it did incur higher communication costs than the baseline approach, the improvements in load balance more than compensated for this overhead. This suggested that the value of locality preservation might be lower than initially assumed, motivating us to explore this tradeoff further.

8.5.3 CDP: Preserving Locality

After observing LPT perform well despite its locality costs, we wanted to explore the potential of a locality-maximizing load-balanced placement. To this end, we developed *Contiguous-DP* (CDP), which uses dynamic programming to select contiguous block ranges to improve load balance while preserving the locality patterns of the baseline.

Consider a simulation with 10 blocks and 4 ranks. With an average of 2.5 blocks per rank, CDP explores allocations like $[2, 2, 3, 3]$ or $[2, 3, 2, 3]$, finding one that minimizes makespan while maintaining contiguous allocations. As a result, CDP has identical locality-preserving properties as baseline.

Formal Description. CDP solves the contiguous partitioning problem: given block costs $w_1, \dots, w_n$ (in SFC order), partition them into r contiguous segments to minimize the maximum segment sum (makespan). Let $W[i] = \sum_{j=1}^i w_j$. The minimum makespan $DP[i][k]$ for partitioning i blocks among k ranks follows ($DP[0][0] = 0$):

$$DP[i][k] = \min_{0 \leq j < i} \max(DP[j][k-1], W[i] - W[j])$$

The algorithm has complexity $O(n^2r)$ in general, where n is the number of blocks and r is the number of ranks. However, we optimize it to $O(nr)$ by considering only two contiguous chunk sizes: $\lceil n/r \rceil$ and $\lfloor n/r \rfloor$. This optimization maintains solution quality while making CDP practical for AMR timescales. This DP formulation guarantees an optimal makespan-minimizing placement within the explored chunk sizes.

In our experiments, CDP improved upon baseline but only reached parity with LPT. While CDP showed lower communication times, its higher synchronization times suggested that perfect locality preservation might be unnecessarily restrictive, motivating our development of our hybrid policy called CPLX, described next.

Scaling CDP With Chunking. The overhead of computing placements with CDP became noticeable at 4096 ranks, prompting us to develop a parallel implementation using hierarchical chunking. This approach divides blocks into c contiguous chunks of approximately equal cost, then applies CDP independently to each chunk using a subset of ranks. At 4096 ranks with chunk size 512, this creates 8 parallel-processed chunks.

While chunking may not find the globally optimal CDP solution, this approximation has minimal impact since CDP’s output serves only as an intermediate step for CPLX. The approach reduces placement overhead while retaining CDP’s solution quality.

8.5.4 CPLX: Combining the Best of Both

Our experiments with LPT and CDP revealed a clear tradeoff—LPT achieved better load balance but higher communication costs, while CDP preserved locality at the expense of some load imbalance. Neither policy consistently outperformed the other in end-to-end runtime, suggesting the potential for a hybrid approach that could combine their strengths.

Developing such a hybrid proved challenging. Our initial attempts to blend the policies produced unpredictable results—small sacrifices in load balance did not translate to proportional gains in locality, and vice versa. We eventually realized that it was easier to selectively break locality in a contiguous placement than to restore locality in an arbitrary one. This insight led to the CPLX design principle: start with a locality-preserving solution from CDP and strategically apply LPT to address the most significant imbalances.

CPLX begins by computing an initial CDP placement that reuses the chunking mechanism for scalability. It then assesses the work allocated to different ranks by sorting them in descending order of load. The policy selects $X\%$ of ranks from either end of this sorted list—the most overloaded and underloaded ranks—for rebalancing via LPT. Including both ends is crucial as rebalancing needs both source and destination ranks to effectively redistribute work.

The parameter X , ranging from 0 to 100%, provides precise control over this locality disruption. At $X=0$ (CPL0), no ranks participate in rebalancing, preserving the locality characteristics of CDP. At $X=100$ (CPL100), all ranks undergo LPT rebalancing, effectively becoming pure LPT. Intermediate values create hybrid behaviors, only disrupting locality within the $X\%$ most imbalanced

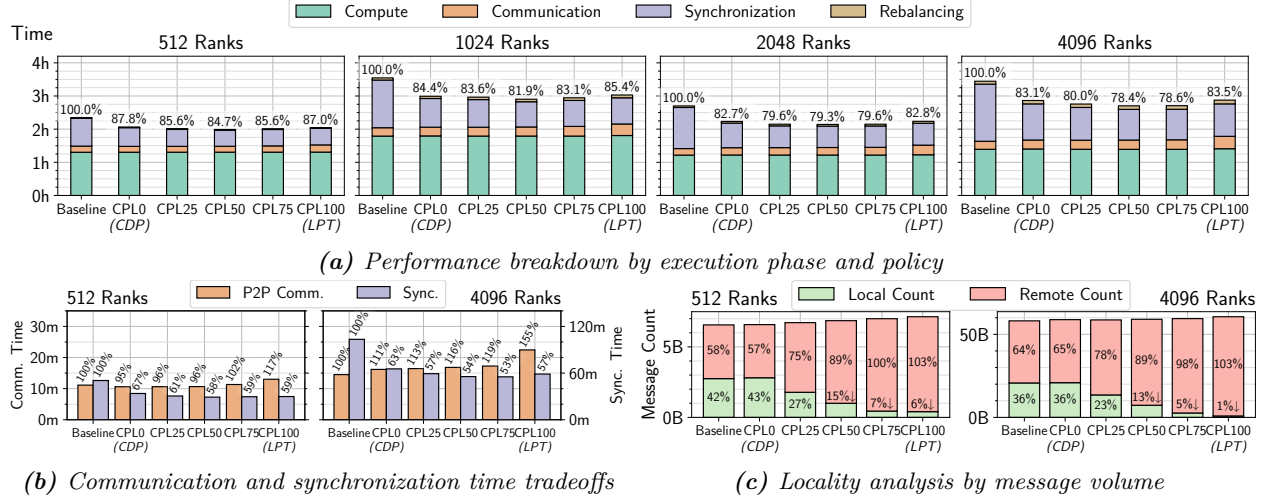


Figure 25: Runtime statistics comparing different load-balancing policies across multiple scales (512–4096 ranks).

(Top) Total runtime, classified into phases. CPL50 performs best, with up to 21.6% runtime reduction over baseline. (Bottom, left) Clear tradeoff emerges between communication time (increasing with X) and synchronization time (decreasing with X), demonstrating how CPLX enables controlled balance between these competing factors. (Bottom, right) Message patterns show that increasing X reduces local in favor of remote communication, providing evidence for the locality-performance tradeoff.

ranks while preserving it elsewhere. This enables CPLX to systematically explore the complete locality-balance tradeoff space while maintaining the performance requirements of AMR codes.

8.6 Evaluation of Placement Policies

ranks	mesh size	t_{total}	t_{lb}	n_{initial}	n_{final}
512	128^3	30,590	1,213	512	2,080
1024	$128^2 \times 256$	43,088	4,576	1,024	3,824
2048	128×256^2	43,042	4,699	2,048	4,848
4096	256^3	53,459	9,392	4,096	8,968

t_{total} : total timesteps, t_{lb} : timesteps invoking load-balancing

n_{init} : initial block count, n_{final} : final block count

Table 2: Problem configurations for Sedov Blast Wave 3D experiments. All configurations use 16^3 block size, initializing with one block per rank to ensure meaningful placement impact at all scales. The number of blocks increases through refinement as the shock wave propagates, with final block counts shown in the rightmost column.

In this section, we evaluate CPLX under real AMR workloads and synthetic microbenchmarks. While CPL100 (pure LPT) and the baseline policy represent known quantities, our key question is whether intermediate values of X can provide meaningful control over the locality-balance tradeoff. For CPLX to be successful, it must achieve three goals: intermediate values should demonstrate

measurable control over both communication patterns and runtime performance, achieve minimum runtime if the tradeoff is worth balancing, and maintain low runtime overhead even in setups with rapid refinement. We evaluate these aspects by comparing different values of X against CPL100 as our reference point.

We primarily evaluate CPLX using the Sedov Blast Wave 3D problem in Phoebus [73], a hydrodynamics code built on top of the Parthenon [62] framework. While we also studied placement in other codes, such as a galaxy cooling setup in AthenaPK [62], results were directionally similar: codes with high compute variability benefit more from better placement, and vice-versa. We describe our hardware setup in §8.6.1, present Sedov results in §8.6.2, and use microbenchmarks to evaluate CPLX’s load balance and locality properties under different synthetic regimes in §8.6.3.

8.6.1 Experimental Setup

We evaluate four configurations of the Sedov Blast Wave problem, as detailed in Table 1. Each run begins with one block per rank, increasing as refinement proceeds, and finally settling at ~two blocks per rank. The table reports initial and final block counts, total steps, and load-balancing invocations.

All experiments ran on the tuned Emulab-backed cluster described in §8.4, with 16 ranks per node, at scales from 512–4096 ranks. Telemetry was collected using the custom profiling stack described in §8.3. We compare the tuned baseline against CPLX with $X \in \{0, 25, 50, 75, 100\}$, and only report runtime gains from placement. Runtime gains from tuning are omitted because of variable impact across different codes and scales.

Hardware Checks. To ensure experimental integrity, all reserved nodes were scanned for hardware issues—thermal throttling, memory errors etc.—before every individual run in each parameter suite using the process described in §8.4. All reported results are from runs where no participating node showed issues either before or after execution.

Software Stack. All experiments used MVAPICH2 v2.3.7 [141], compiled against a tuned version of PSM [142], the userspace library for our QLogic fabric.

8.6.2 Sedov Blast Wave Results

We evaluate five placement policies on the Sedov Blast Wave problem across four scales (512–4096 ranks), comparing baseline performance against CPLX, as X is varied from 0–100. The analysis focuses on overall runtime behavior (Fig. 25a), the synchronization—communication tradeoff (Fig. 25b), and P2P communication patterns (Fig. 25c).

Finding 1: *Baseline: synchronization reaches 50% of total runtime at scale.*

Figure 25a shows total runtime across all policies, decomposed into computation, communication, synchronization, and rebalancing phases. With baseline placement, computation and synchronization dominate the profile, jointly accounting for over 90% of execution time at all scales. Communication and rebalancing remain minor components, contributing approximately 7% and

3% respectively. Synchronization overhead grows sharply with scale—from 35% at 512 ranks to 50% at 4096 ranks, despite the tuned environment. These results demonstrate the challenge of managing dynamic behavior in bulk-synchronous parallel applications. Unless mitigated, runtime is dictated by stragglers, the likelihood of which only goes up with scale.

Finding 2: *CPLX achieves up to 21.6% overall runtime reduction, with impact increasing with scale.*

Figure 25a also shows that all CPLX configurations outperform the baseline, with runtime improvements that grow more pronounced at larger scales. At 4096 ranks, the best-performing variant (CPL100) reduces total runtime by up to **21.6%** relative to baseline. Even at 512 ranks, CPLX achieves a 15.3% reduction, and all values of X tested yield improvements above 12%.

Runtime follows a clear U-shaped curve as a function of X : starting from CPL0 (locality-preserving), decreasing to a minimum with intermediate values, and rising again toward CPL100 (pure LPT). This shape reflects the ability of CPLX to control the load-locality tradeoff in a manner that yields meaningful overall runtime gains.

Crucially, compute time remains flat across all policies, confirming that the total amount of work is invariant to placement. The observed improvements arise entirely from reductions in synchronization overhead, which dominate non-compute runtime in the baseline configuration. Measured as reduction in non-compute time, the impact of CPLX becomes even more pronounced—up to 36% reduction at 4096 ranks.

Finding 3: *CPLX enables consistent control over the load-locality tradeoff, and tradeoff impact increases with scale.*

Figure 25b isolates the core tradeoff by showing P2P communication and synchronization times, normalized against the baseline, for all policy configurations at 512 and 4096 ranks. Intermediate scales are omitted for compactness, but we find them to demonstrate similar trends. Increasing X reduces synchronization time, as expected from better load balance, while increasing communication time due to loss of locality.

We find that modest values of X (25–50) capture almost all of LPT load-balancing benefits, without incurring the commensurate locality cost. While LPT incurs a relative increase of 55% in locality cost, its impact on the overall runtime remains modest as even at 4096 ranks, communication is only 13% of the overall runtime. The increasing spread between intermediate X values and LPT with scale suggests that the runtime impact of balancing the two increases with scale. Given the strong performance of LPT vs commercial ILP solvers (§8.5.2), this is likely close to the practically achievable optimum under our model of AMR placement.

At 512 ranks, some CPLX configurations even reduce communication time relative to baseline—this is explained by the compounding effects captured by boundary communication time. High variance in the preceding compute kernels results in more time spent in `MPI.Wait` by some ranks, and improving load-balance has a positive downstream impact that overrides the locality cost. This is an illustration of higher-order placement effects that we discard for computational tractability.

Finding 4: *P2P message volume statistics confirm locality degradation with increasing X .*

Fig. 25c shows the breakdown of local (intra-node shared memory) and remote (inter-node MPI) P2P messages, normalized to the total baseline message volume, for CPLX variants at 512 and 4096 ranks. As X increases, remote message counts rise while local messages fall, reflecting the fact that CPLX progressively expands LPT coverage at the expense of locality. This tradeoff is enacted mechanically by CPLX, independent of the underlying application or mesh—only the runtime impact of this shift depends on workload characteristics. We explore workload variations further in §8.6.3 using synthetic microbenchmarks.

A subtler effect visible in Fig. 25c is the growth in total MPI-visible message volume with increasing X . This occurs because intra-rank communication—handled via `memcpy` when blocks are co-located—is replaced by MPI messages when placement breaks locality. This shift explains part of the communication time increase observed in Fig. 25b.

Finally, even the baseline placement routes a majority of messages across nodes: at 4096 ranks, **64%** of messages are already remote. This is an intrinsic property of space-filling curves—dimensionality reduction to 1-D preserves only partial spatial locality. CPLX does not introduce a new communication cost class, but extends an existing placement pattern and exposes it as a tunable parameter.

8.6.3 Microbenchmarks

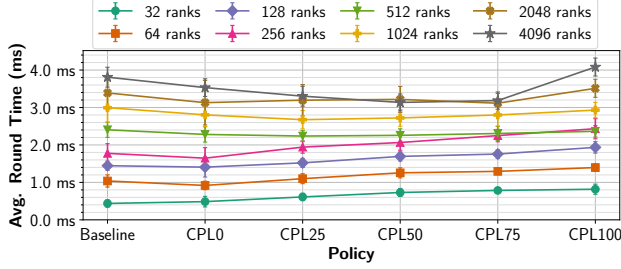
To complement the application-level Sedov results (§8.6.2) and enable controlled evaluation across a broader range of conditions, we developed two microbenchmarks. `commbench` isolates P2P communication under varying placements, while `scalebench` evaluates the effectiveness and computational cost of placement policies under synthetic compute imbalance.

Commbench: Simulating Boundary Communication Patterns.

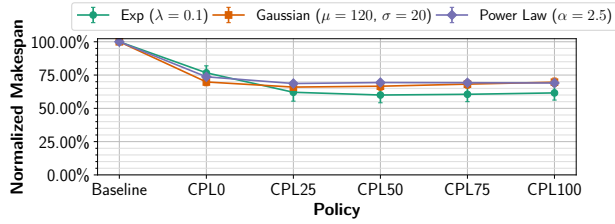
`commbench` is a synthetic microbenchmark that isolates boundary communication and evaluates how placement locality affects end-to-end round latency. It constructs octree-based AMR meshes with realistic refinement, deriving P2P patterns from geometric neighbor relationships (face, edge, vertex) across refinement levels. While inspired by the Halo3D-26 fixed-grid benchmark [148], `commbench` simulates a full AMR placement pipeline and accepts custom placement policies as drop-in modules.

Each communication round emulates a boundary exchange, with blocks exchanging messages based on neighbor type. Message sizes are realistic: face-neighbor exchanges are proportionally larger than edge or vertex ones. Meshes are refined to yield 1–2 blocks per rank, and each round includes all neighbor types. Timing is captured using MPI barriers before and after each round. Results are averaged over 100 rounds and 10 random meshes per policy. We discard cold-start rounds and those with latency above 10 ms, which we found to reflect fabric-level recovery behavior unrelated to placement.

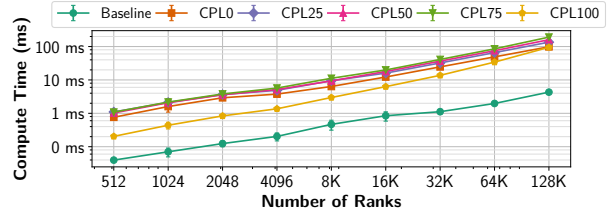
Figure 26a shows average round latency across placement policies as locality decreases from CPL0 to CPL100. While higher locality yields lower latency at small scales (512 ranks and below), a surprising U-shaped trend emerges at larger scales: intermediate values of X outperform both



(a) Boundary communication latency vs. policies (*commbench*)



(b) Load balance quality vs. policies (*scalebench*)



(c) Policy compute cost vs. simulated rank count (*scalebench*)

Figure 26: (Top) Boundary communication round latency vs. locality at different scales, measured using *commbench*. Locality modestly affects round latency ($\pm 0.5\text{ms}$). At larger scales, strict locality preservation surprisingly becomes counterproductive, as it results in communication hotspots relative to hybrid placements.

(Middle) Average makespans from *CPLX* placements with three synthetic distributions. While *LPT* performs best across all distributions, the bulk of the benefits are realized with $X = 25$.

(Bottom) Compute time for *CPLX* as a function of scale, averaged across three synthetic distributions. Compute time stays below 10ms until 16K ranks, and can be reduced beyond that scale using smaller parallel instances.

extremes. This inversion is driven by face-neighbor communication, which increases as meshes get denser. Locality-preserving policies cluster high-traffic neighbors unevenly, increasing per-rank load. Intermediate *CPLX* settings diffuse this clustering without fully randomizing placement, leading to more uniform traffic patterns. Though the latency differences are modest ($\pm 0.5\text{ms}$), the reversal of expected trends underscores how observed system behavior can defy theoretical intuitions. *commbench* provides a practical mechanism for empirically selecting X in different environments.

Scalebench: Evaluating Imbalance and Placement Overhead. *scalebench* evaluates different policies at scales from 512 to 128K ranks, with 1–2 blocks per rank to enable flexible load-balancing. Block costs are drawn from three representative distributions—exponential, Gaussian, and power-law—with variability bounds chosen to create meaningful balancing opportunities while remaining within realistic AMR ranges.

Figure 26b plots the normalized makespan across placement policies (lower is better). We see that *CPL100* (*LPT*) achieves the lowest makespan across distributions, but *CPL0* and *CPL25* capture the bulk of the benefits with much higher locality retention. Figure 26c reports placement computation overhead as a function of scale. Compute costs increase with scale, but remain tractable

(~10ms) up to 16K ranks, rising to 100ms at 128K ranks. At the largest scales, zonal placement architectures can be adopted to mitigate placement overhead—dividing ranks into k zones to compute placement independently and in parallel [149].

8.7 Lessons From Our Placement Exercise

We distill our takeaways from this exercise into the following five lessons.

Lesson 1: *AMR Performance Is Straggler Mitigation.*

High synchronization costs in AMR codes are often attributed to load imbalance. Our experience suggests that this attribution may be premature: stragglers can also arise from tuning artifacts, execution strategy interactions, or hardware-level variability. Placement strategies cannot compensate for unstable system behavior—such instability must be identified and resolved independently before placement can be meaningfully evaluated.

Lesson 2: *Effective Tuning is Empirical and Idiosyncratic.*

Performance tuning is not about correctness, but about aligning system behavior with the workload. Default configurations reflect common-case assumptions that may vary across codes. Tuning, therefore, is both empirical and workload-specific [150–152]. It improves performance directly, while also making application behavior more predictable under placement changes. In our experience, progressive tuning clarified the telemetry structure, reduced confounding noise, and exposed deeper residual issues that would otherwise be misattributed.

Lesson 3: *P2P Communication Is A Key Variability Source.*

Our tuning efforts primarily focused on point-to-point (P2P) communication patterns used in ghost cell and halo exchanges. Unlike collectives, which are structured and easy to benchmark in isolation, P2P exchanges are fine-grained, latency-sensitive, and highly variable. They involve dynamic neighbor counts, varying message sizes, and inconsistent communication paths—each of which can activate different code paths in the network stack. We found these interactions to be the dominant source of variability, and they required targeted attention during tuning.

Lesson 4: *Diagnosis Needs Structured, Queryable Telemetry.*

Understanding why performance deviated required analytics over fine-grained telemetry to surface anomalous low-level behavior. While tracing tools capture such detail, they rely on unstructured formats like JSON [153] or OTF2 [28], which are poorly suited to multi-dimensional analysis across rank, time, and task. We converged on SQL-based analytics over telemetry grouped by timestep and sorted by rank, enabling views of application behavior aligned with synchronization intervals.

While tools like Caliper [154] do support structured telemetry, they—like our ad hoc pipeline—rely on parsing and postprocessing of plaintext data, which we found to slow down our iterative tuning workflows. Binary columnar formats like Arrow [35] and Parquet [155], when paired with in-situ collection, offer a promising foundation for low-latency BSP telemetry by enabling low-overhead parsing and efficient querying via embedded statistics over partitioned data.

Lesson 5: *Placement Optimization is Systems Engineering.*

AMR placement policy design is a systems engineering problem, guided primarily by empirical constraints rather than solely by theoretical optimality. Tradeoffs between compute balance, communication locality, and placement overhead must be evaluated based on observed performance impact, not static preferences. Placement strategies need to be empirically calibrated to each specific deployment context. Their computation must also respect strict time budgets—in our case, under 50 ms per redistribution—which constrains the complexity of the algorithms and necessitates focus on dominant effects.

8.8 Related Work

Variability and Runtime Analysis. Performance variability in HPC environments is well documented [10, 11, 13, 128, 156], with root causes ranging from tuning to contention. Separately, fail-slow hardware has been reported as a source of performance degradation [75, 143]. Similar variability challenges have been identified in the context of collective performance in hyperscale ML clusters [51]. Varbench [12] is a framework to characterize computational variability using a suite of compute kernels. Klenk et al. [25] analyze proxy MPI traces and report significant MPI overheads (60% at 1024 ranks) and attribute them to load imbalance, while also noting limitations of trace analyses. While prior work documents separate causes of stragglers, an end-to-end treatment combining tuning and placement optimization has been missing—a gap addressed by our work.

Performance Telemetry and Tooling. Established performance tools such as TAU [57] and Score-P [139] collect profiles and traces (e.g., OTF2), but lack the queryability required for scalable analytics, a limitation previously noted by Klenk et al. [25]. Caliper [154] supports structured telemetry and programmable aggregation, but relies on plaintext data formats. Parquet [155] provides a compact, binary columnar format with embedded statistics for efficient analytics, but is not widely adopted in performance tools. Hatchet [157] builds a structured in-memory representation from external telemetry sources but assumes prior parsing. Metric frameworks like LDMS [56] offer low-overhead system-level metrics, but often lack application context needed for bottleneck attribution. Yang et al. [158] describe a general critical path analysis framework, while our framework in §8.4.4 is tailored for AMR codes.

AMR and Communication Models. Modern AMR codes either use MPI or asynchronous runtimes like Charm++ [130] and Legion [133]. Neighborhood collectives [156, 159, 160] for boundary communication have been proposed as an alternative to point-to-point messaging, but are currently not used in the AMR codes we evaluated.

Placement and Load Balancing. Graph-based partitioners such as parMETIS [70] and Zoltan [69] handle complex meshes but incur high overhead. Sasidharan et al. [67] propose a SFC-based partitioner with lower overhead than METIS. All graph-based approaches model communication as edge

cuts, which we find poorly correlated with runtime communication overhead. Meta-Balancer [161] focuses on rebalancing triggers, while Zheng et al. [149] propose hierarchical schemes to scale load balancing. Solver-based approaches are used to optimize placement in cloud datacenters [162], but are computationally expensive (multi-hour) and impractical for AMR.

8.9 Conclusions

Two decades ago, Petrini et al. [10] traced *missing performance* in flagship supercomputers to noise and variability across the system stack. Our analysis of contemporary AMR codes shows this diagnostic challenge remains deeply relevant. Modern system complexity, driven by adaptive behaviors tuned for common cases, means performance is highly variable and context-dependent. As a result, simple attributions to load imbalance are often incomplete, obscuring deeper sources of variability. Systematically identifying and mitigating these issues across hardware and software layers was crucial for effective optimization. Only after establishing reliable, interpretable telemetry could our CPLX placement policy demonstrate tunable control over the load-locality tradeoff, achieving up to 21.6% runtime improvement over tuned baselines. Our experience suggests that low-latency analytics built on relational telemetry models may offer a viable foundation for resolving the persistent opacity of performance root causes.

9 ORCA Paper

Abstract

The *bulk-synchronous parallel* (BSP) paradigm powers critical computational workloads from scientific simulations to large-scale machine learning training. Running on tens of thousands of processors, these systems are uniquely sensitive to performance variations that current observability approaches struggle to diagnose—post-hoc analysis of massive traces introduces latency and rigidity that obstructs timely diagnosis.

We present ORCA, a system for closed-loop BSP observability. ORCA enables operators to declaratively specify views materialized on demand via a tree-based overlay providing coordinated collection, in-situ SQL analytics, and timestep-consistent control. On a real AMR code at 4096 ranks, ORCA collects detailed traces at 1–2% overhead versus 56–81% for existing tracers, executes queries orders of magnitude faster, and reduces data volume by multiple orders of magnitude through in-situ analytics. In an agentic evaluation of the closed-loop workflow, a performance anomaly that took months to diagnose manually is localized in 27 minutes.

9.1 Introduction

The *bulk-synchronous parallel* (BSP) [163] paradigm powers critical computational workloads, from scientific simulations to large-scale machine learning training. These systems are uniquely fragile: at the scale of tens of thousands of processors, lockstep execution means a single straggler can block cluster-wide progress at every synchronization point. As ML training pushes BSP toward 100,000

GPUs and scientific workloads grow in tandem, the cost of undiagnosed performance pathologies has become economically untenable. Yet the causes are diverse and highly context-dependent [10, 13, 33, 51, 60, 75, 76, 81, 128, 164–172]—load imbalance, poor overlap, hardware variability—varying with the cluster, workload, scale, and configuration. Metrics [56, 79] provide baseline visibility but rarely enough workload structure to attribute causes. Tracing [57, 59, 154, 173, 174] can capture the fine-grained patterns needed for diagnosis, but prevailing approaches—exhaustive collection to disk followed by expensive *extract-transform-load* (ETL) [33, 175]—cannot provide feedback at the speed problems demand. In our study of placement in AMR codes [33], isolating the root causes of several performance artifacts took months, while fixes typically took days. Rapid visibility into production pathologies remains one of the highest-yield paths to efficiency.

We argue for always-on, steerable observability as a production capability for BSP workloads—not just faster post-hoc analysis, but continuous visibility with the ability to escalate on demand. The evolution of distributed tracing [176–178] offers an instructive parallel. In microservices, tracing evolved to preserve causal relationships between requests—the unit of work in asynchronous systems—so tail latency is continuously monitored and attributable to specific services. In BSP, collectives induce causality: performance anomalies manifest as stragglers that stall collectives. And because BSP is a single coordinated workload, the system can unify observation and intervention—enabling operators to steer collection on demand, test hypotheses live, and act on feedback, all while the job runs. Real-time diagnostics are already proving valuable in ML training [60, 81, 169–172, 179] for problems ranging from straggler localization to training correctness, but each system implements a bespoke collection and analytics pipeline. A common observability substrate is needed for BSP workloads, providing low-overhead on-fabric transport, BSP-aware coordination semantics for telemetry aggregation and control, and programmable interfaces for analytics (§9.3).

We present a *tree-based overlay network* (TBON) for real-time observability and control of large-scale BSP applications, called ORCA (*Observability with Real-time Control and Aggregation*). ORCA treats telemetry as columnar streams, transformed via a SQL-based analytics interface called *OrcaFlow* en route to *sinks* such as dashboards or parallel filesystems. A key contribution is the *timestep dataframe* abstraction: building on the insight that global synchronization divides BSP telemetry into causally independent windows, ORCA transforms telemetry from a rank-major to a timestep-major representation (§9.4.1). An RDMA-based TBON-optimized transport (§9.4.2) enables low-overhead collection of these dataframes. They can then be transformed via SQL-based analytics, routed to a dashboard, or persisted as partitioned Parquet for zero-ETL post-mortems. Distributed query planning with operator pushdown minimizes data movement across the overlay (§9.4.3). A control plane (§9.4.4) provides atomic and timestep-consistent control semantics, guaranteeing correctness for interventions such as new analytics, probes, parameter updates. ORCA combines a modern columnar analytics stack [35, 82, 155] with RDMA and BSP-aligned data and control paths to realize low-overhead, real-time, closed-loop feedback.

We evaluate ORCA on a real AMR code at up to 4096 CPU ranks.

- In conventional tracing workflows, ORCA incurs ~2% runtime overhead (vs 56–81% for state-of-

the-art tracers), produces up to $42\times$ smaller traces for equivalent telemetry, and enables up to $6900\times$ faster analyses than existing trace formats (§9.6.1).

- In-situ analytics enable low-overhead needle-in-the-haystack workflows—outlier `MPI_Wait` calls are isolated in real-time at sub-2% runtime overhead, while discarding up to 99.999% of data at MPI ranks (§9.6.2).
- In an agentic evaluation of its closed-loop capabilities using an LLM agent as a proxy for a human operator, ORCA enables localizing a performance anomaly—the same bug from our AMR study that took months of manual investigation—in 27 minutes, while reducing data volume by up to $144\times$ (§9.6.3).

More broadly, ORCA demonstrates that declarative SQL analytics can be embedded on the fabric alongside a live MPI application, enabling streaming analytics workflows beyond observability. ORCA is open source [180].

9.2 Background and Motivation

Bulk synchronous parallel (BSP) [163] workloads in both scientific computing and machine learning training pose unique efficiency challenges. These jobs, increasingly spanning tens of thousands of nodes, execute in lockstep and synchronize frequently via collective communication. At these scales, BSP is uniquely fragile: a single straggler can block cluster-wide progress at synchronization points. Their root causes are highly context-dependent, ranging from load imbalance and suboptimal overlap to tuning-induced variability, and have been documented extensively [10–13, 33, 51, 75, 76, 128, 143, 165, 166]. Despite extensive studies, general-purpose mechanisms for operational visibility into BSP application performance remain elusive.

9.2.1 Motivation: Closing the Feedback Loop

Whereas monitoring targets *known unknowns* via a fixed set of metrics such as CPU and memory utilization, performance pathologies in BSP workloads are dominated by *unknown unknowns*. Observability [181–184] in this context is the ability to resolve them by constructing the right views from telemetry and refining them until causal attribution is possible.

Dominant observability paradigms for BSP workloads make this loop slow and rare. Diagnosing pathologies requires rerunning applications under progressively heavier diagnostics, from lightweight aggregates collected as profiles to fine-grained traces captured as per-rank event logs. At scale, these traces are too large to inspect visually, and programmatic analyses require expensive *extract-transform-load* (ETL) [175] before queries can be served. Additional instrumentation or interventions require repeating this workflow multiple times to isolate root causes and verify fixes. As a result, operators are forced to choose between expensive investigations and ad-hoc workarounds.

This gap motivates a different model: always-on observability as an operational capability, with relevant views derived in real-time, hypotheses tested live, and collection steered accordingly while the job is still running.

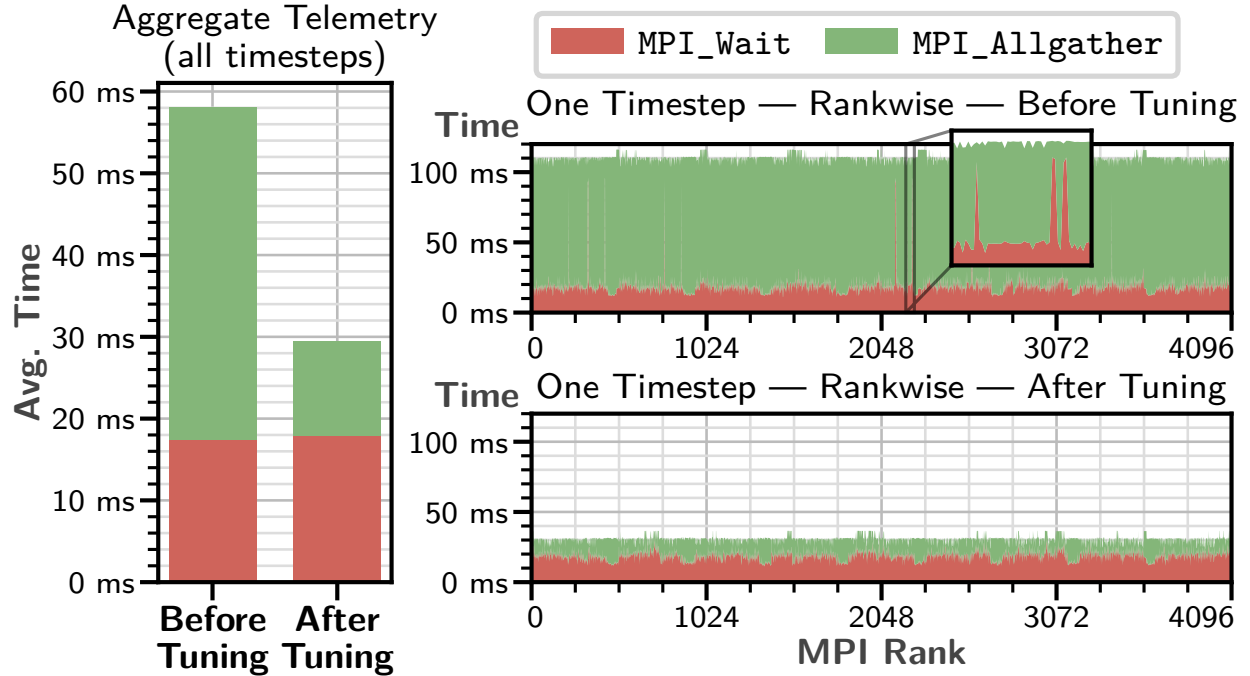


Figure 27: Telemetry from an AMR code where a volatile `MPI_Wait` precedes `MPI_Allgather`. Aggregate telemetry (left) misattributes straggler-induced `MPI_Wait` as elevated collective cost. Fine-grained telemetry (right) makes the cause obvious but requires joining all ranks’ telemetry.

9.2.2 Case Studies from HPC and ML

9.2.2.1 Optimizing Placement in AMR

Our study of placement and load-balancing in *adaptive mesh refinement* (AMR) codes [33] illustrates this gap concretely. While widely known for load-imbalance challenges [26], we found that genuine load-imbalance was indistinguishable from other hardware, software, and tuning artifacts without fine-grained telemetry. Figure 27 shows one such example: tail latency spikes in `MPI_Wait` that only manifested at scale would get attributed to collectives in aggregate telemetry. Each artifact took us months of iterative diagnoses, while mitigations would typically only take a few days.

Diagnosing each of these issues required multiple iterations of fine-grained capture and ETL-heavy analysis over formats such as CSV. The mitigations identified were specific to our cluster and workload. Even after these artifacts were fixed, our placement policies demonstrated a fundamental tradeoff between compute load balance and communication locality — the right balance between the two varies across workloads, scales, and architectures. Translating these optimizations to production environments would require repeating this time-consuming exercise — underscoring the need for infrastructure that makes such investigations routine.

9.2.2.2 Parallels from the Pytorch Ecosystem

The PyTorch [80] ecosystem exhibits the same pattern. Kineto [174] captures fine-grained traces on demand as JSON. Dynolog [185] provides node-local control to trigger capture. *Holistic Trace Analysis* (HTA) [186] computes derived views such as activity breakdowns and straggler detection. Together, these form a closed loop, but with the same shortcomings we faced in our AMR study. HTA is a single-node Pandas [64] script that cannot scale to cluster-wide traces without expensive ETL. Dynolog provides best-effort actuation with no consistency guarantees, whereas reliable construction of views like Figure 27 requires complete telemetry for each collective instance.

The wider ML training literature reinforces this pattern. Recent systems address pathologies such as straggler localization [60, 171, 172, 179, 187], fault diagnosis [169, 170], and training correctness [81], but each reimplements subsets of collection, aggregation, and control from scratch. A common substrate would allow detection strategies to be expressed as analytics queries over telemetry, composable and deployable without rebuilding the infrastructure beneath them.

9.2.3 Existing Approaches and Gaps

Monitoring. Monitoring enables basic visibility via metrics captured at sampled intervals, but lacks the workload context for causal attribution. Prometheus [79] is a time-series database for system counters, while LDMS [56] extends coverage for application-level statistics such as Kokkos [188]. They can not support synchronized on-demand collection necessary for views such as Figure 27.

Profiling and Tracing. Tools such as TAU [57], Score-P [173], and Kineto [174] can provide both high-level aggregates or fine-grained telemetry via traces, but their workflows are post hoc by design. Traces can characterize short runs, but cannot provide always-on visibility over long-running jobs. In the absence of any real-time feedback to guide collection, the post hoc paradigm also incentivizes overcollection, leading to large datasets and long delays. Formats like OTF2 [28] are not designed for analytics, and visual inspection for anomalies is impractical at scale with billions of events.

Dataframe Analytics. Recent tools such as Caliper [154], DFTracer [59], Pipit [189], and Pytorch HTA [186] have adopted Pandas [64] and Dask-based [190] dataframe analytics interfaces to analyze telemetry, but require expensive ETL [175] to parse entire traces written as per-rank logs in plaintext formats — the working set for which scales with $\text{ranks} \times \text{timesteps}$. Columnar formats like Parquet [155] offer efficient post-hoc analytics, but do not by themselves address overcollection, working set scaling, or real-time feedback, and remain largely unadopted in tracing workflows. In a simple internal port of HTA analyses from JSON to Parquet, we observed 50× faster data loading and 18× faster end-to-end analysis.

Streaming and In-situ Analytics. ADIOS [191] enables fixed-function in-situ workflows [21] on scientific data, typically run as another MPI application. MRNet [192, 193] is a tree-based overlay used as a reduction backend for commercial debuggers, but provides a compiled packet-based programming model. Neither enable runtime-programmable dataframe-style analytics needed for

real-time BSP observability. Cloud streaming systems such as Flink [194] and Kafka [195] are designed for independent event streams over IP networks. They lack fabric-aware transport and BSP-aligned coordinated collection, precluding views like Figure 27.

9.2.4 The Missing Substrate

Diagnosing the root cause behind Figure 27 required capabilities beyond what any single tool in the taxonomy provides. At node scale, eBPF [138] provides a template: dynamic probes, programmatic aggregation, low overhead in the common case with depth when needed. It enabled controlled and targeted access to telemetry within `MPI_Wait` after alternatives were exhausted. Hypotheses about interrupts and scheduler effects were tested and discarded in hours. But eBPF is node-local, and our workflow involved ad-hoc orchestration, uncoordinated collection, and manual post-hoc correlation. Extending this to workload scale—tens of thousands of BSP ranks—requires a substrate that provides the same depth on demand, treating observability not as distributed logging, but as constructing views directly from running applications via a programmable interface. The next section states the requirements such a substrate must satisfy.

9.3 Requirements for Closed-Loop Observability

We now state the key requirements for closed-loop observability. The goal is real-time feedback through interactive views over running applications, while retaining the ability to capture detailed traces when deeper investigation is required.

BSP-Aware Semantic Guarantees. BSP observability requires guarantees on both data and control paths. On the data path, this means coordinated collection: detecting the straggler at a barrier requires telemetry from all ranks for that barrier instance. Sampling yields uncoordinated subsets insufficient for root cause attribution. Control path semantics must similarly align with workload structure. Control inputs must be applied consistently and atomically at synchronization boundaries across all ranks. While uncoordinated updates may be tolerable in some cases, they break correctness for interventions such as hyperparameter changes — a timestep-consistent control plane generalizes across use cases.

Low-Cost, Low-Latency Feedback. Practical observability requires a view-driven model: emitting only what is needed to materialize requested views. First, overhead must be proportional to view complexity — lightweight aggregates for continuous monitoring should incur minimal cost, while detailed traces remain available on demand. Second, feedback must be fast, as the latency of feedback bounds the latency of intervention. Finally, the system must minimize induced perturbation — BSP workloads are jitter-sensitive, and overhead must scale with view complexity.

Declarative and Flexible Analytics. No predefined set of analyses can anticipate every pathology. Operators must be able to specify views and collection policies via a programmable interface at runtime, without recompilation or restart. The collection interface must also be source and schema-

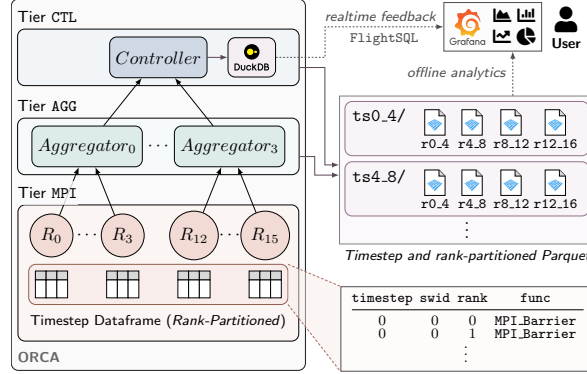


Figure 28: ORCA architecture. A three-tier TBON with application ranks (MPI) at the leaves, intermediate aggregators (AGG), and a controller (CTL) at the root. Telemetry originates as columnar timestep dataframes, is transformed by SQL plan operators at each tier, and persisted via DuckDB and Parquet sinks — enabling workflows from real-time feedback via lightweight metrics to zero-ETL post-hoc analytics.

agnostic, as telemetry may be emitted by any layer of the runtime stack, including MPI, runtime libraries, or custom annotations.

These requirements are in tension. Coordinated semantics demand global completeness, low overhead requires not disrupting the application, and low feedback latency requires streaming to materialize views quickly instead of accumulating data locally. The programming model must be expressive enough for diagnostic queries, while remaining compatible with low-overhead execution. Our design combines OLAP primitives (Arrow [35], Parquet [155], DataFusion [82]) with RDMA for low-overhead transport, aligning data partitioning with synchronization boundaries, analytics with a reduction topology, and control with timestep semantics.

9.4 Design

ORCA is a programmable *tree-based overlay network* (TBON) with a controller (CTL) at the root and aggregators (AGG) as the intermediate scale-out tier, attached to application ranks (MPI) as the leaf tier. CTL and AGG run on dedicated nodes outside the MPI domain and communicate with the application via on-fabric RPCs. The TBON provides asynchronous data and control planes. Data originates as columnar Arrow streams at leaf ranks, flows through the TBON, and is persisted via sinks. Two sinks are currently implemented: Parquet [155] for high-volume telemetry, and DuckDB [91] for real-time metrics and dashboards. The architecture is shown in Figure 28. We present ORCA as a progression of capabilities that successively shorten the observability feedback loop:

- The columnar data model (§9.4.1) and transport layer (§9.4.2) enable a workflow similar to conventional tracers, but more efficient to collect, store, and analyze.
- In-situ analytics (§9.4.3) and the OrcaFlow interface enable real-time feedback by transforming telemetry en-route to sinks. OrcaFlow chains SQL statements and map operators over multiple

streams, routing results to configurable sinks.

- The control plane (§9.4.4) enables online interventions via TS2PC, a speculative variant of two-phase commit that ensures commands are atomically applied at all ranks at the same timestep.

9.4.1 Data Model: Timestep Dataframes

The logical unit of collection, analysis, and persistence in ORCA is a *timestep dataframe*. Applications emit named and *typed* streams with predefined Arrow schemas. Multiple streams may exist with different schemas, but tuples within a single stream must conform to a fixed schema. Each collective creates a new *synchronization window*, identified by a unique monotonic identifier (**swid**) that is assigned to all events within that window. A timestep is an application-level construct that may span multiple **swids**, as applications typically invoke multiple collectives within a timestep. Timestep dataframes comprise all tuples from all ranks within that timestep. These dataframes are logical constructs formed from rank-partitioned shards — they arrive at different tiers at different times via an asynchronous pipeline, but are analyzed and persisted as a single unit.

Typed Streams Enable Columnar Analytics. While JSON [196] and OTF2 [28] traces require expensive ETL before dataframe analytics, ORCA emits typed Arrow [35] data at the source and retains columnar structure throughout the pipeline — enabling zero-copy in-situ analytics and zero-ETL post-hoc analyses via Parquet [155] at rest.

Timesteps Are Causally Partitioned. Timestep dataframes enable efficient analytics by capturing the causal independence of BSP execution. At blocking collectives, all ranks wait for stragglers and execution resets. Each timestep is therefore an independent unit of analysis. Pathologies are attributed first to a straggler rank that delayed a collective, then to events in the dataframe that led to that straggler. The evolution of distributed tracing in loosely-coupled microservices — from treating traces as bags of events [176] to capturing causal edges between events via *request IDs* [177, 178] — reflects a similar evolution in request-based systems, but is a poor fit for BSP execution structure.

Instead of accumulating telemetry into per-rank logs, ORCA optimizes for analytics over timestep dataframes by partitioning telemetry along (*timestep, rank*) as the primary key before persisting as Parquet. Among HPC systems, Hatchet [157] postprocesses traces to build callflow graph indexes — a secondary index that is redundant when the primary key is effective. ORCA simply preserves the primary key for both in-situ and post-hoc analyses. As opposed to traces written as per-rank logs, this transposition from a rank-major to a timestep layout enables efficient post mortems, as data for any timestep only requires reading a fraction of the trace regardless of the workload duration (see Figure 28).

9.4.2 Transport

ORCA’s transport layer is designed to move Arrow dataframes asynchronously over the fabric across tiers. Overhead is minimized via an incast-optimized, receiver-driven mechanism that avoids

synchronized pushes and overlaps data movement with application execution. As the CTL and AGG nodes are fully asynchronous and throughput-optimized, fabric QoS features can be used to relegate ORCA traffic to a lower priority class, relying on idle fabric capacity for low-perturbation transfer. For telemetry persisted as Parquet, dedicated cores on AGG are used to compress and persist the data once collected.

The transport layer is agnostic to how telemetry is generated. Different profiling mechanisms accumulate telemetry into per-stream *SchemaCollectors* — local buffers that emit Arrow dataframes when flushed. After each timestep, all collectors on each rank are flushed to generate per-stream Arrow dataframes, which are then serialized into RDMA-registered memory. Ranks then send a metadata RPC to their aggregators containing RDMA descriptors for the serialized data. Synchronous end-of-timestep serialization into local buffers avoids jitter from mid-timestep flushes. Ranks then resume execution, while aggregators collect these dataframes via asynchronous RDMA GETs. As data is already prepartitioned by timestep and rank, the collection pipeline simply preserves this partitioning. While shuffling can create partitioning along arbitrary dimensions, ORCA does not currently implement it so as to maintain predictable ingestion throughput.

After multiple iterations, we developed three key mechanisms for stable operation over a TBON:

- Aggregators must issue RDMA GETs at controlled concurrency to prevent catastrophic throughput collapse, as RDMA NICs have limited resources for tracking active RDMA operations (as low as 1–2 on older fabrics). The pull-based model, which resembles receiver-driven flow control, lets aggregators stay in control of data volume and is essential for TBON incast topologies to work.
- Because the collection pipeline yields dataframes in timestep order, it is vulnerable to head-of-line blocking from missing shards, risking *out-of-memory* errors. We address this with a priority queue at aggregators that prioritizes lower timesteps.
- While the system must be provisioned with adequate bandwidth to ensure minimal impact from backpressure, it is essential for stability. Aggregators communicate backpressure by withholding RPC ACKs, and ranks block if outstanding RPC count exceeds a configured limit.

9.4.3 In-Situ Analytics for Real-time Feedback

The Arrow-based timestep dataframe abstraction enables zero-copy in-situ SQL analytics at every tier of the TBON, enabling low-latency materialization of views and drastically reducing persisted data volume. OrcaFlow is a SQL-based programming interface that turns the TBON into a programmable reduction tree, transforming timestep dataframes en route to sinks — currently partitioned Parquet for offline analysis and DuckDB for real-time metrics. For analyses difficult to express in SQL, *map* operators allow invoking arbitrary precompiled kernels.

OrcaFlow is backed by a TBON-aware distributed planner and query engine built on Apache DataFusion. Each flow compiles into *tier plans* describing transforms to apply, data to forward, and sinks to execute locally. The planner automatically places both transforms and sinks, pushing operators that reduce data volume as close to the source as possible. MPI and AGG operate on rank-

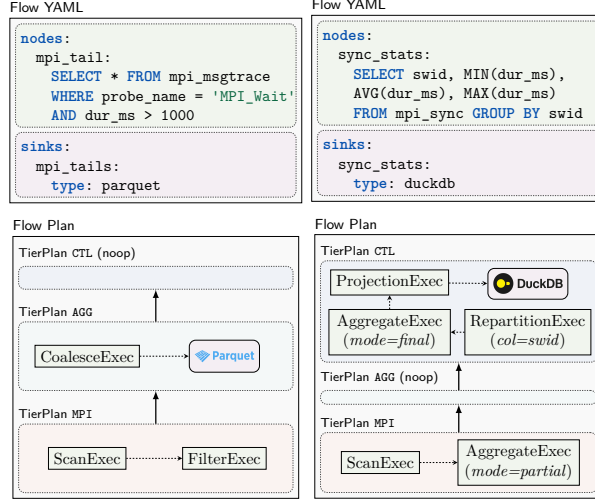


Figure 29: OrcaFlow compilation showing flows and their compiled tier plans. Flows chain SQL transforms over telemetry streams en route to configurable sinks. Both operators and sinks are placed automatically. (Left) A selective filter pushed to MPI, discarding most data at source. (Right) Aggregations also reduce data volume — pushed-down partial aggregations emit fixed-size intermediates regardless of input volume.

partitioned shards and execute per-partition operators: selection, projection, partial aggregation. Cross-partition operations like joins must run on CTL, the root aggregator. Sinks are placed on the lowest tier that produces their stream (no sinks on MPI, DuckDB only on CTL). Figure 29 illustrates with two flows:

- (Left) A selective tracing flow for `MPI_Wait`s > 10ms pushes filters all the way to the MPI tier. Results are coalesced at AGG and written as Parquet. Most telemetry for pathologies like Figure 27 gets discarded at source.
- (Right) An aggregation computing per-collective statistics splits into a partial aggregation at MPI and a final merge at CTL. Each leaf emits one row per group regardless of input volume — sketches and quantile digests fit the same model, producing fixed-size intermediates.

The programming model resembles batch analytics frameworks like Spark [197], but targets a multi-tier TBON, remains in-memory until sink ingestion, and avoids distributed shuffling to maintain predictable throughput. All tiers collect dataframes asynchronously but analyze them in timestep order. Per-partition compute budget scales with MPI and AGG, while cross-partition work is limited to a single node (CTL). This suffices for residual work after filtering and pre-aggregation. Post-mortems remain the preferred workflow for expensive cross-partition analyses, but still benefit from ORCA’s in-situ filtering, preaggregation, and partitioned Parquet.

9.4.4 Control Plane for Steerable Observability

A complete observability system must enable users to intervene in response to feedback, closing the loop between observation and action. In BSP workloads, this means steering collection and tuning application parameters at runtime in response to observed pathologies. Prior work

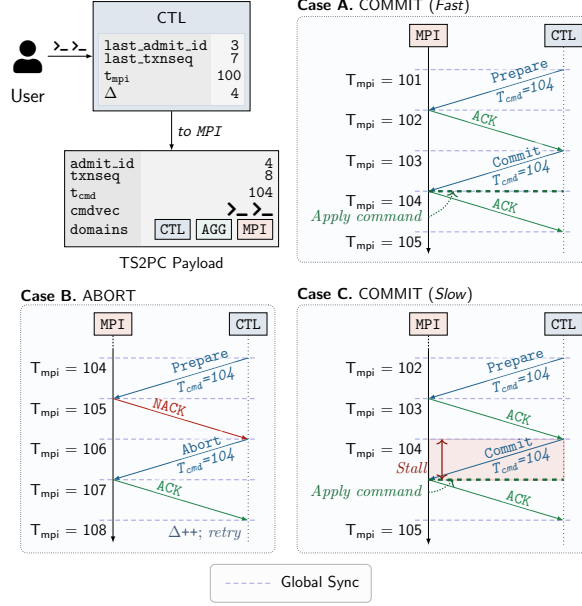


Figure 30: Timestep-linked two-phase commit (TS2PC). Commands are proposed for a future timestep $T_{cmd} = T_{mpi} + \Delta$. (A) Normal case: ranks agree and apply the command before executing T_{cmd} . (B) Abort: ranks have passed T_{cmd} and vote to abort, command is retried with a larger Δ . (C) Stall: ranks have prepared but not received commit/abort. Execution blocks until resolution to preserve correctness.

has explored auto-tuning for specific use cases. ORCA instead provides general mechanisms for timestep-consistent feedback and control.

9.4.4.1 Requirements

The control plane must be asynchronous, atomic, and timestep-consistent. The controller is not part of the collective domain, so control must be asynchronous and must not stall application progress. Commands must be acknowledged by all ranks before being executed, and they must execute on all ranks at the same timestep to preserve BSP semantics.

9.4.4.2 The TS2PC Protocol

ORCA implements the control plane as a replicated state machine using *timestep-linked two-phase commit* (TS2PC). Commands are opaque payloads — the protocol only decides when to commit and apply them. As the controller is not part of the MPI domain, it maintains a lagging view of the application’s current timestep ($= T_{app}$) via asynchronous notifications from Rank 0. TS2PC proposes commands for a future timestep $T_{cmd} = T_{app} + \Delta$, where Δ is an automatically learned speculative margin. The TBON assists the protocol by broadcasting requests and reducing responses — any `NAK` in the subtree results in a single `NAK` at the controller. Figure 30 illustrates the protocol via three cases.

- *Case A (Commit)*: Ranks agree to commit if they have not yet reached T_{cmd} , and apply the command before executing T_{cmd} .
- *Case B (Abort)*: If ranks are past T_{cmd} , they vote to abort. Δ is incremented and the transaction is automatically retried, quickly converging to the natural latency of the TBON.
- *Case C (Stall)*: A subtlety arises when ranks have prepared but not yet received commit/abort by the time T_{cmd} arrives. Correctness requires stalling execution until commit/abort is received, so the prepared command can be applied before T_{cmd} .

As a result, TS2PC enables a control plane that is atomic, timestep-consistent, and asynchronous in the common case.

9.5 Implementation

The core ORCA framework consists of 40K lines of code (80% C++, 20% Rust). ORCA uses the Mercury [83] RPC library for on-fabric communication via libfabric [198] and UCX [199] providers. The Rust portion consists of OrcaFlow’s DataFusion-based query planning and execution on each node, while the bulk of the functionality — bootstrap, transport, and control — lies on the C++ side.

9.5.1 Integration

ORCA provides two integration surfaces: the *OrcaClient* shim for applications, and *SchemaCollectors* for profiling interfaces.

OrcaClient. The *OrcaClient* shim links with the application and exposes a `PostTimestepAdvance()` hook, called after every timestep. ORCA can be activated at runtime by preloading the full library, otherwise these hooks become noops. `PostTimestepAdvance()` captures all data path work, including draining collectors, executing tier plans, dispatching data, and handling control messages.

SchemaCollectors. SchemaCollectors define stream schemas and capture events via `AddRow`, with one collector per stream. At the end of each timestep, ORCA drains collectors into per-stream Arrow dataframes and enqueues them for analysis. Our implementation currently packages basic collectors for MPI [49] and Kokkos [188]. Additional collectors may be defined by clients. Dynamic instrumentation via eBPF [138] and DynInst [200] remains future work.

9.5.2 Overlay: Bootstrap and Topology

The ORCA overlay bootstraps via a TCP-based discovery protocol for the initial handshake with the controller, and then switches to on-fabric RPCs once Mercury addresses are exchanged. Routes are then constructed as per the selected TBON topology, and available aggregators are automatically assigned to ranks. Each overlay entity is assigned a 32-bit ID called `ovid` to enable uniform addressing across all tiers. Multiple topologies are supported, including `DIRECT`, where all MPI ranks directly connect to the overlay, and `MPIREP`, where ranks choose node-local representatives to

reduce aggregator incast. Additional topologies can be plugged into the framework. For simplicity, we only discuss the DIRECT topology in this paper.

9.5.3 Data Path

As discussed in §9.4.3, the data path is asynchronous but timestep-ordered — each tier executes its tier plans on dataframes in timestep order. The head-of-line blocking concerns are mitigated via a *(timestep, rank)*-ordered priority queue for pending GETs at AGG and CTL. Some key implementation aspects are described below:

- Each MPI rank emits one metadata RPC after every timestep regardless of data volume if ORCA is enabled. This is necessary for correctness, as higher tiers can only mark a timestep complete once the entire subtree has reported.
- Although Arrow analytics is zero-copy locally, ORCA performs a single memcpy from RDMA-registered memory to allow registered memory handles to be recycled faster.
- ORCA nodes are highly multithreaded and provision large thread pools for decode, compression, and Parquet I/O, as sustaining O(GB/s) requires multiple cores even with efficient compression codecs such as Snappy. While Arrow decode is highly efficient, costs still accumulate for complex flows at scale, with O(10K) data fragments received each second.
- Tier plan execution on aggregators and controller is multithreaded. The same rank-partitioning strategy used to shard timestep dataframes across tiers is used to partition shards across cores.

9.5.4 Control Plane: TS2PC

The TS2PC implementation consists of *inner* and *outer* layers. The inner layer enables a single transaction with a unique monotonic ID called `txnseq`, targeting a particular T_{cmd} .

Δ -learning. The outer TS2PC layer automatically retries commands with an incremented Δ in case of aborts. As a result, Δ quickly converges to the natural latency of the TBON.

Concurrency. Correctness necessitates that only one transaction be in flight at any given time, as intermittent failures can result in commands being committed out of order, potentially breaking correctness. However, a single transaction can contain an arbitrary number of commands applied as a batch.

Handling Pauses. The control plane supports pausing and resuming the application as primitives for more complex workflows. An interesting case emerges from the interaction of the paused state with TS2PC — when a resume is committed, it will be enqueued to be applied at a future timestep never reached by a paused application, creating a deadlock. Our solution recognizes that the paused state makes the application synchronous with the user and renders TS2PC unnecessary for timestep consistency. Commands in this state circumvent the TS2PC protocol and are executed directly.

Cross-tier Consistency. Commands such as flow updates trigger tier plan updates at all tiers, and must be applied at a consistent timestep across the entire TBON. This is challenging as the higher

tiers are asynchronous and observe timestep at different times — these tiers are defined to observe a timestep when they process its data. Cross-tier consistency is supported via introducing control *domains* in the outer layer, allowing the entire overlay to be addressable via timestep-consistent control.

1. Each tier corresponds to a domain, and each transaction contains a bitmask corresponding to the domains to which it applies. Flow updates, for example, apply to all domains, thus all bits are set.
2. All transactions are committed by MPI regardless of the domain. The bitmask is only checked at *apply* time, and commands are committed only if the domain bit corresponding to that tier is set. As a result, MPI acts as the single source of truth on the application timestep.
3. CTL and AGG act on control messages by *snooping* on commit messages. Commands applying to that tier are replicated internally before broadcasting the commit to the subtree. As the inner protocol guarantees correctness regardless of commits’ arrival at MPI (Figure 30, Case C) this ensures that commands for T_{cmd} are enqueued at all tiers before T_{cmd} is observed.

9.6 Evaluation

We evaluate ORCA by progressively enabling its capabilities. As a passive telemetry collector, we compare runtime overhead (§9.6.1.1), storage efficiency (§9.6.1.2), and analytics performance (§9.6.1.3) against state-of-the-art tracers. We then demonstrate online analytics (§9.6.2) and control workflows (§9.6.3) — capabilities with no existing counterparts.

Hardware Setup. Experiments run on a 600-node research cluster with QLogic Truescale fabric (Intel Xeon E5-2670, 16 cores/node, 64GB RAM, 40 Gbps QLogic 7340 NICs) and a 20-node Lustre filesystem with 3GB/s aggregate bandwidth.

Software Setup. We use the Sedov Blast Wave 3D [74] AMR code from the Parthenon [62] and Phoebus [73] frameworks at 512–4096 CPU ranks, with 16 ranks/node. ORCA uses one aggregator node per 1024 ranks plus one controller node. All experiments use MVAPICH2 v2.3.7 [141] with a tuned PSM [201] library for our fabric. While simulation nodes use PSM, ORCA communication uses Mercury [83] over verbs because of fabric restrictions.

This setup represents a strenuous observability workload: CPU codes make every cycle visible as runtime overhead, the legacy fabric lacks QoS support and forces ORCA onto a non-native interface (verbs), and the AMR code’s multiple communication patterns — multiple collectives within a timestep, halo exchanges, and frequent load balancing — make it highly jitter-sensitive.

9.6.1 Tracing Overhead and Efficiency

We compare ORCA with four state-of-the-art tracing tools — TAU [57], Score-P [173], Caliper [154], and DFTracer [59] — along three axes: runtime overhead, storage efficiency, and analytics performance.

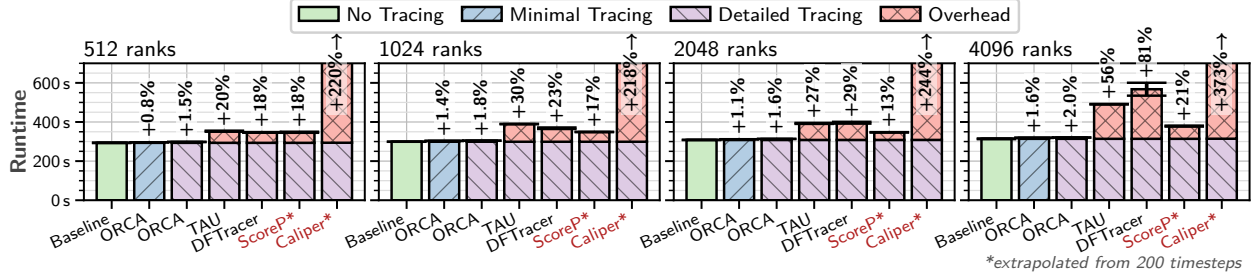


Figure 31: Runtime overhead for a detailed tracing profile on an AMR code at 512–4096 ranks. An additional minimal profile (collective-only) for ORCA shows always-on overhead. ORCA maintains 0.8–2% overhead across both profiles, while TAU and DFTracer degrade from 18–20% at 512 ranks to 56–81% at 4096 ranks. *extrapolated from 200 timesteps due to stability issues.

Collection Profiles. We set up ORCA with two tracing profiles: a lightweight profile that only captures MPI collectives, and a detailed profile that also captures all MPI point-to-point messages and Kokkos events, except some extremely high-volume events such as `MPI_Test` and `MPI_Iprobe`. For ORCA, the lightweight profile measures the common-case overhead of always-on observability, while the detailed profile measures the worst-case overhead of capturing all MPI and Kokkos events. All other tracing tools are configured with an identical detailed collection profile. Unless otherwise specified, all experiments run for 2000 timesteps, and we report the average runtime from three runs. Unlike tracing tools, ORCA is deployed with additional resources — one aggregator node per 1024 ranks, plus one controller node.

9.6.1.1 Runtime Overhead

Figure 31 compares the runtime overhead of ORCA and other tracing tools for 512–4096 ranks.

- ORCA remains within 0.8–2% overhead across all scales and profiles, while that of TAU and DFTracer degrades significantly — from 18–20% at 512 ranks to 56–81% at 4096 ranks (Score-P and Caliper discussed separately below).
- At 4096 ranks, ORCA moves ~550GB on-wire (~111GB after compression) over a shared fabric without QoS within 308s at under 2% overhead — demonstrating that RDMA-based telemetry pipelines can coexist with tightly-coupled BSP communication.
- The marginal increase in ORCA overhead between the lightweight and detailed profiles is only ~0.5%, despite collecting over 200× more telemetry (524MB vs 111GB, respectively) — underscoring the efficiency of Arrow serialization and asynchronous RDMA-based collection.

Why ORCA Works. TAU and DFTracer degrade steadily because they flush trace buffers to storage as they fill, which produces uncoordinated flushing patterns. This, coupled with a kernel-based data path for storage, introduces jitter that compounds at scale. ORCA, in comparison, uses an inherently efficient format, serializes synchronously into RDMA-registered buffers, and transports via efficient RDMA GETs.

Caveats on Score-P and Caliper. Because of difficulties running them for the full 2000 timesteps,

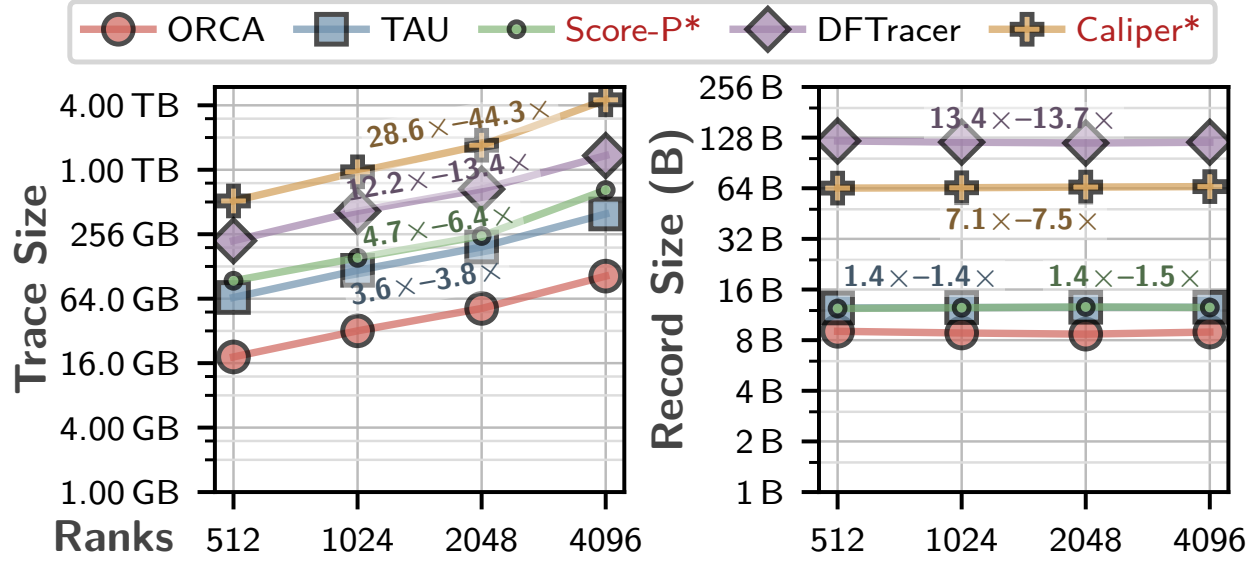


Figure 32: Trace size (left) and per-record size (right) comparison for detailed profile. ORCA traces are 3.6–44× smaller than other formats. Per-record size isolates format efficiency from record amplification — other tracers emit 2.5–3.3× more records than ORCA for the same collection profile.

all numbers for Score-P and Caliper are extrapolated from 200 timesteps. Both tools are designed for short characterization workloads where data is buffered in memory and flushed once at the end. Score-P additionally requires two passes — the first to estimate buffer sizes — and frequently crashed with out-of-memory errors even with 64MB trace buffers. Caliper supported periodic flushing, but the path was buggy and performance was significantly degraded even after fixes.

9.6.1.2 Storage Efficiency

Figure 32 compares the total size, and the per-record size of the traces emitted by ORCA and other tracers. ORCA writes Parquet, TAU and Score-P are configured for OTF2, DFTracer writes jsonl, and Caliper writes its custom plaintext `.cali` format. Key findings:

- ORCA consistently emits the smallest traces — 3.6–3.8× smaller than TAU, and up to 44.3× smaller than Caliper.
- The differences can be partly explained by format efficiency — OTF2, being a binary format, is only 1.4× larger than ORCA, while plaintext formats (`json` and `cali`) are 7–13× larger.

Record Amplification. Accounting for format efficiency, other tracing tools emit 2.5–3.3× more records than ORCA, despite the same collection profile.

1. All tracers except DFTracer (and ORCA) generate separate entry and exit records for each traced function. ORCA and DFTracer generate one record, which contains both start time and duration. This is both more efficient and allows functions to be filtered by duration without requiring any preprocessing to pair the entry and exit records.
2. Additionally, TAU incurs additional amplification from hardcoded derived events — for every

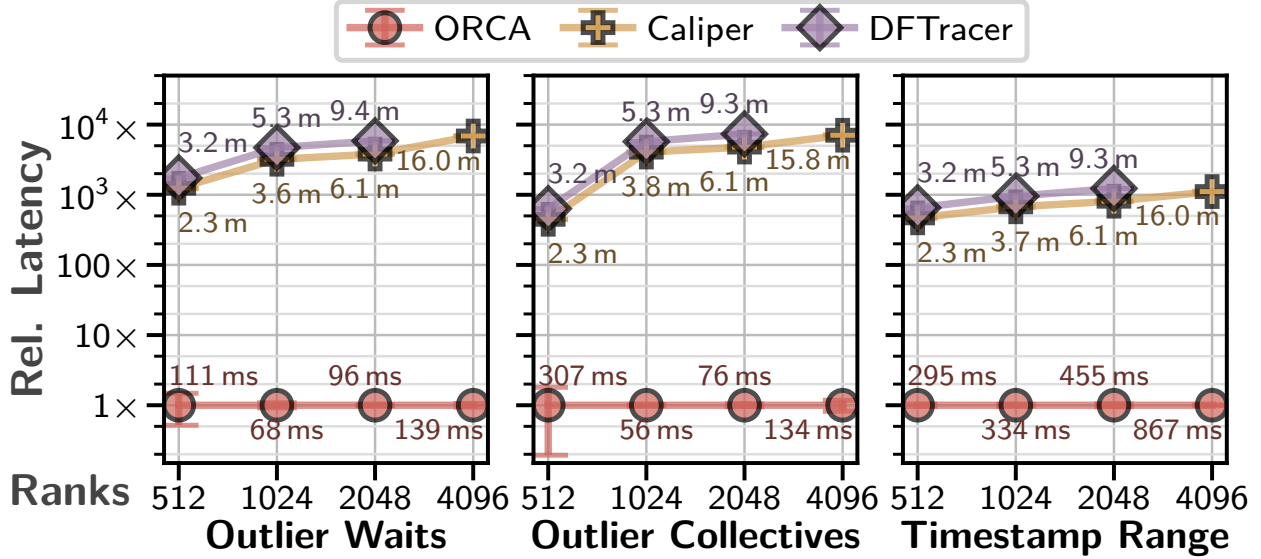


Figure 33: Post-mortem query performance. ORCA completes queries in hundreds of milliseconds, 3–4 orders of magnitude faster than DFTracer and Caliper. Performance for plaintext formats is dominated by parsing, not query complexity. TAU and Score-P are omitted as OTF2 is not a queryable format.

MPI send and receive, it computes a function event, a message event, and an event for messages received during certain phases such as `MPI.Wait`.

3. Flush-induced jitter, especially with Score-P and Caliper, creates stragglers that other ranks wait on by polling — generating additional events that inflate trace size.

Why ORCA Works. ORCA’s storage efficiency stems from two design choices. First, Parquet yields an inherently compact representation through binary columnar layout, dictionary encoding, and Snappy compression. Second, separating derived events from collection — computing them via OrcaFlow rather than baking them into the tracer — prevents record amplification from creeping in as features accumulate.

9.6.1.3 Query Performance

We evaluate analytics performance using a suite of post-mortem queries (Figure 33). All queries run on traces from a smaller 20-timestep run, executed cold with caches cleared, latencies averaged over 3 runs.

Queries. We use three queries of increasing complexity. *Outlier Waits* is a simple filter identifying `MPI.Wait` events exceeding 1 ms. *Outlier Collectives* aggregates collective events by `swid` and filters for outliers (max duration > 10 ms). *Timestamp Range* returns all events within the first second of execution—a query that might power a visualization frontend, and notably not a dimension ORCA partitions on.

Analytics Tools. All queries execute on a single 16-core node. DFTracer and Caliper use their

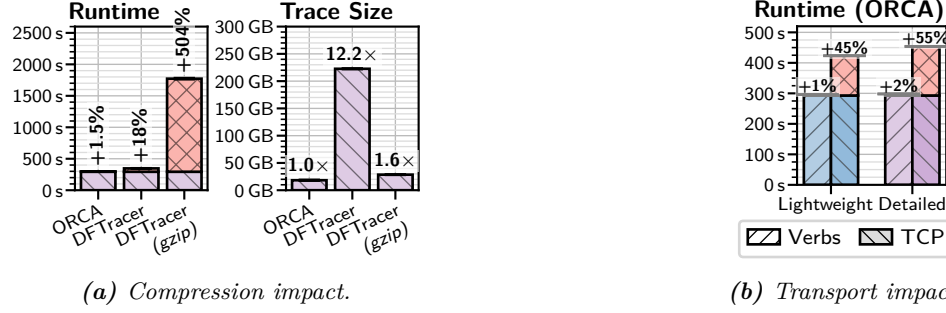


Figure 34: (Left) Compression reduces trace size but induces severe jitter on simulation ranks. ORCA avoids this by offloading compression to dedicated aggregators. (Right) TCP transport increases ORCA overhead from 1–2% to 45–55%. Arrow-RDMA co-design is essential for low overhead.

bundled Python-based analytics libraries. ORCA uses unmodified Polars [202] with no partition awareness for simplicity—a handicapped configuration as the metadata from all Parquet files must be scanned for all queries. TAU and Score-P are excluded as OTF2 is not a queryable format.

Findings.

- ORCA queries complete in hundreds of milliseconds, 3–4 orders of magnitude faster than DFTracer and Caliper. The gap widens with scale.
- Query times for DFTracer and Caliper do not vary with query complexity. This is because performance is bound by parsing, not computation—plaintext formats must be parsed and loaded as dataframes before any queries can be computed.

Why ORCA Works. While all analytics-oriented tracers ultimately load data into dataframes, they persist traces as plaintext formats requiring expensive ETL to materialize dataframes before any query executes. ORCA preserves dataframe structure end-to-end, persisting directly as Parquet—enabling zero-ETL post-hoc analytics. Additionally, the layout confers two kinds of pruning: file-level pruning via (timestep, rank) partitioning, and intra-file pruning via rowgroup statistics. Our benchmark does not exploit the former—unmodified Polars with lazy execution must scan all file footers. Adopting Parquet-based lakehouse metadata formats [203, 204] would add a catalog layer over partitioned Parquet, making partition pruning automatically available via standard query engines [91, 202]. Even without partition pruning, ORCA outperforms by 3–4 orders of magnitude. Conventional formats offer no such pruning—as traces grow, larger analytics clusters must be provisioned for ETL before queries can begin.

9.6.1.4 Tradeoff Analysis

We isolate the impact of key architectural decisions using a 512-rank run (Figure 34).

Compression. DFTracer supports gzip compression, which reduces its trace size from 12.2× that of ORCA to 1.6× (Fig. 34a), but at the cost of increasing runtime overhead from 18% to 504%. Compression on MPI ranks is unviable because it competes with the application for CPU, amplifying jitter. ORCA offloads compression to dedicated aggregator cores, keeping it off the

Flow	Query
Slow Wait Counts	SELECT COUNT(*) FROM mpi_wait WHERE duration > 1ms → DuckDB
Slow Wait Traces	SELECT * FROM mpi_wait WHERE duration > 1ms → Parquet
Load Imbal- ance	SELECT rank, SUM(duration) FROM kokkos WHERE name IN <compute kernels> GROUP BY rank → stats → Parquet PERCENTILES(stats, [50,90,99]) → DuckDB

Table 3: *In-situ workflows evaluated in Section 9.6.2, ranging from lightweight metrics and selective tracing on the communication path to aggregates over compute kernels.*

critical path. Additionally, compressed jsonl cannot be queried without full decompression, while ORCA’s causal partitioning and Parquet’s rowgroup structure reduce decompression overhead.

Transport Layer. Switching ORCA from `verbs` to the libfabric TCP provider increases overhead from 1–2% to 45–55% (Fig. 34b). Even with the same Arrow-based data path, TCP suffers because retrieval incurs CPU overhead on simulation ranks, introducing jitter. This gap reflects the complementary nature of ORCA’s transport choices: Arrow enables efficient serialization into contiguous RDMA-friendly buffers, and RDMA allows asynchronous retrieval with minimal CPU overhead while the application runs undisturbed. These results underscore the cost of TCP-based transport for tightly-coupled BSP workloads. GPU-based codes may tolerate the higher transport overhead, but event-based streaming tools still lack the columnar, causally-partitioned data path that enables ORCA’s analytical efficiency.

9.6.2 ORCA-Native In-Situ Workflows

While ORCA can enable conventional tracing workflows, it is fundamentally designed for in-situ telemetry analytics — directly materializing views from a running application. Plan operators can be pushed all the way to the MPI tier, leveraging the application’s own resources for data reduction at source. We now evaluate these capabilities, measuring the runtime overhead of operator pushdown, and its effectiveness at reducing captured data volume.

Workflows. Table 3 shows three flows of increasing complexity, designed for AMR codes. The first flow demonstrates a monitoring workflow: measuring the volume of `MPI.Wait` exceeding 1ms, while the second flow persists all outlier events to Parquet for offline analysis. The third flow tracks compute load imbalance in AMR codes — it filters for known compute kernels, computes duration aggregates by rank, and streams details to Parquet and percentiles to DuckDB for real-

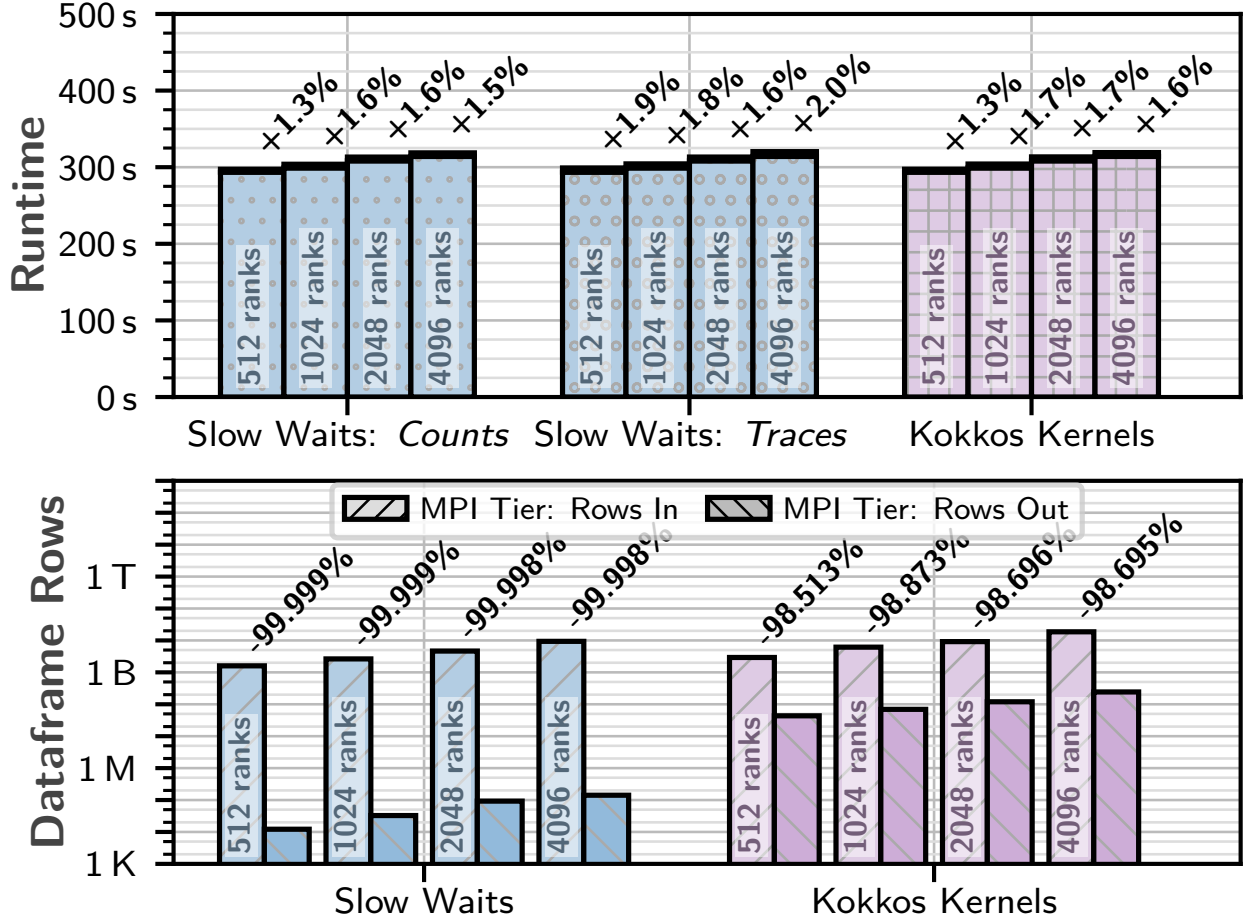


Figure 35: Runtime overhead (top) and MPI tier data reduction (bottom) for in-situ analytics with operator pushdown. Flows range from monitoring communication outliers to compute load balance. Pushdown discards 98.5–99.999% of data at sub-2% overhead, despite operators executing on MPI ranks.

time dashboards.

Observations. Figure 35 shows the runtime overhead and the data reduction at MPI for the three flows.

- All workflows incur an overhead of $\leq 2\%$ across all scales despite executing pushed-down operators on the MPI tier of a CPU simulation, illustrating the efficiency of Arrow-based analytics. Computation budget for pushdown only grows with modern GPU codes.
- Pushdown reduces transmitted data volume by 98.5% – 99.999%. Leveraging MPI nodes for analytics reduces fabric utilization, residual work at upper tiers, and shortens the feedback loop. The *capture-everything* nature of traditional tracing is redundant for monitoring and debugging—a small fraction of aggregated and filtered data suffices to materialize the required views.
- The `MPI_Wait` flows do not sample—they perform continuous predicate evaluation for every single `MPI_Wait`, emitting only outliers. Comprehensive continuous observability of this type is

impractical without pushdown.

9.6.3 Closing the Loop: Agentic Debugging

While ORCA’s control plane was designed for human operators, the emergence of LLM-based coding agents—capable of issuing commands and analyzing outputs—offers a way to validate the end-to-end feedback loop. To evaluate whether ORCA’s interfaces are expressive enough for autonomous debugging, we use an LLM agent as a proxy for a human operator—tasking it with localizing the anomaly from Figure 27, the same bug that took months to localize manually.

Setup. The anomaly is reproduced within the same AMR code used in previous experiments at 4096 ranks, using a placement policy that exacerbates the anomaly’s frequency. The code is instrumented with a hierarchy of Kokkos region annotations, with additional annotations in functions of interest added by us. Claude Code (Opus 4.5) [205] ran on the head node with access to the ORCA controller via CLI and the codebase. The agent was tasked with monitoring collectives for outliers and progressively collecting only the Kokkos and MPI events necessary to localize the problem. The LLM issued all ORCA commands and performed all data analysis using off-the-shelf Polars [202] on the head node.

Timeline. Figure 36 shows the debugging session: user and LLM interactions at top, per-timestep event volume at bottom. After a warmup phase, the LLM began the localization exercise at 272s by monitoring collectives for outliers (①).

1. At 384s, the LLM analyzed collective percentiles and identified outlier stragglers. It installed a flow to collect Kokkos events with duration > 1 ms (②).
2. At 510s, it began investigating collected Kokkos events, correlated outliers with the collective pattern, and attributed the anomaly to a `map.clear()` call during AMR redistribution.
3. At 1042s, the user rejected the `map.clear()` hypothesis as an inadequate explanation for latencies between 50–100ms. The LLM investigated further and identified that `map.clear()` triggered a buffer destructor calling `MPI.Wait`.
4. At 1191s, the user asked it to prove the `MPI.Wait` hypothesis using ORCA. The LLM proposed capturing all `MPI.Wait` events, but the user rejected this and asked for a duration-based filter. A revised flow with a 1 ms filter was accepted (③).
5. At 1335s, the LLM confirmed outlier waits as the root cause. Asked to trace the control flow leading to the anomalous `MPI.Wait`, it generated a report at 1605s. The application then resumed with baseline collective monitoring (①).

Observations. The exercise validates ORCA’s closed-loop capabilities. The agent autonomously correlated high-level telemetry with code, requested targeted detail, and traced the root cause to where instrumentation ended — completing in 27 minutes what took months of manual investigation. Reverting to baseline monitoring afterwards reduced data volume by 144×. User interventions were minor: rejecting an incomplete but accurate explanation and asking for basic duration-based filtering. The exercise was made easier with some additional instrumentation in relevant regions,

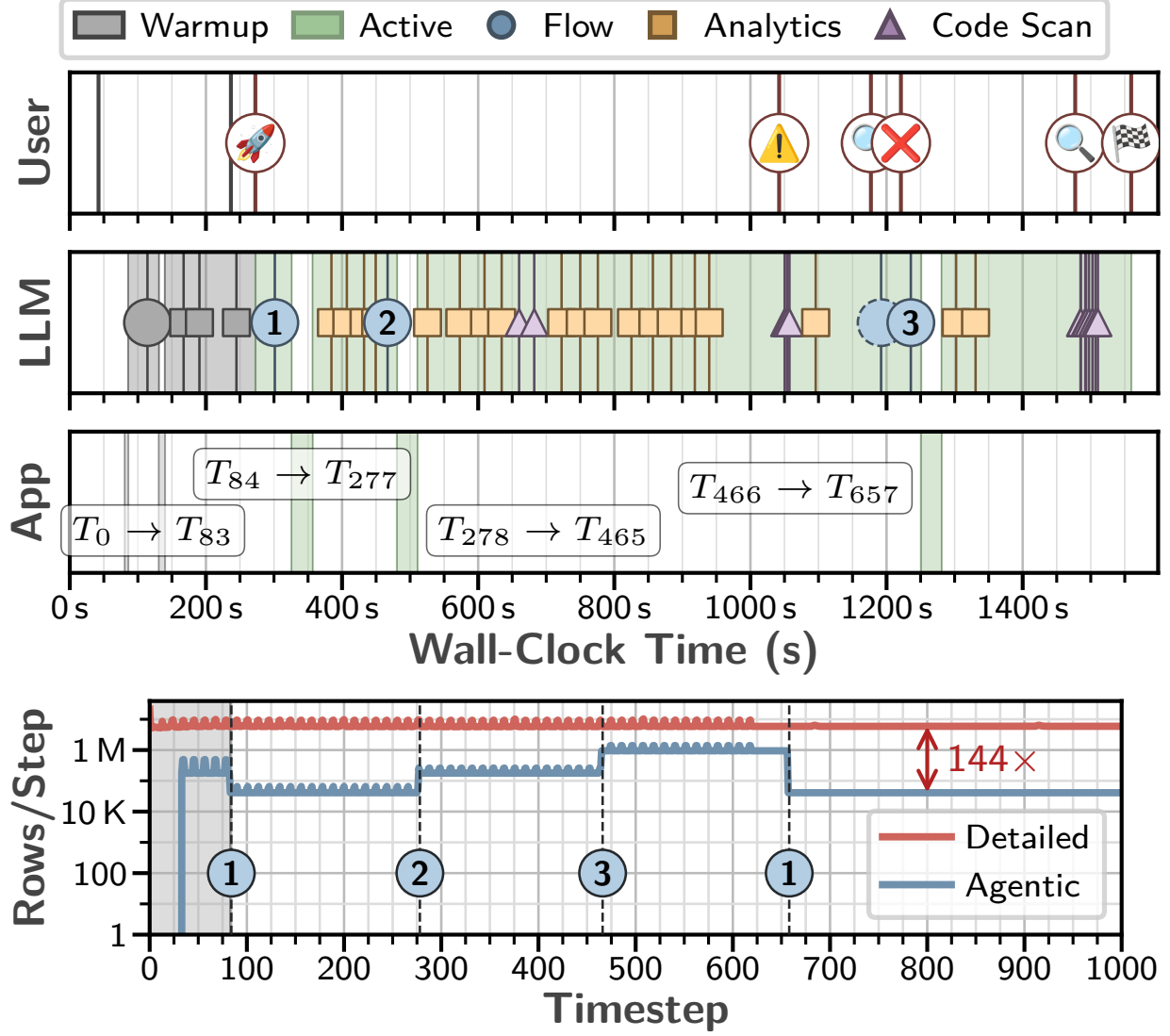


Figure 36: Visual transcript of an agentic debugging session demonstrating closed-loop capabilities. An LLM agent is tasked with localizing the anomaly from Fig. 27 using high-level metrics, progressively collecting targeted telemetry to identify the source. The agent operates largely autonomously, producing the complete call path in 27 minutes. Reverting to baseline monitoring reduces data volume by 144 $\times$.

but the result still demonstrates that ORCA’s feedback loop enables online localization workflows as long as the right instrumentation is present. Support for dynamic instrumentation [138, 200] would close this gap, enabling investigations to continue into the MPI and fabric layers.

Manual investigation had previously traced the anomalous spikes to misplaced ACKs within the fabric driver, triggering an expensive timeout/retransmit path — subsequently fixed with an asynchronous drain queue. Beyond localization, ORCA supports always-on production monitoring of mitigations: the same signature that identified the anomaly can be deployed as a metric, enabling regressions to be detected immediately.

9.7 Related Work

Tool Infrastructure and Tracing. MRNet [192] introduced TBONs for scalable tool infrastructure, but its compiled C++ filter model has not evolved with modern analytics. Fay [206] and Snicket [207] compile declarative queries into distributed probes that filter and aggregate at source. ORCA combines both: a TBON with Arrow interchange, declarative SQL, and BSP-aligned control. Dynamic instrumentation via eBPF [138] and DynInst [200] extends reach into runtime and kernel layers. Sketch-based telemetry reduction [208] aligns with ORCA’s in-situ aggregation model.

ML Training Observability. Recent ML training systems address pathologies from straggler localization [60, 168, 171, 172, 179] to fault diagnosis [169, 170] to training correctness [81], each building application-specific collection and analysis pipelines. Mycroft [187] uses collective structure for real-time root-cause analysis, working backward from stalls to causes. Unlike ORCA’s timestep dataframe abstraction, Mycroft analyzes approximate telemetry slices by querying a database, without completeness guarantees or in-situ reduction.

Unified Analytics. Arrow is emerging as common interchange for tabular HPC data —TileDB [85] and arrow-zarr [209] bridge array stores and SQL engines [82]. SmartNICs are being explored as execution targets for Arrow-based data pipelines [89, 210]— a declarative engine like ORCA’s could offload plan fragments to such devices transparently. In-situ indexing systems like DeltaFS [30] and CARP [31] deploy shufflers for embedded in-situ hash and range-partitioning. ORCA’s TBON could accommodate shuffling as an intra-tier operation. As these ecosystems converge, common infrastructure can subsume analytics workflows ranging from observability to checkpoint indexing to scientific queries.

9.8 Conclusions

BSP workloads require always-on observability as a production capability, capable of materializing actionable views and enabling rapid intervention. Traditional tracing works against this: the post-hoc paradigm incentivizes overcollection, requires large analytics clusters, and delays feedback. ORCA inverts this by pushing analytics to the source. Timestep dataframes and columnar representation enable multi-tier SQL analytics with operator pushdown—sub-2% overhead, up to 99.999% data reduction at source, agentic anomaly localization in minutes rather than months—all while contending for cluster resources with a live CPU simulation. GPU workloads and modern fabric QoS capabilities only increase the headroom available for such pushed-down workflows. Views materialize on demand, so only the stories—not the raw data—exit the fabric. The hero run need not be followed by a hero analysis.

References

- [1] Sambit Das et al. “Large-Scale Materials Modeling at Quantum Accuracy: Ab Initio Simulations of Quasicrystals and Interacting Extended Defects in Metallic Alloys”. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. URL: <https://dl.acm.org/doi/10.1145/3581784.3627037>.
- [2] Ryan Stocks et al. “Breaking the Million-Electron and 1 EFLOP/s Barriers: Biomolecular-Scale Ab Initio Molecular Dynamics Using MP2 Potentials”. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*. SC ’24. URL: <https://dl.acm.org/doi/10.1109/SC41406.2024.00015>.
- [3] Aaron Grattafiori et al. *The Llama 3 Herd of Models*. arXiv: 2407.21783 [cs]. URL: <http://arxiv.org/abs/2407.21783>. Pre-published.
- [4] *Meta Trains Llama 4 on a 100,000+ H100 GPU Supercluster - NADDOD Blog*. URL: <https://www.naddod.com/blog/meta-trains-llama-4-on-a-100-000-h100-gpu-supercluster>.
- [5] Robi Rahman. *AI Training Cluster Sizes Increased by More than 20x since 2016*. Epoch AI. URL: <https://epoch.ai/data-insights/training-cluster-size>.
- [6] William M. Jones, John T. Daly, and Nathan DeBardeleben. “Application Monitoring and Checkpointing in HPC: Looking towards Exascale Systems”. In: *Proceedings of the 50th Annual Southeast Regional Conference*. URL: <https://dl.acm.org/doi/10.1145/2184512.2184574>.
- [7] Assaf Eisenman et al. “{Check-N-Run}: A Checkpointing System for Training Deep Learning Recommendation Models”. In: URL: <https://www.usenix.org/conference/nsdi22/presentation/eisenman>.
- [8] Xinyu Lian et al. *Universal Checkpointing: A Flexible and Efficient Distributed Checkpointing System for Large-Scale DNN Training with Reconfigurable Parallelis*. Version 3. arXiv: 2406.18820 [cs]. URL: <http://arxiv.org/abs/2406.18820>. Pre-published.
- [9] Torsten Hoeftler, Timo Schneider, and Andrew Lumsdaine. “Characterizing the Influence of System Noise on Large-Scale Applications by Simulation”. In: *SC ’10: Proceedings of the 2010 ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis*. URL: <https://ieeexplore.ieee.org/document/5645455>.
- [10] Fabrizio Petrini, Darren J. Kerbyson, and Scott Pakin. “The Case of the Missing Supercomputer Performance: Achieving Optimal Performance on the 8,192 Processors of ASCI Q”. In: *Proceedings of the 2003 ACM/IEEE Conference on Supercomputing*. URL: <https://dl.acm.org/doi/10.1145/1048935.1050204>.
- [11] Orcun Yildiz et al. “On the Root Causes of Cross-Application I/O Interference in HPC Storage Systems”. In: *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. URL: <https://ieeexplore.ieee.org/document/7516071>.
- [12] Brian Kocoloski and John Lange. “Varbench: An Experimental Framework to Measure and Characterize Performance Variability”. In: *Proceedings of the 47th International Conference on Parallel Processing*. URL: <https://dl.acm.org/doi/10.1145/3225058.3225125>.
- [13] Bilge Acun, Phil Miller, and Laxmikant V. Kale. “Variation Among Processors Under Turbo Boost in HPC Systems”. In: *Proceedings of the 2016 International Conference on Supercomputing*. URL: <https://dl.acm.org/doi/10.1145/2925426.2926289>.
- [14] A. C. Bauer et al. “In Situ Methods, Infrastructures, and Applications on High Performance Computing Platforms”. In: *Computer Graphics Forum* 35.3 (). URL: <https://onlinelibrary.wiley.com/doi/10.1111/cgf.12930>.
- [15] Hank Childs et al. “A Terminology for in Situ Visualization and Analysis Systems”. In: *The International Journal of High Performance Computing Applications* 34.6 (). URL: <https://doi.org/10.1177/1094342020935991>.
- [16] Matthew Larsen et al. “Ascent: A Flyweight In Situ Library for Exascale Simulations”. In: *In Situ Visualization for Computational Science*. Ed. by Hank Childs, Janine C. Bennett, and Christoph Garth.
- [17] Utkarsh Ayachit et al. “ParaView Catalyst: Enabling In Situ Data Analysis and Visualization”. In: *Proceedings of the First Workshop on In Situ Infrastructures for Enabling Extreme-Scale Analysis and Visualization*. URL: <https://dl.acm.org/doi/10.1145/2828612.2828624>.

- [18] Seif Ibrahim et al. “Interactive in Situ Visualization and Analysis Using Ascent and Jupyter”. In: *Proceedings of the Workshop on In Situ Infrastructures for Enabling Extreme-Scale Analysis and Visualization*. URL: <https://dl.acm.org/doi/10.1145/3364228.3364232>.
- [19] Pascal Grosset and James Ahrens. “Lightweight Interface for In Situ Analysis and Visualization of Particle Data”. In: *ISAV’21: In Situ Infrastructures for Enabling Extreme-Scale Analysis and Visualization*. URL: <https://dl.acm.org/doi/10.1145/3490138.3490143>.
- [20] Matthew Larsen et al. “The ALPINE In Situ Infrastructure: Ascending from the Ashes of Strawman”. In: *Proceedings of the In Situ Infrastructures on Enabling Extreme-Scale Analysis and Visualization*. ISAV’17. URL: <https://dl.acm.org/doi/10.1145/3144769.3144778>.
- [21] Greg Eisenhauer et al. *Streaming Data in HPC Workflows Using ADIOS*. arXiv: 2410.00178 [cs]. URL: <http://arxiv.org/abs/2410.00178>. Pre-published.
- [22] Bin Dong, Surendra Byna, and Kesheng Wu. “SDS-sort: Scalable Dynamic Skew-Aware Parallel Sorting”. In: *HPDC 2016 - Proceedings of the 25th ACM International Symposium on High-Performance Parallel and Distributed Computing*.
- [23] Hari Sundar, Dhairya Malhotra, and George Biros. “HykSort: A New Variant of Hypercube Quicksort on Distributed Memory Architectures”. In: *Proceedings of the International Conference on Supercomputing*.
- [24] Alexander Rasmussen et al. “TritonSort: A Balanced Large-Scale Sorting System”. In: *Proceedings of the 8th USENIX Conference on Networked Systems Design and Implementation*. NSDI’11.
- [25] Benjamin Klenk and Holger Fröning. “An Overview of MPI Characteristics of Exascale Proxy Applications”. In: *High Performance Computing: 32nd International Conference, ISC High Performance 2017, Frankfurt, Germany, June 18–22, 2017, Proceedings*. URL: https://doi.org/10.1007/978-3-319-58667-0_12.
- [26] Anshu Dubey et al. “A Survey of High Level Frameworks in Block-Structured Adaptive Mesh Refinement Packages”. In: *Journal of Parallel and Distributed Computing* 74.12 (). URL: <https://linkinghub.elsevier.com/retrieve/pii/S0743731514001178>.
- [27] Brian J. N. Wylie et al. “15+ Years of Joint Parallel Application Performance Analysis/Tools Training with Scalasca/Score-P and Paraver/Extrac Toolsets”. In: *Future Generation Computer Systems* 162 (). URL: <https://www.sciencedirect.com/science/article/pii/S0167739X24004187>.
- [28] Eschweiler Dominic et al. “Open Trace Format 2: The Next Generation of Scalable Trace Formats and Support Libraries”. In: *Advances in Parallel Computing*. URL: <https://www.medra.org/servlet/aliasResolver?alias=iospress&issn=0927-5452&volume=22&spage=481>.
- [29] Surendra Byna et al. “Parallel I/O, Analysis, and Visualization of a Trillion Particle Simulation”. In: *International Conference for High Performance Computing, Networking, Storage and Analysis, SC*.
- [30] Qing Zheng et al. “Scaling Embedded In-Situ Indexing with DeltaFS”. In: *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*. URL: <https://ieeexplore.ieee.org/document/8665820>.
- [31] Ankush Jain et al. “CARP: Range Query-Optimized Indexing for Streaming Data”. In: *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*.
- [32] Carsten Burstedde et al. “Scalable Adaptive Mantle Convection Simulation on Petascale Supercomputers”. In: *2008 SC - International Conference for High Performance Computing, Networking, Storage and Analysis*. URL: <http://ieeexplore.ieee.org/document/5214248/>.
- [33] Ankush Jain et al. “Lessons from Profiling and Optimizing Placement in AMR Codes”. In: *2025 IEEE International Conference on Cluster Computing (CLUSTER)*.
- [34] *Pytorch/Kineto*. pytorch. URL: <https://github.com/pytorch/kineto>.
- [35] *Apache Arrow*. Apache Arrow. URL: <https://arrow.apache.org/>.
- [36] *TOP500 List - June 2025 — TOP500*. URL: <https://top500.org/lists/top500/list/2025/06/>.
- [37] Gregory F Pfister. “An Introduction to the Infiniband Architecture”. In: *High performance mass storage and parallel I/O* 42.617–632 ().
- [38] Duncan Roweth et al. “HPE Slingshot Launched into Network Space”. In: *Cray User Group* ().

- [39] J Metz. “Empowering AI Workloads in Ultra Ethernet Consortium”. In: *2024 IEEE Photonics Society Summer Topicals Meeting Series (SUM)*. URL: <https://ieeexplore.ieee.org/document/10614558/>.
- [40] Nathan DeBardeleben et al. “GPU Behavior on a Large HPC Cluster”. In: *Euro-Par 2013: Parallel Processing Workshops*. Ed. by Dieter an Mey et al.
- [41] Addison Snell and Laura Segervall. “HPC Application Support for GPU Computing”. In: ().
- [42] Peter J Braam and Philip Schwan. “Lustre: The Intergalactic File System”. In: *Ottawa Linux Symposium*. Vol. 8. 11.
- [43] Michael Hennecke. “Daos: A Scale-out High Performance Storage Stack for Storage Class Memory”. In: *Supercomputing frontiers* 40 ().
- [44] *Hammerspace Data Platform*. Hammerspace. URL: <https://hammerspace.com/hammerspace-global-data-platform/>.
- [45] *The VAST Platform White Paper*. URL: <https://www.vastdata.com>.
- [46] Glenn Lockwood. *LLM Training without a Parallel File System*. URL: <https://blog.glennklockwood.com/2025/02/llm-training-without-parallel-file.html>.
- [47] Gary Grider. “LANL Platform Planning and Update” (HPC User Forum 2023). URL: <https://hpcuserforum.com/wp-content/uploads/2023/09/Gary-Grider-LANL-Platform-Planning-and-Update.pdf>.
- [48] Robert Bird et al. “VPIC 2.0: Next Generation Particle-in-Cell Simulations”. In: *IEEE Transactions on Parallel and Distributed Systems* 33.4 (). URL: <https://ieeexplore.ieee.org/document/9444146/>.
- [49] “MPI: A Message-Passing Interface Standard”. In: ().
- [50] *NVIDIA Collective Communications Library (NCCL)*. NVIDIA Developer. URL: <https://developer.nvidia.com/nccl>.
- [51] Adithya Gangidi et al. “RDMA over Ethernet for Distributed Training at Meta Scale”. In: *Proceedings of the ACM SIGCOMM 2024 Conference*. URL: <https://dl.acm.org/doi/10.1145/3651890.3672233>.
- [52] Bing Xie et al. “Characterizing Output Bottlenecks in a Supercomputer”. In: *SC ’12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. URL: <https://ieeexplore.ieee.org/document/6468446/>.
- [53] Pandian Raju et al. “PebblesDB: Building Key-Value Stores Using Fragmented Log-Structured Merge Trees”. In: *SOSP 2017 - Proceedings of the 26th ACM Symposium on Operating Systems Principles*.
- [54] Utkarsh Ayachit et al. “The SENSEI Generic In Situ Interface”. In: *2016 Second Workshop on In Situ Infrastructures for Enabling Extreme-Scale Analysis and Visualization (ISAV)*. URL: <http://ieeexplore.ieee.org/document/7836400/>.
- [55] Ciprian Docan, Manish Parashar, and Scott Klasky. “DataSpaces: An Interaction and Coordination Framework for Coupled Simulation Workflows”. In: *Proceedings of the 19th ACM International Symposium on High Performance Distributed Computing*. HPDC ’10. URL: <https://dl.acm.org/doi/10.1145/1851476.1851481>.
- [56] Benjamin Schwaller. *Integrated System and Application Continuous Performance Monitoring and Analysis Capability (Final)*. SAND2021-11954, 1822583, 700207. URL: <https://www.osti.gov/servlets/purl/1822583/>.
- [57] Sameer S. Shende and Allen D. Malony. “The Tau Parallel Performance System”. In: *Int. J. High Perform. Comput. Appl.* 20.2 (). URL: <https://doi.org/10.1177/1094342006064482>.
- [58] *Holistic Trace Analysis — Holistic Trace Analysis 0.5.0 Documentation*. URL: <https://hta.readthedocs.io/en/latest/index.html>.
- [59] Hariharan Devarajan et al. “DFTracer: An Analysis-Friendly Data Flow Tracer for AI-Driven Workflows”. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*. SC ’24. URL: <https://dl.acm.org/doi/10.1109/SC41406.2024.00023>.
- [60] Weihao Cui et al. *XPUTimer: Anomaly Diagnostics for Divergent LLM Training in GPU Clusters of Thousand-Plus Scale*. Version 1. arXiv: 2502.05413. URL: <http://arxiv.org/abs/2502.05413>. Pre-published.

- [61] *Characterization of the DOE Mini-apps*. URL: <https://portal.nersc.gov/project/CAL/doe-miniapps.htm>.
- [62] Philipp Grete et al. “Parthenon—a Performance Portable Block-Structured Adaptive Mesh Refinement Framework”. In: *The International Journal of High Performance Computing Applications* 37.5 (). URL: <https://doi.org/10.1177/10943420221143775>.
- [63] Weiqun Zhang et al. “AMReX: Block-structured Adaptive Mesh Refinement for Multiphysics Applications”. In: *The International Journal of High Performance Computing Applications* 35.6 (). URL: <https://doi.org/10.1177/10943420211022811>.
- [64] *Pandas - Python Data Analysis Library*. URL: <https://pandas.pydata.org/>.
- [65] Robert Schulze et al. “ClickHouse - Lightning Fast Analytics for Everyone”. In: *Proceedings of the VLDB Endowment* 17.12 (). URL: <https://dl.acm.org/doi/10.14778/3685800.3685802>.
- [66] M. S. Warren and J. K. Salmon. “A Parallel Hashed Oct-Tree N-body Algorithm”. In: *Proceedings of the 1993 ACM/IEEE Conference on Supercomputing - Supercomputing '93*. URL: <http://portal.acm.org/citation.cfm?doid=169627.169640>.
- [67] Aparna Sasidharan, John M. Dennis, and Marc Snir. “A General Space-filling Curve Algorithm for Partitioning 2D Meshes”. In: *2015 IEEE 17th International Conference on High Performance Computing and Communications, 2015 IEEE 7th International Symposium on Cyberspace Safety and Security, and 2015 IEEE 12th International Conference on Embedded Software and Systems*. URL: <https://ieeexplore.ieee.org/document/7336274/>.
- [68] Abhinav Bhatele et al. “Applying Graph Partitioning Methods in Measurement-Based Dynamic Load Balancing”. In: ().
- [69] Umit V. Catalyurek et al. “Hypergraph-Based Dynamic Load Balancing for Adaptive Scientific Computations”. In: *2007 IEEE International Parallel and Distributed Processing Symposium*. URL: <http://ieeexplore.ieee.org/document/4227986/>.
- [70] Dominique Lasalle and George Karypis. “Multi-Threaded Graph Partitioning”. In: *2013 IEEE 27th International Symposium on Parallel and Distributed Processing*. URL: <https://ieeexplore.ieee.org/document/6569814>.
- [71] Carsten Burstedde, Lucas C. Wilcox, and Omar Ghattas. “P4est: Scalable Algorithms for Parallel Adaptive Mesh Refinement on Forests of Octrees”. In: *SIAM Journal on Scientific Computing* 33.3 (). URL: <https://epubs.siam.org/doi/10.1137/100791634>.
- [72] R. L. Graham. “Bounds on Multiprocessing Timing Anomalies”. In: *SIAM Journal on Applied Mathematics* 17.2 (). URL: <https://epubs.siam.org/doi/10.1137/0117039>.
- [73] Brandon Barker et al. *Phoebus: Performance Portable GRRMHD for Relativistic Astrophysics*. Version 2. arXiv: 2410.09146 [astro-ph]. URL: <http://arxiv.org/abs/2410.09146>. Pre-published.
- [74] Sedov, Leonid Ivanovich. *Similarity and Dimensional Methods in Mechanics*. URL: <http://archive.org/details/l-i-sedov.-similarity-and-dimensional-methods-in-mechanics>.
- [75] Haryadi S. Gunawi et al. “Fail-Slow at Scale: Evidence of Hardware Performance Faults in Large Production Systems”. In: *ACM Transactions on Storage* 14.3 (). URL: <https://dl.acm.org/doi/10.1145/3242086>.
- [76] Bilge Acun and Laxmikant V. Kale. “Mitigating Processor Variation through Dynamic Load Balancing”. In: *2016 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. URL: <http://ieeexplore.ieee.org/document/7529982/>.
- [77] Andreas Knüpfer et al. “Score-P: A Joint Performance Measurement Run-Time Infrastructure for Periscope, Scalasca, TAU, and Vampir”. In: *Tools for High Performance Computing 2011*. Ed. by Holger Brunst et al.
- [78] *InfluxDB — Real-time Insights at Any Scale*. InfluxData. Sat, 15 Jan 2022 15:32:09 +0000. URL: <https://www.influxdata.com/home/>.
- [79] *Prometheus - Monitoring System & Time Series Database*. URL: <https://prometheus.io/>.
- [80] Adam Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems*. Vol. 32. URL: <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>.

- [81] Yuxuan Jiang et al. *Training with Confidence: Catching Silent Errors in Deep Learning Training with Automated Proactive Checks*. arXiv: 2506.14813 [cs]. URL: <http://arxiv.org/abs/2506.14813>. Pre-published.
- [82] Andrew Lamb et al. “Apache Arrow DataFusion: A Fast, Embeddable, Modular Analytic Query Engine”. In: *Companion of the 2024 International Conference on Management of Data*. URL: <https://dl.acm.org/doi/10.1145/3626246.3653368>.
- [83] Jerome Soumagne et al. “Mercury: Enabling Remote Procedure Call for High-Performance Computing”. In: *2013 IEEE International Conference on Cluster Computing (CLUSTER)*. URL: <http://ieeexplore.ieee.org/document/6702617/>.
- [84] Paul Grun et al. “A Brief Introduction to the OpenFabrics Interfaces - A New Network API for Maximizing High Performance Application Efficiency”. In: *2015 IEEE 23rd Annual Symposium on High-Performance Interconnects*. URL: <http://ieeexplore.ieee.org/document/7312664/>.
- [85] *TileDB • Designed for Discovery*. TileDB. URL: <https://tiledb.com/>.
- [86] Byung S Lee and Ron Musick. “MeshSQL: The Query Language for Simulation Mesh Data”. In: *Information Sciences* 159.3 (). URL: <https://www.sciencedirect.com/science/article/pii/S0020025503001981>.
- [87] M. Kersten et al. “SciQL, a Query Language for Science Applications”. In: *Proceedings of the EDBT/ICDT 2011 Workshop on Array Databases*. AD ’11. URL: <https://dl.acm.org/doi/10.1145/1966895.1966896>.
- [88] Alireza Rezaei Mahdiraji. “Toward Unstructured Mesh Algebra and Query Language”. In: *Proceedings of the 2014 SIGMOD PhD Symposium*. SIGMOD’14 PhD Symposium. URL: <https://dl.acm.org/doi/10.1145/2602622.2602626>.
- [89] Jianshen Liu et al. *Processing Particle Data Flows with SmartNICs*. arXiv: 2210.06827 [cs]. URL: <http://arxiv.org/abs/2210.06827>. Pre-published.
- [90] Craig Ulmer et al. “Extending Composable Data Services into SmartNICs”. In: *2023 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*.
- [91] Mark Raasveldt and Hannes Mühleisen. “DuckDB: An Embeddable Analytical Database”. In: *Proceedings of the 2019 International Conference on Management of Data*. URL: <https://dl.acm.org/doi/10.1145/3299869.3320212>.
- [92] *Arrow Flight SQL — Apache Arrow V20.0.0*. URL: <https://arrow.apache.org/docs/format/FlightSql.html>.
- [93] *Grafana — Query, Visualize, Alerting Observability Platform*. Grafana Labs. URL: <https://grafana.com/grafana/>.
- [94] *Parquet*. Apache Parquet. URL: <https://parquet.apache.org/>.
- [95] Hank Childs. “In Situ Processing”. In: (). URL: <https://escholarship.org/uc/item/3st8x19d>.
- [96] MikeRayMSFT. *Clustered and Nonclustered Indexes - SQL Server*. URL: <https://learn.microsoft.com/en-us/sql/relational-databases/indexes/clustered-and-nonclustered-indexes-described?view=sql-server-ver16>.
- [97] Mike Ray. *Clustered and Nonclustered Indexes - SQL Server*. URL: <https://learn.microsoft.com/en-us/sql/relational-databases/indexes/clustered-and-nonclustered-indexes-described?view=sql-server-ver16>.
- [98] Jerry Chou, Kesheng Wu, and Prabhat. “FastQuery: A Parallel Indexing System for Scientific Data”. In: *2011 IEEE International Conference on Cluster Computing*. URL: <https://ieeexplore.ieee.org/document/6061134>.
- [99] Prasanna Ganesan, Mayank Bawa, and Hector Garcia-Molina. “Online Balancing of Range-Partitioned Data with Applications to Peer-to-Peer Systems”. In: *Proceedings of the Thirtieth International Conference on Very Large Data Bases - Volume 30* (Toronto, Canada). VLDB ’04.
- [100] Ioannis Konstantinou, Dimitrios Tsoumakos, and Nectarios Koziris. “Fast and Cost-Effective Online Load-Balancing in Distributed Range-Queriable Systems”. In: *IEEE Transactions on Parallel and Distributed Systems* 22.8 ().

- [101] Yifan Qiao et al. “Closing the B+-Tree vs. LSM-tree Write Amplification Gap on Modern Storage Hardware with Built-in Transparent Compression”. In: *20th USENIX Conference on File and Storage Technologies (FAST 22)*. URL: <https://www.usenix.org/conference/fast22/presentation/qiao>.
- [102] K. Mani Chandy and Leslie Lamport. “Distributed Snapshots: Determining Global States of Distributed Systems”. In: *ACM Transactions on Computer Systems (TOCS)* ().
- [103] Qing Zheng et al. “Software-Defined Storage for Fast Trajectory Queries Using a deltaFS Indexed Massive Directory”. In: *Proceedings of PDSW-DISCS 2017 - 2nd Joint International Workshop on Parallel Data Storage and Data Intensive Scalable Computing Systems - Held in Conjunction with SC 2017: The International Conference for High Performance Computing, Networking, Storage and Analysis*.
- [104] Robert B. Ross et al. “Mochi: Composing Data Services for High-Performance Computing Environments”. In: *Journal of Computer Science and Technology* 35.1 (). URL: <https://doi.org/10.1007/s11390-020-9802-0>.
- [105] Rolf Rabenseifner. “Optimization of Collective Reduction Operations”. In: *Computational Science - ICCS 2004*. Ed. by Marian Bubak et al.
- [106] Rolf Rabenseifner and Jesper Larsson Träff. “More Efficient Reduction Algorithms for Non-Power-of-Two Number of Processors in Message-Passing Parallel Systems”. In: *Recent Advances in Parallel Virtual Machine and Message Passing Interface*. Ed. by Dieter Kranzlmüller, Péter Kacsuk, and Jack Dongarra.
- [107] Benny Vigil. *Trinity Advanced Technology System Overview*. LA-UR-14-28143, 1160100. URL: <https://www.osti.gov/servlets/purl/1160100/>.
- [108] Brian F. Cooper et al. “Benchmarking Cloud Serving Systems with YCSB”. In: *Proceedings of the 1st ACM Symposium on Cloud Computing*. SoCC '10. URL: <https://dl.acm.org/doi/10.1145/1807128.1807152>.
- [109] Timothy Prickett Morgan. *Aurora Rising: A Massive Machine For HPC And AI*. The Next Platform. URL: <https://www.nextplatform.com/2023/05/23/aurora-rising-a-massive-machine-for-hpc-and-ai/>.
- [110] Patrick O’Neil et al. “The Log-Structured Merge-Tree (LSM-tree)”. In: *Acta Informatica* ().
- [111] Douglas Comer. “Ubiquitous B-Tree”. In: *ACM Comput. Surv.* 11.2 (). URL: <https://dl.acm.org/doi/10.1145/356770.356776>.
- [112] Yuhong Zhong et al. “XRP: In-Kernel Storage Functions with eBPF”. In: ().
- [113] Pandian Raju et al. “PebblesDB: Building Key-Value Stores Using Fragmented Log-Structured Merge Trees”. In: *SOSP 2017 - Proceedings of the 26th ACM Symposium on Operating Systems Principles*.
- [114] Fang Zheng et al. “PreData – Preparatory Data Analytics on Peta-Scale Machines”. In: *2010 IEEE International Symposium on Parallel & Distributed Processing (IPDPS)*. URL: <https://ieeexplore.ieee.org/document/5470454>.
- [115] Venkatram Vishwanath, Mark Hereld, and Michael E. Papka. “Toward Simulation-Time Data Analysis and I/O Acceleration on Leadership-Class Systems”. In: *2011 IEEE Symposium on Large Data Analysis and Visualization*. URL: <https://ieeexplore.ieee.org/document/6092178>.
- [116] Venkatram Vishwanath et al. “Topology-Aware Data Movement and Staging for I/O Acceleration on Blue Gene/P Supercomputing Systems”. In: *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*. URL: <https://dl.acm.org/doi/10.1145/2063384.2063409>.
- [117] Ron A. Oldfield et al. “Trilinos I/O Support Trios”. In: *Scientific Programming* 20.2 (). URL: <https://dl.acm.org/doi/abs/10.1155/2012/842791>.
- [118] Junmin Gu et al. “Querying Large Scientific Data Sets with Adaptable IO System ADIOS”. In: *Supercomputing Frontiers*. Ed. by Rio Yokota and Weigang Wu.
- [119] Kesheng Wu. “FastBit: An Efficient Indexing Technology for Accelerating Data-Intensive Science”. In: *Journal of Physics: Conference Series* 16.1 (). URL: <https://dx.doi.org/10.1088/1742-6596/16/1/077>.
- [120] Will Usher et al. “Adaptive Spatially Aware I/O for Multiresolution Particle Data Layouts”. In: *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)* (). URL: <https://ieeexplore.ieee.org/document/9460496/>.

- [121] Matthaios Olma et al. “Slalom: Coasting through Raw Data via Adaptive Partitioning and Indexing”. In: *Proceedings of the VLDB Endowment* 10.10 (). URL: <https://dl.acm.org/doi/10.14778/3115404.3115415>.
- [122] Stavros Maroulis et al. “Resource-Aware Adaptive Indexing for in Situ Visual Exploration and Analytics”. In: *The VLDB Journal* 32.1 (). URL: <https://doi.org/10.1007/s00778-022-00739-z>.
- [123] Lanyue Lu et al. “WiscKey: Separating Keys from Values in SSD-Conscious Storage”. In: *ACM Trans. Storage* 13.1 (). URL: <https://dl.acm.org/doi/10.1145/3033273>.
- [124] Huanchen Zhang et al. “SuRF: Practical Range Query Filtering with Fast Succinct Tries”. In: *Proceedings of the 2018 International Conference on Management of Data*. SIGMOD ’18. URL: <https://dl.acm.org/doi/10.1145/3183713.3196931>.
- [125] Michael P Vittingow. “A Decade of Adaptive Mesh Refinement in CTH”. In: ().
- [126] M. T. P. Liska et al. “H-AMR: A New GPU-accelerated GRMHD Code for Exascale Computing with 3D Adaptive Mesh Refinement and Local Adaptive Time Stepping”. In: *The Astrophysical Journal Supplement Series* 263.2 (). URL: <https://dx.doi.org/10.3847/1538-4365/ac9966>.
- [127] Ankush Jain. *AMR Tools*. URL: <https://github.com/pdlfs/amr-tools>.
- [128] Brian Van Straalen et al. “Scalability Challenges for Massively Parallel AMR Applications”. In: *2009 IEEE International Symposium on Parallel & Distributed Processing*. URL: <http://ieeexplore.ieee.org/document/5161014/>.
- [129] Florian Schornbaum and Ulrich Rüde. “Extreme-Scale Block-Structured Adaptive Mesh Refinement”. In: *SIAM Journal on Scientific Computing* 40.3 (). arXiv: 1704.06829 [cs]. URL: <http://arxiv.org/abs/1704.06829>.
- [130] Bilge Acun et al. “Parallel Programming with Migratable Objects: Charm++ in Practice”. In: *SC ’14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. URL: <https://ieeexplore.ieee.org/abstract/document/7013040>.
- [131] Qingyu Meng, Justin Luitjens, and Martin Berzins. “Dynamic Task Scheduling for the Uintah Framework”. In: *2010 3rd Workshop on Many-Task Computing on Grids and Supercomputers*. URL: <https://ieeexplore.ieee.org/document/5699431/?arnumber=5699431>.
- [132] Gregor Daiß et al. *Asynchronous-Many-Task Systems: Challenges and Opportunities – Scaling an AMR Astrophysics Code on Exascale Machines Using Kokkos and HPX*. arXiv: 2412.15518 [cs]. URL: <http://arxiv.org/abs/2412.15518>. Pre-published.
- [133] Michael Bauer et al. “Legion: Expressing Locality and Independence with Logical Regions”. In: *SC ’12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. URL: <https://ieeexplore.ieee.org/document/6468504>.
- [134] Andrew Emerick. “Enzo-E/Cello Project: Enabling Exa-Scale Astrophysics”. In: ().
- [135] Brian T.N. Gunney and Robert W. Anderson. “Advances in Patch-Based Adaptive Mesh Refinement Scalability”. In: *Journal of Parallel and Distributed Computing* 89 (). URL: <https://linkinghub.elsevier.com/retrieve/pii/S0743731515002129>.
- [136] Torsten Hoefler et al. “A Case for Standard Non-blocking Collective Operations”. In: *Recent Advances in Parallel Virtual Machine and Message Passing Interface*. Ed. by Franck Cappello, Thomas Herault, and Jack Dongarra. Vol. 4757. URL: http://link.springer.com/10.1007/978-3-540-75416-9_22.
- [137] *Perf: Linux Profiling with Performance Counters*. URL: <https://perfwiki.github.io/main/>.
- [138] *What Is eBPF? An Introduction and Deep Dive into the eBPF Technology*. URL: <https://ebpf.io/what-is-ebpf/>.
- [139] Dieter an Mey et al. “Score-P: A Unified Performance Measurement System for Petascale Applications”. In: *Competence in High Performance Computing 2010*. Ed. by Christian Bischof et al.
- [140] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-completeness*.
- [141] Dhabaleswar Kumar Panda et al. “The MVAPICH Project: Transforming Research into High-Performance MPI Library for HPC Community”. In: *Journal of Computational Science*. Case Studies in Translational Computer Science 52 (). URL: <https://www.sciencedirect.com/science/article/pii/S1877750320305093>.
- [142] “Intel® Performance Scaled Messaging 2 (PSM2) — Programmer’s Guide”. In: ().

- [143] Ruiming Lu et al. “PERSEUS: A Fail-Slow Detection Framework for Cloud Storage Systems”. In: ().
- [144] Tobias Hilbrich et al. “MUST: A Scalable Approach to Runtime Error Detection in MPI Programs”. In: *Tools for High Performance Computing 2009*. Ed. by Matthias S. Müller et al. URL: https://link.springer.com/10.1007/978-3-642-11261-4_5.
- [145] Simon D. Hammond et al. “Profiling and Debugging Support for the Kokkos Programming Model”. In: *High Performance Computing*. Ed. by Rio Yokota et al. Vol. 11203. URL: http://link.springer.com/10.1007/978-3-030-02465-9_53.
- [146] Leslie Lamport. “Time, Clocks, and the Ordering of Events in a Distributed System”. In: 21.7 ().
- [147] Gurobi Optimization, LLC. *Gurobi Optimizer Reference Manual*. URL: <https://www.gurobi.com>.
- [148] Michael P Vittitow. “Structural Simulation Toolkit (SST)”. In: ().
- [149] Gengbin Zheng et al. “Periodic Hierarchical Load Balancing for Large Supercomputers”. In: *The International Journal of High Performance Computing Applications* 25.4 (). URL: <https://journals.sagepub.com/doi/10.1177/1094342010394383>.
- [150] Donald A. Kruse, Whit Schonbein, and Matthew G. F. Dosanjh. “A Statistical Analysis of HPC Network Tuning”. In: *Proceedings of the SC ’23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*. URL: <https://dl.acm.org/doi/10.1145/3624062.3624114>.
- [151] Robert Mijaković, Michael Firlbach, and Michael Gerndt. “An Architecture for Flexible Auto-Tuning: The Periscope Tuning Framework 2.0”. In: *2016 2nd International Conference on Green High Performance Computing (ICGHPC)*. URL: <https://ieeexplore.ieee.org/document/7508066/>.
- [152] Simone Pellegrini, Radu Prodan, and Thomas Fahringer. “Tuning MPI Runtime Parameter Setting for High Performance Computing”. In: *2012 IEEE International Conference on Cluster Computing Workshops*. URL: <https://ieeexplore.ieee.org/document/6355867/>.
- [153] Google. *Catapult - The Trace-Event File Format*. URL: <https://chromium.googlesource.com/catapult/+HEAD/docs/trace-event-format.md>.
- [154] David Boehme et al. “Caliper: Performance Introspection for HPC Software Stacks”. In: *SC ’16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. URL: <https://ieeexplore.ieee.org/abstract/document/7877125>.
- [155] *Apache Parquet*. Apache Parquet. URL: <https://parquet.apache.org/docs/overview/>.
- [156] Torsten Hoefer and Timo Schneider. “Optimization Principles for Collective Neighborhood Communications”. In: *2012 International Conference for High Performance Computing, Networking, Storage and Analysis*. URL: <http://ieeexplore.ieee.org/document/6468467/>.
- [157] Abhinav Bhatele, Stephanie Brink, and Todd Gamblin. “Hatchet: Pruning the Overgrowth in Parallel Profiles”. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. URL: <https://dl.acm.org/doi/10.1145/3295500.3356219>.
- [158] C.-Q. Yang and B.P. Miller. “Critical Path Analysis for the Execution of Parallel and Distributed Programs”. In: *[1988] Proceedings. The 8th International Conference on Distributed*. URL: <http://ieeexplore.ieee.org/document/12538/>.
- [159] Torsten Hoefer et al. “The Scalable Process Topology Interface of MPI 2.2”. In: *Concurrency and Computation: Practice and Experience* 23.4 (). URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/cpe.1643>.
- [160] S Mahdiah Ghazimirsaeed. “Efficient Designs for Distributed MPI Neighborhood Collectives”. In: ().
- [161] Harshitha Menon et al. “Automated Load Balancing Invocation Based on Application Characteristics”. In: *2012 IEEE International Conference on Cluster Computing*. URL: <https://ieeexplore.ieee.org/document/6337800/?arnumber=6337800>.
- [162] Andrew Newell et al. “RAS: Continuously Optimized Region-Wide Datacenter Resource Allocation”. In: *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. SOSP ’21. URL: <https://dl.acm.org/doi/10.1145/3477132.3483578>.
- [163] Leslie G. Valiant. “A Bridging Model for Parallel Computation”. In: *Commun. ACM* 33.8 (). URL: <https://dl.acm.org/doi/10.1145/79173.79181>.
- [164] Xinhao Kong et al. “Collie: Finding Performance Anomalies in RDMA Subsystems”. In: URL: <https://www.usenix.org/conference/nsdi22/presentation/kong>.

- [165] Zhibo Xuan et al. “Identifying Performance Inefficiencies of Parallel Program With Spatial and Temporal Trace Analysis”. In: *IEEE Transactions on Parallel and Distributed Systems* 36.7 (). URL: <https://ieeexplore.ieee.org/document/10982439/>.
- [166] Xin You et al. “GVARP: Detecting Performance Variance on Large-Scale Heterogeneous Systems”. In: *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. URL: <https://ieeexplore.ieee.org/document/10793232/>.
- [167] Abhinav Bhatele et al. “The Case of Performance Variability on Dragonfly-based Systems”. In: *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. URL: <https://ieeexplore.ieee.org/document/9139880>.
- [168] Ziji Chen et al. *Detecting Anomalies in Machine Learning Infrastructure via Hardware Telemetry*. arXiv: 2510.26008 [cs]. URL: <http://arxiv.org/abs/2510.26008>. Pre-published.
- [169] Yangtao Deng et al. “Minder: Faulty Machine Detection for Large-scale Distributed Model Training”. In: URL: <https://www.usenix.org/conference/nsdi25/presentation/deng>.
- [170] Jianbo Dong et al. “Evolution of Aegis: Fault Diagnosis for AI Model Training Service in Production”. In: URL: <https://www.usenix.org/conference/nsdi25/presentation/dong>.
- [171] Zhiyi Yao et al. “Holmes: Localizing Irregularities in LLM Training with Mega-scale GPU Clusters”. In: URL: <https://www.usenix.org/conference/nsdi25/presentation/yao>.
- [172] Tianyuan Wu et al. “GREYHOUND: Hunting Fail-Slows in Hybrid-Parallel Training at Scale”. In: URL: <https://www.usenix.org/conference/atc25/presentation/wu-tianyuan>.
- [173] Andreas Knüpfer et al. “Score-P: A Joint Performance Measurement Run-Time Infrastructure for Periscope, Scalasca, TAU, and Vampir”. In: *Tools for High Performance Computing 2011*. Ed. by Holger Brunst et al. URL: http://link.springer.com/10.1007/978-3-642-31476-6_7.
- [174] *Pytorch Kineto*. pytorch. URL: <https://github.com/pytorch/kineto>.
- [175] Izzet Yildirim et al. “IOMax: Maximizing Out-of-Core I/O Analysis Performance on HPC Systems”. In: *Proceedings of the SC ’23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*. SC-W ’23. URL: <https://dl.acm.org/doi/10.1145/3624062.3624191>.
- [176] Paul Barham et al. “Using Magpie for Request Extraction and Workload Modelling”. In: URL: <https://www.usenix.org/conference/osdi-04/using-magpie-request-extraction-and-workload-modelling>.
- [177] Rodrigo Fonseca et al. “{X-Trace}: A Pervasive Network Tracing Framework”. In: URL: <https://www.usenix.org/conference/nsdi-07/x-trace-pervasive-network-tracing-framework>.
- [178] Benjamin H. Sigelman et al. “Dapper, a Large-Scale Distributed Systems Tracing Infrastructure”. In: (). URL: <https://research.google.com/pubs/pub36356.html?ref=highscalability.com>.
- [179] Yu Guan et al. *PerfTracker: Online Performance Troubleshooting for Large-scale Model Training in Production*. Version 2. arXiv: 2506.08528 [cs]. URL: <http://arxiv.org/abs/2506.08528>. Pre-published.
- [180] Anonymous Authors. *ORCA: Observability with Real-time Control and Aggregation*.
- [181] Charity Majors. *Observability: What’s in a Name?* Honeycomb. URL: <https://www.honeycomb.io/blog/observability-whats-in-a-name>.
- [182] Majors, Charity. *Observability Is a Many-Splendored Definition*. charity.wtf. URL: <https://charity.wtf/2020/03/03/observability-is-a-many-splendored-thing/>.
- [183] *Observability Primer*. OpenTelemetry. URL: <https://opentelemetry.io/docs/concepts/observability-primer/>.
- [184] Cindy Sridharan. *Distributed Systems Observability*. URL: <https://www.oreilly.com/library/view/distributed-systems-observability/9781492033431/>.
- [185] *Facebookincubator/Dynolog*. Meta Incubator. URL: <https://github.com/facebookincubator/dynolog>.
- [186] *Facebookresearch/HolisticTraceAnalysis*. Meta Research. URL: <https://github.com/facebookresearch/HolisticTraceAnalysis>.

- [187] Yangtao Deng et al. “Mycroft: Tracing Dependencies in Collective Communication Towards Reliable LLM Training”. In: *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*. SOSP ’25. URL: <https://doi.org/10.1145/3731569.3764848>.
- [188] Christian R. Trott et al. “Kokkos 3: Programming Model Extensions for the Exascale Era”. In: *IEEE Transactions on Parallel and Distributed Systems* 33.4 (). URL: <https://ieeexplore.ieee.org/document/9485033>.
- [189] Abhinav Bhatele et al. *Pipit: Scripting the Analysis of Parallel Execution Traces*. Version 2. arXiv: 2306.11177 [cs]. URL: <http://arxiv.org/abs/2306.11177>. Pre-published.
- [190] Matthew Rocklin. “Dask: Parallel Computation with Blocked Algorithms and Task Scheduling”. In: URL: <https://doi.curvenote.com/10.25080/Majora-7b98e3ed-013>.
- [191] William F. Godoy et al. “ADIOS 2: The Adaptable Input Output System. A Framework for High-Performance Data Management”. In: *SoftwareX* 12 (). URL: [https://www.softxjournal.com/article/S2352-7110\(19\)30256-0/fulltext](https://www.softxjournal.com/article/S2352-7110(19)30256-0/fulltext).
- [192] P.C. Roth, D.C. Arnold, and B.P. Miller. “MRNet: A Software-Based Multicast/Reduction Network for Scalable Tools”. In: *SC ’03: Proceedings of the 2003 ACM/IEEE Conference on Supercomputing*. URL: <https://ieeexplore.ieee.org/abstract/document/1592924>.
- [193] William R. Williams et al. “Dyninst and MRNet: Foundational Infrastructure for Parallel Tools”. In: *Tools for High Performance Computing 2015*. Ed. by Andreas Knüpfer et al.
- [194] Paris Carbone et al. “Apache Flink™: Stream and Batch Processing in a Single Engine”. In: *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering* 36.4 ().
- [195] Jay Kreps, Neha Narkhede, Jun Rao, et al. “Kafka: A Distributed Messaging System for Log Processing”. In: *Proceedings of the NetDB*. Vol. 11. 2011.
- [196] *Catapult - The Trace-Event File Format*. URL: <https://chromium.googlesource.com/catapult/+HEAD/docs/trace-event-format.md>.
- [197] Matei Zaharia et al. “Spark: Cluster Computing with Working Sets”. In: *Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing*. HotCloud’10.
- [198] Paul Grun et al. “A Brief Introduction to the OpenFabrics Interfaces - A New Network API for Maximizing High Performance Application Efficiency”. In: *2015 IEEE 23rd Annual Symposium on High-Performance Interconnects*. URL: <https://ieeexplore.ieee.org/document/7312664>.
- [199] Pavel Shamis et al. “UCX: An Open Source Framework for HPC Network APIs and Beyond”. In: *2015 IEEE 23rd Annual Symposium on High-Performance Interconnects*. URL: <https://ieeexplore.ieee.org/document/7312665>.
- [200] Andrew R. Bernat and Barton P. Miller. “Anywhere, Any-Time Binary Instrumentation”. In: *Proceedings of the 10th ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools*. PASTE ’11. URL: <https://dl.acm.org/doi/10.1145/2024569.2024572>.
- [201] *PSM: Performance Scaled Messaging*. Intel® Corporation. URL: <https://github.com/intel/psm>.
- [202] *Polars*. URL: <https://www.pola.rs/>.
- [203] *Apache Iceberg - Apache Iceberg™*. URL: <https://iceberg.apache.org/>.
- [204] Michael Armbrust et al. “Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores”. In: *Proceedings of the VLDB Endowment* 13.12 (). URL: <https://dl.acm.org/doi/10.14778/3415478.3415560>.
- [205] *Claude Code Overview*. Claude Code Docs. URL: <https://code.claude.com/docs/en/overview>.
- [206] Úlfar Erlingsson et al. “Fay: Extensible Distributed Tracing from Kernels to Clusters”. In: *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*. URL: <https://dl.acm.org/doi/10.1145/2043556.2043585>.
- [207] Jessica Berg et al. “Snicket: Query-Driven Distributed Tracing”. In: *Proceedings of the Twentieth ACM Workshop on Hot Topics in Networks*. URL: <https://dl.acm.org/doi/10.1145/3484266.3487393>.
- [208] Milind Srivastava and Vyas Sekar. “Circa: Re-imagining Network Telemetry from an Approximation-First Perspective”. In: *Proceedings of the ACM SIGCOMM 2024 Conference: Posters and Demos*. ACM SIGCOMM Posters and Demos ’24. URL: <https://dl.acm.org/doi/10.1145/3672202.3673742>.

- [209] *Datafusion-Contrib/Arrow-Zarr*. datafusion-contrib. URL: <https://github.com/datafusion-contrib/arrow-zarr>.
- [210] Craig Ulmer et al. “Extending Composable Data Services into SmartNICs”. In: *2023 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. URL: <https://ieeexplore.ieee.org/document/10196516>.